



Towards Trustworthy Federated Learning via Cryptographic Design

Dissertation
submitted to
Sorbonne Université
in partial fulfillment of the requirements for the degree of
Doctor of Philosophy

Author:
Aftab Akram

To be defended on December 17th 2026, in front of a committee composed of:

<i>Reviewers</i>	Prof. Sébastien Canard – Télécom Paris, France Dr. Qiang Tang – LIST, Luxembourg
<i>Examiners</i>	Prof. Pascal Lafourcade – Université Clermont Auvergne, France Prof. Zekeriya Erkin – TU Delft, Netherlands Dr. Khawla Mallat – SAP, France
<i>Supervisor</i>	Prof. Melek Önen – EURECOM, France
<i>Co-supervisor</i>	Dr. Clémentine Gritti – CITI Lab, INSA Lyon, France
<i>Invited</i>	Dr. Ferran Alborch – EURECOM, France



Vers un apprentissage fédéré digne de confiance grâce à la conception cryptographique

Thèse
soumise à
l'Université Sorbonne
en vue de l'obtention du diplôme de
Docteur en Philosophie

Auteur :
Aftab Akram

À défendre le Décembre 17th 2026, devant un comité composé de :

To be defended on December 17th 2026, in front of a committee composed of:

<i>Reviewers</i>	Prof. Sébastien Canard – Télécom Paris, France Dr. Qiang Tang – LIST, Luxembourg
<i>Examiners</i>	Prof. Pascal Lafourcade – Université Clermont Auvergne, France Prof. Zekeriya Erkin – TU Delft, Netherlands Dr. Khawla Mallat – SAP, France
<i>Supervisor</i>	Prof. Melek Önen – EURECOM, France
<i>Co-supervisor</i>	Dr. Clémentine Gritti – CITI Lab, INSA Lyon, France
<i>Invited</i>	Dr. Ferran Alborch – EURECOM, France

Contents

1	Introduction	7
1.1	Federated Learning	7
1.1.1	Requirements for Trustworthy FL	10
1.2	Requirements of Trustworthy FL	13
1.3	Research Challenges	14
1.4	Contributions and Outline	17
1.5	List of Publications	18
2	FL Background and SA Building Blocks	21
2.1	Federated Learning	21
2.1.1	FL Training	22
2.1.2	FedAvg for Aggregation	23
2.2	Cryptographic Primitives	24
2.2.1	Pseudo-Random Generator	24
2.2.2	Key Agreement Scheme	25
2.2.3	Symmetric Encryption	27
2.2.4	Threshold Secret Sharing	28
2.3	Secure Aggregation	31
2.3.1	Security Requirements	32
2.3.2	Joye-Libert SA Scheme	33
2.3.3	Threshold Secure Aggregation	34
2.4	Differential Privacy	39
2.4.1	Differential Privacy Mechanisms	41
2.5	Membership Inference Attack	42
2.6	Notation	44
3	SAAFL: Secure Aggregation for Label-Aware Federated Learning	47
3.1	Introduction	48
3.2	Background	49
3.2.1	FedLA Algorithm	49

3.3	Towards Privacy-preserving FedLA	51
3.3.1	Threat model	51
3.3.2	Challenges when Integrating SA with FedLA	51
3.3.3	Our Solution - Overview	52
3.4	Related Work	53
3.5	TJL-A: A Label-Aware Threshold Secure Aggregation Scheme	54
3.5.1	Correctness	56
3.5.2	Security	57
3.6	SAAFL: Secure Aggregation for Label-Aware Federated Learning	58
3.6.1	Overview	58
3.6.2	Setup Phase	58
3.6.3	Training Phase	61
3.6.4	Security	63
3.7	Experimental Evaluation	63
3.7.1	Experimental Setup	64
3.7.2	Experimental Results	64
3.8	Conclusion	66
4	Privacy-preserving Federated Unlearning	67
4.1	Introduction	67
4.2	Related Work	69
4.3	Problem Setting	71
4.3.1	Federated Unlearning	71
4.3.2	Divisible Distribution	72
4.3.3	Design Goals and Threat Model	72
4.3.4	FedRecovery	73
4.3.5	FedRecovery+	75
4.3.6	A Round-Dependent Clipping Parameter	78
4.3.7	Final Design Considerations	81
4.4	Threshold Secure Aggregation with Forget Functionality	83
4.4.1	Our TSA Extension	84
4.4.2	Ideal Functionality of TSA with Forget Functionality	84
4.5	Privacy-Preserving Federated Unlearning	89
4.5.1	Our PPFU Construction	89
4.6	Experimental Evaluation	92
4.6.1	Experimental Setup	94
4.6.2	Unlearning Analysis	95
4.6.3	PPFU Performance	98
4.7	Conclusion and Future Work	101

5	Privacy-preserving and Fairness-aware Secure Aggregation in Federated Learning	103
5.1	Introduction	103
5.2	Related Work	105
5.3	Background	106
5.3.1	Fairness in Machine Learning	106
5.3.2	PrivFairFL-Pre	108
5.3.3	Symmetric Geometric Discrete Laplace Mechanism	109
5.4	Privacy-preserving Fairness Aware FL	109
5.4.1	Threat Model	110
5.4.2	Limitations	110
5.4.3	Our Approach	110
5.5	SaFer-FL	111
5.5.1	Setup Phase	112
5.5.2	Fairness-weight Calculation Phase	112
5.5.3	Training Phase	113
5.6	Experiments	115
5.6.1	Experimental Setting	115
5.6.2	Pre-processing of MovieLens-1M and COMPAS	115
5.6.3	Evaluation Metrics	116
5.6.4	Experimental Results	116
5.6.5	Complexity Analysis	117
5.7	Conclusion	117
6	Robust Blockchain-based Federated Learning	119
6.1	INTRODUCTION	119
6.1.1	Problem Statement	120
6.1.2	Our Contributions	121
6.2	Related Work	122
6.2.1	Multi-Party Computation (MPC)	122
6.2.2	Differential Privacy (DP)	122
6.2.3	Homomorphic Encryption (HE)	122
6.3	Background	123
6.3.1	Secret Network	123
6.3.2	Trusted Execution Environments (TEEs)	125
6.3.3	Robust Techniques	125
6.4	Robust SN-based FL Solution	127
6.4.1	Overview	127
6.4.2	SN-based System	128
6.5	Experimental Evaluation	130
6.5.1	Experimental Setting	130

6.5.2	Experimental Results	131
6.5.3	Conclusion	136
7	Conclusion and Future Work	139

List of Figures

1.1	Overview of the FL process.	9
2.1	Overview of the FL training process.	23
2.2	Membership Inference Attack.	43
3.1	Detailed description of the <i>Setup Phase</i> of SAAFL	60
3.2	Detailed description of the FL Round in SAAFL	62
3.3	Testing accuracy for FedLA, FedAvg, FedLA-FTSA, and SAAFL with $unique_c = 1$, $noniid_s = 0.7$	65
4.1	Figure explaining the workings of FedRecovery, taken from [1].	73
4.2	Computation of parameters C_k with parameters $\ell_1 = 3, \ell_2 =$ $4, \tau_{\max} = 100, \tau_{\text{flip}} = 40, C_{\text{tot}} = 5, C_{\min} = 0.0475$	79
4.3	Description of FedRecovery+	82
4.4	Ideal functionality $\mathcal{F}_{\text{TSA}}$ with Forget Functionality.	85
4.5	Description of the <i>Setup Phase</i> of PPFU	91
4.6	Description of the PPFU Online Phase.	93
4.7	Performance of MIA.	99
5.1	Detailed description of SaFer-FL algorithm.	114
6.1	Overview of our FL scenario.	129
6.2	Accuracy results for KRUM, Trim Mean, and Median under FOE attacks over the medical dataset.	133
6.3	Accuracy results for KRUM, Trim Mean, and Median under the four attacks over the medical dataset.	134
6.4	Accuracy results for KRUM, Trim Mean, and Median under the four attacks over the MNIST dataset.	135

List of Tables

2.1	Recurring notation used throughout the thesis.	45
3.1	Computation and Communication Cost of SAAFL with $n = 100$, $k_\tau = 0.8$, $f = 0.2$	66
4.1	Parameters for clipping used in evaluation.	94
4.2	Accuracy for models trained from scratch ($\tilde{w}$), unlearned with FedRecovery+ (w_u) and fine-tuned with extra learning rounds after unlearning (Retrain). Cells are colored in green when the model is more accurate than $\tilde{w}$	96
4.3	Comparison for FedRecovery and FedRecovery+.	98
4.4	Benchmarks for TSA with Forget functionality and PPFU Frameworks with parameter size of 421642.	100
5.1	Utility and fairness on MovieLens-1M and COMPAS for the federated baselines and our method (SaFer-FL), with and without differential privacy. Accuracy is in percent; ΔEODD and ΔEOP are fairness gaps (lower is fairer).	117
6.1	Communication and computational costs of Krum, Trim Mean and Median for the medical scenario. “BW” denotes bandwidth.	132
6.2	Performance comparison on the MNIST dataset with 79,510 model parameters. (* Such a value is suggested in [2] but is not explained.)	136

Abstract

Federated Learning (FL) enables multiple clients to collaboratively train a shared model while keeping their training data local. A central server distributes a global model to the clients, collects their model updates, and aggregates them to produce the next global model. FL faces several trustworthiness challenges: Model updates can leak information about clients' data, including through Membership Inference Attacks (MIAs), while client dropouts and heterogeneous data can affect aggregation. FL may also require federated unlearning, fairness, and robustness against malicious clients. This thesis addresses these challenges through cryptographic design, focusing primarily on Secure Aggregation (SA) and its execution to support these requirements while considering computational and communication costs.

More precisely, we first propose SAAFL, a SA framework that supports data heterogeneity, robustness against client dropouts and privacy protection. We then propose PPFU, a privacy-preserving federated unlearning solution that enables unlearning while maintaining privacy protection and dropout robustness. We further investigate fairness in FL and propose SaFer, a privacy-preserving and fairness-aware SA framework. Finally, we address adversarial robustness and aggregation integrity through a blockchain-based architecture and Trusted Execution Environment (TEE). The architecture enables robust aggregation methods to operate on protected client inputs while remaining robust against poisoning attacks.

In general, this thesis studies how cryptographic SA schemes can be adapted to address several FL trustworthiness requirements.

Résumé¹

L'apprentissage fédéré (Federated Learning, FL) permet à plusieurs clients d'entraîner conjointement un modèle partagé tout en conservant leurs données d'entraînement localement. Un serveur central distribue un modèle global aux clients, collecte leurs mises à jour de modèle et les agrège afin de produire le modèle global suivant.

Le FL fait face à plusieurs défis liés à la fiabilité du processus d'apprentissage : les mises à jour des modèles peuvent révéler des informations sur les données des clients, notamment au travers d'attaques par inférence d'appartenance (Membership Inference Attacks, MIAs), tandis que les déconnexions de clients et l'hétérogénéité des données peuvent affecter l'agrégation. Le FL peut également nécessiter la mise en œuvre de l'oubli fédéré (federated unlearning), de mécanismes d'équité et de robustesse face aux clients malveillants. Cette thèse répond à ces défis par une approche fondée sur la conception cryptographique, en se concentrant principalement sur l'agrégation sécurisée (Secure Aggregation, SA) et son adaptation pour répondre à ces exigences, tout en prenant en compte les coûts de calcul et de communication.

Plus précisément, nous proposons tout d'abord SAAFL, un framework d'agrégation sécurisée qui prend en charge l'hétérogénéité des données, la robustesse face aux déconnexions de clients et la protection de la vie privée. Nous proposons ensuite PPFU, une solution d'oubli fédéré préservant la vie privée, qui permet de supprimer la contribution d'un client tout en maintenant la protection de la vie privée et la robustesse face aux déconnexions. Nous étudions également l'équité dans le FL et proposons SaFer, un framework d'agrégation sécurisée préservant la vie privée et prenant en compte l'équité. Enfin, nous abordons la robustesse face aux comportements adverses et l'intégrité de l'agrégation au moyen d'une architecture fondée sur une blockchain et un environnement d'exécution de confiance (Trusted Execution Environment, TEE). Cette architecture permet à des méthodes d'agrégation robustes de fonctionner sur des entrées protégées des clients tout en restant robustes face aux attaques par empoisonnement.

¹This summary was translated with the help of `chagpt.com`

Dans l'ensemble, cette thèse étudie comment des schémas cryptographiques d'agrégation sécurisée peuvent être adaptés pour répondre à plusieurs exigences de fiabilité du processus d'apprentissage fédéré.

Acknowledgments

I would like to begin by expressing my profound gratitude to all those who have contributed to the completion of this PhD thesis. It is with great pleasure that I acknowledge the invaluable support, guidance, and encouragement I have received from numerous individuals throughout this journey.

I am immensely grateful to my supervisor, Prof. Melek Önen, for her invaluable guidance and support throughout my PhD. Her expertise, thoughtful feedback, and high standards have been instrumental in shaping this research and improving the quality of this thesis. I am particularly grateful for her continuous encouragement, patience, and trust throughout the different stages of this journey. Her guidance has not only helped me develop as a researcher but has also taught me the importance of rigor, clarity, and perseverance in scientific research. I am truly fortunate to have had the opportunity to work under her supervision and to learn from her throughout these years.

I am also deeply grateful to my co-supervisor, Dr. Clémentine Gritti, for her support, valuable insights, and guidance throughout this work. Her contributions and discussions have been greatly appreciated and have enriched my research journey.

I would like to express my sincere appreciation to the members of my thesis committee for accepting to review and evaluate this work. I am grateful to my reviewers, Prof. Sébastien Canard and Dr. Qiang Tang, for agreeing to review this thesis. I would also like to thank my examiners, Prof. Pascal Lafourcade, Prof. Zekeriya Erkin, and Dr. Khawla Mallat, for agreeing to serve on my thesis committee. I am equally grateful to Dr. Ferran Alborch for accepting the invitation to participate in my thesis defense. Beyond his role in the thesis committee, I am also grateful to Ferran for the opportunity to work together and for the many valuable discussions and exchanges we have had during our collaboration. His insights and collegial support have been greatly appreciated.

I am deeply thankful to EURECOM for providing an inspiring academic environment and the resources and facilities necessary to conduct this re-

search. The interactions with researchers, colleagues, and members of the institute have contributed greatly to my academic and professional development.

I offer my deepest gratitude to my family for their exceptional support throughout this journey. I am profoundly grateful to my father and mother, whose unconditional love, sacrifices, prayers, and unwavering belief in me have been a constant source of strength throughout my life. Their encouragement has given me the confidence to pursue my goals and overcome the challenges along the way. I would not have reached this point without the values they instilled in me and the countless sacrifices they made for my future.

I would like to express my deepest gratitude to my wife, Ramisha Aftab, for her unwavering love, patience, understanding, and support throughout the long and demanding years of my PhD. Her presence and encouragement gave me strength during the most challenging moments of this journey, and I am deeply grateful for all the sacrifices she made along the way. I am also immensely grateful to my son, Muhammad Ayyan Aftab, who, at just one year old, has brought immense joy and happiness into our lives. His smile and presence have been a constant source of warmth and motivation, making this journey all the more meaningful.

To my parents, my wife, and my son, this achievement is as much yours as it is mine. Your love and support have been the foundation upon which I have built this journey. I dedicate this achievement to you with all my heart.

Chapter 1

Introduction

1.1 Federated Learning

The rapid advancement of artificial intelligence (AI) and machine learning (ML) has led to significant progress across many application areas. ML models are now used in healthcare [3, 4, 5], finance [6, 7, 8], retail [9], and mobile applications [10, 11] for tasks such as disease detection, fraud detection, demand forecasting, and speech recognition. These advances have been driven by improvements in ML algorithms, computing infrastructure, and data availability making AI increasingly capable and widely adopted. The 2026 AI Index reports that 88% of surveyed organizations used AI in at least one business function in 2025, alongside continued growth in AI model development, computing capacity, and investment¹.

As ML applications grow, they increasingly require data from multiple users, locations, and institutions. ML models learn patterns from the data used for training, and data from only one source may not cover all the cases that the model will encounter in practice. For example, a medical model trained using data from one hospital may not work equally well for patients from other hospitals because the patients, medical devices, and clinical practices may be different. Using data from multiple hospitals can provide the model with a wider range of datasets and help it perform well on data from hospitals that were not included in the original training data. Therefore, access to data from multiple sources is important for developing ML models that can work well in different environments.

In order to train ML model using data from multiple parties, these parties may need to transfer their data to a central repository for training. Although this makes the data available for joint training, bringing usually sensitive data

¹<https://hai.stanford.edu/ai-index/2026-ai-index-report>.

together in one place creates several challenges:

- Firstly, centralizing data results in a **single point of failure**, since a incident in the central repository can affect data collected from multiple sources. For example, if a centralized healthcare system becomes unavailable, patient data from multiple hospitals may be inaccessible.
- Secondly, centralization increases **privacy risks**, as sensitive information from multiple sources is stored in one place. For example, a centralized medical database may contain patients’ health records, diagnoses, and treatment information. Unauthorized access or disclosure of such information can lead to a privacy breach.
- Finally, the **General Data Protection Regulation² (GDPR)** sets requirements for how personal data is collected and processed. It requires personal data to be processed lawfully, fairly, and transparently. It also requires principles such as purpose limitation and data minimization to be followed.

These limitations make centralized collection and processing of sensitive data challenging and hence motivated the development of collaborative learning approaches, which allow multiple data owners to jointly train ML models using their local datasets without centralizing the data. Instead of transferring data to a common location, each participant keeps its data locally and contributes to the learning process through the sharing of information derived from the data.

Federated Learning (FL) [12] is one of the prominent approaches of collaborative learning, enabling multiple clients to jointly train a shared ML model while keeping their data locally. In a typical FL setting, a central FL server sends an initial model to the participating FL clients, as illustrated in Figure 1.1 . Each client trains the model locally using its own dataset and only sends the resulting model update to the server. The server then combines these updates to create a new global model and sends it back to the clients. This process is repeated over several rounds, allowing the model to learn from datasets across different clients.

For example, Walmart uses FL for forecasting product demand in its stores [9]. Each store collects its own sales data, such as the number of products sold over time. These data can be used to predict future product demand and help stores plan their inventory. In an FL setting, a central Walmart FL server first sends an initial demand-forecasting model to participating stores.

²<https://eur-lex.europa.eu/eli/reg/2016/679/oj>.

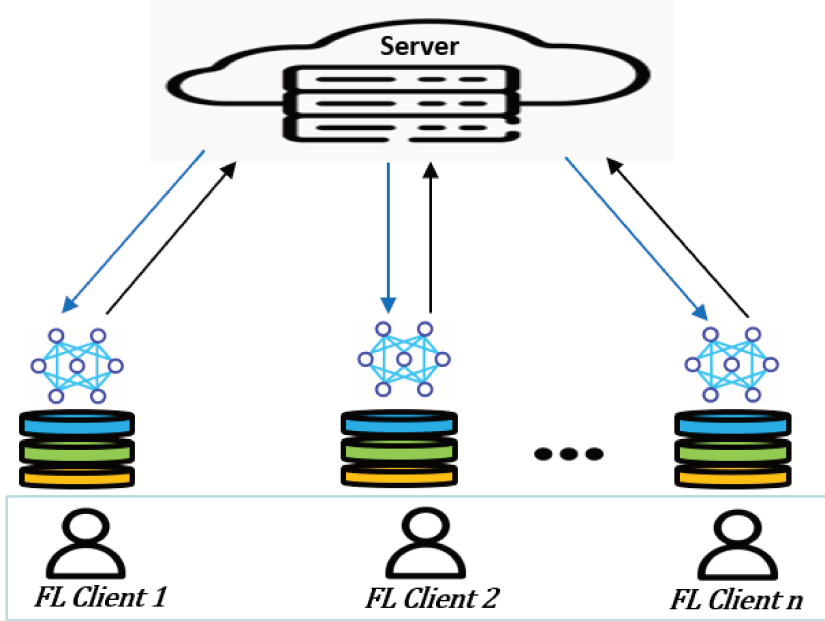


Figure 1.1: Overview of the FL process.

Each store trains the model locally using its own sales data and then sends the resulting model update to the central FL server. The server aggregates the updates from all participating stores to create global model. The FL server then sends global model back to the stores for another round of local training. This process is repeated over multiple rounds, allowing the global model to learn from sales data across different stores without requiring their local data to be shared with the central FL server.

Thanks to its collaborative nature, FL can also help address privacy and regulatory concerns beyond avoiding a single point of failure. In particular, clients can keep their data locally and share only the information needed for model training, limiting the direct exposure of sensitive data to the server. This decentralized approach can also facilitate compliance with data protection requirements by allowing clients to retain greater control over their data.

Although FL addresses the concerns of data centralization, keeping training data locally does not completely guarantee a trustworthy learning process. Clients still send model updates to the server, which may reveal information about their local data. For example, MIA (Membership inference attack) [13] can determine whether a specific sample was used for training or not. Moreover, independently from its collaborative nature, FL introduces other challenges, e.g., participating clients may have different data, capabilities,

and levels of trust. The next section discusses these challenges in more details.

1.1.1 Requirements for Trustworthy FL

One of the main motivations behind FL is to enable collaborative learning while reducing the privacy risks associated with sharing data. By keeping training data at participating clients, FL avoids the need for transferring sensitive datasets to a central repository. However, keeping data local is only one part of building a trustworthy FL system. The learning process still requires communication and coordination among participating clients and server, which introduces new challenges.

Privacy still remains a challenge in FL because clients send model updates to the FL server during training. Although these updates do not contain the original training data, they are generated from local data and may still reveal information about them. One example is MIA [13], in which an attacker tries to determine whether a specific sample was used to train the model or not. The attacker observes how the model responds to samples that were used during training and samples that were not used. For example, the model may give higher confidence or lower prediction error for samples it has seen during training. The attacker then gives a target sample to the model and checks whether its response is similar to the responses of the training samples. If it is yes, the attacker may infer that the target sample was also used during training. Therefore, additional mechanisms are needed to protect information that may be revealed through model updates.

FL must also account for **Client Dropouts**. In practical FL settings, clients may become unavailable because of network failures, limited computational resources, device disconnections, or other unexpected events. Such dropouts pose challenges for privacy-preserving FL mechanisms that rely on the participation of clients and information associated with their contributions. The absence of clients can change the set of available updates and negatively affect model performance and convergence. More importantly, in privacy-preserving aggregation, a dropout may prevent the protocol from recovering the information required to reconstruct the aggregate, such as masked updates or secret shares. If the required information cannot be recovered from the remaining clients, the server may be unable to compute the aggregate for that round. Therefore, a trustworthy FL system must tolerate client dropouts.

Another challenge is **Data Heterogeneity**. Most FL systems use Federated Averaging (FedAvg) [12] as the aggregation mechanism. In this approach, the server computes a simple average of the clients' updates to ob-

tain the global model. FedAvg assumes that client data are independent and identically distributed (IID). Nevertheless, in practice, clients often have different amounts and distributions of data, resulting in heterogeneous (non-IID) data. Because clients learn from different data distributions, their local model updates can differ significantly. When these updates are averaged, underrepresented clients may have less influence on the global model, which may prevent their local information from being adequately reflected in the resulting model. Addressing such heterogeneity may require the server to assign different weights to client updates based on local dataset information, such as the distribution of labels. Therefore, a trustworthy FL system must support data heterogeneity without any implication on the privacy of data.

The **Unlearning** is another requirement for trustworthy FL. The GDPR grants individuals the **right to be forgotten**, allowing them to request the deletion of their personal data under certain conditions [14]. In FL, a client’s data contributes to model training over multiple rounds, and its influence becomes embedded in the global model through its model updates. If a client subsequently requests that its data be forgotten, its influence should be removed from the trained model. **Federated unlearning** (FU) [1] provides a mechanism to address this requirement by removing the influence of a client’s data from the trained model. Ideally, the resulting model should behave as if the client’s data had never been used for training. However, this is challenging because the client’s updates are combined with those of other clients across multiple rounds, making its contribution difficult to isolate after training. Privacy-preserving mechanisms can make this separation even more challenging by hiding individual client contributions. Therefore, a trustworthy FL system must support FU while preserving the privacy of the remaining clients.

Fairness is also an important concern in FL: Clients may have different data distributions and levels of contribution. For example, when using the COMPAS [15] dataset, clients may have different distributions of individuals across sensitive attributes, such as gender, and outcome labels, such as whether an individual is classified as a high or low risk of recidivism. Such unbalanced distributions can cause the global model to perform differently across demographic groups. Identifying and addressing these disparities may require client-specific information about the distribution of sensitive attributes and labels. However, accessing or revealing such information can compromise the privacy of client datasets. Therefore, a trustworthy FL system should improve fairness across different groups while preserving the privacy of client-specific data. These differences in client data distributions also reflect data heterogeneity in FL.

Adversarial behavior presents another important challenge in FL where

the participating clients may be malicious or compromised. Such clients can manipulate their local model updates before sending them to the server. For example, a malicious client may scale its model update by a large factor so that its contribution has a stronger effect on the global model. This can reduce the global model’s accuracy or cause it to perform incorrect predictions. Identifying malicious contributions generally requires analyzing the model updates submitted by individual clients, while privacy protection aims to prevent such model updates from being exposed. This creates a fundamental tension between privacy and adversarial robustness. Therefore, a trustworthy FL system should preserve client privacy while enabling effective detection and mitigation of malicious contributions.

Beyond the aforementioned requirements, **Transparency, Explainability, and Verifiability** are also important for trustworthy FL. **Transparency** [16] concerns understanding and verifying how an FL system operates and how decisions are made throughout the learning process. This includes information about client participation, training procedures, model updates, aggregation, and the rules governing these operations. A trustworthy FL system should provide sufficient information about these processes to allow the resulting global model and its training procedure to be understood. Building on transparency, **Explainability** [17] helps users understand why an FL model produces a particular prediction or decision. This is particularly relevant when FL models are used in sensitive domains such as healthcare and finance, where users may need to understand the reasons behind model decisions. A trustworthy FL system should provide meaningful explanations without requiring access to sensitive client data. However, generating detailed explanations may require information about client data, which can reveal sensitive information and conflict with privacy requirements. Therefore, trustworthy FL should provide meaningful explanations while limiting the disclosure of information that could compromise client privacy. Finally, **Verifiability** [18] complements these properties by enabling the correctness of an FL system and its results to be checked throughout the learning process. This includes verifying client-side training, server-side processing, aggregation, and global model generation. A trustworthy FL system should provide sufficient evidence to independently verify the correctness of these operations.

Taken together, these requirements define the key properties of a trustworthy FL system. However, addressing them simultaneously is challenging because improving one requirement may affect another, leading to inherent trade-offs. For example, stronger privacy protection can limit the information available for detecting malicious clients or assessing fairness. Therefore, designing a trustworthy FL system requires carefully balancing these requirements and managing the resulting trade-offs while maintaining the

effectiveness of the overall learning process.

This thesis focuses on a selected set of these requirements and investigates how to achieve them while preserving the privacy of client data in FL. The following section identifies the requirements considered in this thesis and defines the scope of the research problems addressed in the subsequent chapters.

1.2 Requirements of Trustworthy FL

Based on the scope defined in the previous section, this section specifies the requirements used to characterize a trustworthy FL system. These requirements cover different aspects of the learning process, from protecting client information to ensuring reliable participation, robust model training, and maintaining model performance. Each requirement represents a property that the system should satisfy to support trustworthy FL. The requirements are defined individually to clarify their objectives and provide the basis for evaluating the mechanisms developed in this thesis.

- **Privacy (R1):** Privacy is a fundamental requirement of trustworthy FL. A trustworthy system should protect the sensitive information of the clients throughout the learning process, including their datasets and derived information, such as model updates.
- **Dropout Robustness (R2):** In FL, clients may become unavailable during training due to communication failures, limited computational resources, or other unexpected factors. A trustworthy FL system should remain functional when some clients become unavailable and should be able to continue the learning process using the available clients. In privacy-preserving FL, it should additionally ensure that client dropouts do not compromise the privacy or correctness of the aggregation.
- **Data Heterogeneity (R3):** In practical FL scenarios, client datasets often differ in their distributions, sizes, and characteristics. This heterogeneity can affect the training process, convergence, and model performance. A trustworthy FL system should be able to learn effectively from diverse client datasets and maintain consistent model performance across different data distributions.
- **Legal Compliance (R4):** A trustworthy FL system should comply with the GDPR **right to be forgotten**. It should allow a client

to request the removal of the influence of its data from the trained model. Federated unlearning (FU) should remove the client’s contribution without affecting the contributions of other clients. The resulting model should maintain its accuracy and performance after unlearning.

- **Fairness (R5):** Fairness is also an essential requirement for trustworthy FL, as clients may have different data distributions, quantities, and representation. Biased or imbalanced data can cause the global model to perform better for some groups while providing poor results for others. A trustworthy system should reduce such biases and ensure fair outcomes across different groups.
- **Adversarial Robustness (R6):** Adversarial robustness is essential for protecting FL against malicious clients. Such clients may manipulate their local updates to reduce model performance or introduce targeted behavior. A trustworthy FL system should limit the impact of these malicious contributions and prevent them from compromising the global model.
- **System Performance (R7):** A trustworthy FL system should maintain good model performance while addressing privacy and other requirements. The mechanisms used to protect client data or support trustworthy learning should not significantly reduce the accuracy or effectiveness of the global model. Therefore, the system should provide the required protections while maintaining good model performance.

1.3 Research Challenges

As discussed earlier, keeping client data local does not, by itself, ensure privacy in FL. One approach to address the privacy concern in FL is **Secure Aggregation (SA)**. SA [19, 20, 21] is a protocol involving multiple clients and an aggregator (server), where each client holds a private input and the aggregator computes an aggregate, such as the sum of these inputs, without learning the individual values. SA can be constructed using various cryptographic techniques, including homomorphic encryption (HE) [22, 23, 24], secure multiparty computation (MPC) [25, 26, 27], and masking [28, 29]. Other privacy-preserving mechanisms, such as differential privacy (DP) [30] and trusted execution environments (TEEs) [31], can also protect client information and may be combined with or used as alternatives to SA. This thesis focuses on cryptographic approaches to SA, which are explored in the following chapters.

SA was not originally designed specifically for FL, but it can be adapted to protect client model updates. In FL, client updates represent the private inputs, while the server acts as the aggregator. The server obtains the aggregated update only without accessing individual client updates, making SA an important mechanism for addressing Privacy (R1). However, client dropouts must also be considered because some clients may become unavailable during training. This is achieved by distributing the information required for aggregation among the participating clients so that the aggregate can still be reconstructed despite some client dropouts (R2). On the other hand, hiding individual client updates also limits the information available to the server, which can make it difficult to address other requirements, including Data Heterogeneity (R3), Legal Compliance (R4), Fairness (R5), and Adversarial Robustness (R6). The difficulty of simultaneously preserving client private updates and supporting these additional requirements forms the basis for the research challenges considered in this thesis. The central question is how cryptographic SA can be extended to support the broader requirements of trustworthy FL while maintaining privacy, model performance, and practical efficiency. Based on these interactions, four research challenges are identified and selected:

- **C1: Privacy, Dropout Robustness, and Data Heterogeneity (R1 + R2 + R3).** In practical FL, client datasets are often heterogeneous, with differences in data distributions and sizes. Handling such heterogeneity may require the aggregation process to use client-specific information, such as the label distributions, to determine weights for the corresponding client updates. These weights are then used to scale the client updates before they are aggregated. However, cryptographic SA is generally designed to securely compute the sum of client inputs, making such client-specific computations difficult to perform while keeping individual updates private. Therefore, additional mechanisms are needed to incorporate client-specific weights into the SA process. Moreover, when some clients drop out, the client-specific information required for the aggregation may also become unavailable, making it difficult to correctly compute the aggregation. Therefore, the challenge is to design a privacy-preserving aggregation mechanism that can handle heterogeneous client data and remain robust to client dropouts, while maintaining model performance and reasonable computational and communication costs.
- **C2: Privacy, Dropout Robustness, and Legal Compliance (R1 + R2 + R4).** To comply with the legal requirement of the right

to be forgotten, FU requires the influence of a client’s updates to be removed from the global model following the client’s request to leave the federation. In conventional FL, the server can access individual client updates and can therefore identify the update of the client requesting unlearning. However, SA reveals the aggregated result only and keeps individual updates hidden. Consequently, the server cannot directly identify and remove the contribution of a specific client after aggregation. This creates the challenge of removing the influence of the unlearning client during privacy-preserving FL. To perform this removal, the server may need the participation of the remaining clients to compute the required aggregated result without the unlearning client’s contribution. If some of these clients also become unavailable during unlearning, their required contributions may be missing, making it difficult to complete the unlearning process. The challenge is therefore to perform privacy-preserving unlearning despite client unavailability while preserving the performance of the resulting model and keeping the additional computational and communication costs reasonable.

- **C3: Privacy and Fairness (R1 + R5).** Fairness in FL may require the aggregation process to consider information about client data or group representation, such as the distribution of different demographic groups (e.g., gender) across clients. Such information can be used to determine fairness-aware weights for the corresponding client updates before aggregation. SA is generally designed to securely compute the sum of client inputs, making it difficult to perform these client-specific computations while keeping individual updates private. Therefore, additional mechanisms are needed to incorporate fairness-aware weights into the SA process. Therefore, the challenge is to design a privacy-preserving aggregation mechanism that can support fairness-aware weighting while maintaining model accuracy and reasonable computational and communication costs.
- **C4: Privacy and Adversarial Robustness (R1 + R6).** In FL, malicious clients may manipulate their local model updates to influence the global model or reduce its performance. Detecting and limiting such malicious contributions may require the server to inspect individual client updates. However, cryptographic SA hides individual updates from the server and reveals only the aggregated result. As a result, the server cannot directly examine individual contributions to identify or filter malicious updates before aggregation. The challenge is therefore to provide robustness against malicious clients while preserving the

privacy of client’s model updates. The aggregation process should also remain correct and maintain model performance without introducing excessive computational and communication costs.

1.4 Contributions and Outline

This thesis investigates how cryptographic design can be used to strengthen the trustworthiness of FL. While SA provides an important foundation for protecting the confidentiality of client updates, it does not by itself address the broader requirements of trustworthy FL. This thesis therefore extends SA-based approaches to address additional challenges arising in practical FL deployments, including client dropouts, data heterogeneity, the right to be forgotten, fairness, adversarial robustness, and efficiency. The proposed cryptographic SA mechanisms are designed to support these requirements while preserving the confidentiality of client contributions and maintaining the utility of the resulting global model.

Chapter 2 presents the technical foundations required throughout the thesis. It provides an overview of FL, including the training process and the Federated Averaging (FedAvg) [12] aggregation algorithm. It then introduces the cryptographic primitives used in the proposed protocols, including pseudo-random generators, key agreement schemes, symmetric encryption, and threshold secret sharing. The chapter subsequently presents Secure Aggregation, its security requirements, and threshold SA. Finally, it introduces the notions of Differential Privacy (DP) and its mechanisms such as the Laplace and Gaussian mechanisms .

Chapter 3 addresses **C1: Privacy, Dropout Robustness, and Data Heterogeneity (R1 + R2 + R3)**. For this purpose, we develop a privacy-preserving label-aware aggregation protocol that enables the server to compute client-specific aggregation weights based on local label distributions without learning these distributions. We combine label-aware weighting with a threshold SA protocol to securely compute the weighted model update. The protocol further uses secret sharing to reconstruct the required secrets in the presence of client dropouts, allowing the aggregation to proceed without revealing individual client updates.

Chapter 4 addresses **C2: Privacy, Dropout Robustness, and Legal Compliance (R1 + R2 + R4)**. For this purpose, we develop a privacy-preserving FU protocol that combines an improved FU approach, named FedRecovery [1], with an extension of SA incorporating Forgotten functionality. The proposed protocol removes the contribution of a target client from the trained global model without exposing individual client updates to the

server. The extended SA protocol also supports client dropouts during the unlearning process, allowing the target client's contribution to be removed while preserving the confidentiality of the updates of the remaining clients.

Chapter 5 addresses **C3: Privacy and Fairness (R1 + R5)**. We develop a privacy-preserving and fairness-aware aggregation protocol that extends SA to incorporate client-specific fairness information, such as gender-label distributions, without exposing sensitive client information. The protocol enables the computation of fairness-aware aggregation weights over protected client statistics and securely aggregates the corresponding model updates, thereby incorporating fairness considerations into the global model while preserving the confidentiality of client contributions.

Chapter 6 addresses the challenge of achieving privacy and adversarial robustness in FL (**C4: R1 + R6**). We use a blockchain-based architecture with TEEs to protect client updates during aggregation. Client updates are processed within TEEs, where robust aggregation techniques are executed through smart contracts to identify and limit the influence of adversarial updates. The blockchain consensus mechanism further provides integrity for the aggregation process by maintaining a verifiable and distributed record of the computation.

Finally, Chapter 7 concludes the thesis by summarizing the main contributions and findings and identifying directions for future research toward more secure, reliable, efficient, and trustworthy FL systems.

1.5 List of Publications

The research presented in this thesis has led to the following publications:

- Akram, Aftab, Clémentine Gritti, Mohd Hazali, Mohamed Halip, Diyana Kamarudin, Marini Mansor, Syarifah Bahiyah Rahayu, and Melek Önen. "Robust Blockchain-based Federated Learning." *11th International Conference on Information Systems Security and Privacy*, 2025.
- Akram, Aftab, Harry NH Pham, Melek Önen, and Clémentine Gritti. "SAAFL: Secure Aggregation for Label-Aware Federated Learning." in *IFIP International Conference on ICT Systems Security and Privacy Protection*, pp. 32-46. Cham: Springer Nature Switzerland, 2025.
- El Bacha, Tonia, Aftab Akram, Ibtissam Harrouche, and Melek Önen. "Privacy-Preserving and Fairness-Aware Secure Aggregation in Federated Learning." *Privacy in Statistical Databases Conference*, 2026.

- Akram, Aftab, Ferran Alborch, Clémentine Gritti, and Melek Önen.
"Privacy-Preserving Federated Unlearning." *Under submission*.

Chapter 2

FL Background and SA Building Blocks

This chapter introduces the technical concepts required for the remainder of the thesis. We first present Federated Learning (FL), including its architecture, training process, and FedAvg. We then introduce the main cryptographic primitives used in this thesis. These primitives are subsequently used to introduce Secure Aggregation (SA). Finally, we discuss Differential Privacy (DP) and Membership Inference Attacks (MIA) as complementary concepts for privacy protection and privacy evaluation in FL respectively. Together, these concepts provide the foundation for the privacy-preserving and trustworthy FL solutions developed in the subsequent chapters.

2.1 Federated Learning

As introduced in Chapter 1, FL [12] is a collaborative learning framework that enables multiple clients to jointly train a shared global model while keeping their private dataset local. The learning process is coordinated by a FL server, who initializes the global model and distributes it to the participating clients. Each FL client then trains the received model locally, using its private dataset, without sharing the data with the server. After local training, the clients send their trained model updates or gradients to the server. A gradient represents the direction and magnitude by which the model parameters should be adjusted to reduce the local training loss, while a model update generally refers to the resulting change in the model parameters after one or more local training steps. The server aggregates these contributions to obtain an updated global model, which is then distributed back to the clients for the next round of local training. This process is repeated over multiple

communication rounds, allowing the clients to collaboratively improve the global model while keeping their data local.

2.1.1 FL Training

Let w_τ denote the global model at communication round τ . Let $\mathcal{U}$ denote the set of n FL clients. Each client $u \in \mathcal{U}$ owns a private dataset D_u , and let $w_{u,\tau}$ denote the local model obtained by client u after local training in round τ . The initial global model is denoted by w_0 . Let $f_u(w)$ denote the local loss function of client u evaluated on its private dataset D_u .

At the beginning of the FL training process, the FL server initializes the global model w_0 . In each communication round $\tau > 0$, the server selects a subset of clients $\mathcal{U}_\tau \subseteq \mathcal{U}$ to participate in that round and sends the current global model $w_{\tau-1}$ to the selected clients. Thus, the set of participating clients may vary across communication rounds. Each selected client $u \in \mathcal{U}^\tau$ uses $w_{\tau-1}$ as the starting point for local training and optimizes the model using its private dataset D_u .

After local training, a participating client can communicate either its computed gradient or its resulting local model update to the server. In the gradient-based formulation, client u computes the local gradient $\nabla f_u(w_{\tau-1})$, which represents the direction and magnitude of the change in the model parameters required to reduce its local training loss. In the model-update formulation, the client instead sends its locally trained model $w_{u,\tau}$ as shown in Figure 2.1 or equivalently the corresponding model update $\Delta w_{u,\tau} = w_{u,\tau} - w_{\tau-1}$. The server then aggregates the contributions received from the participating clients to construct the updated global model w_τ . This sequence of global model distribution, local training, client-to-server communication, and server-side aggregation is repeated over multiple communication rounds.

To formalize the gradient-based formulation, each client u computes a local gradient $\nabla f_u(w_{\tau-1})$ using its private dataset D_u . The server can update the global model by aggregating the local gradients as

$$w_\tau = w_{\tau-1} - \frac{\eta}{n} \sum_{u \in \mathcal{U}_\tau} \nabla f_u(w_{\tau-1}), \quad (2.1)$$

where η is the learning rate used during training. In this formulation, the clients communicate gradients, and the server applies the aggregated gradient to the current global model.

Alternatively, in the model-update formulation, each client performs local training step starting from $w_{\tau-1}$ and obtains a local model $w_{u,\tau}$. The

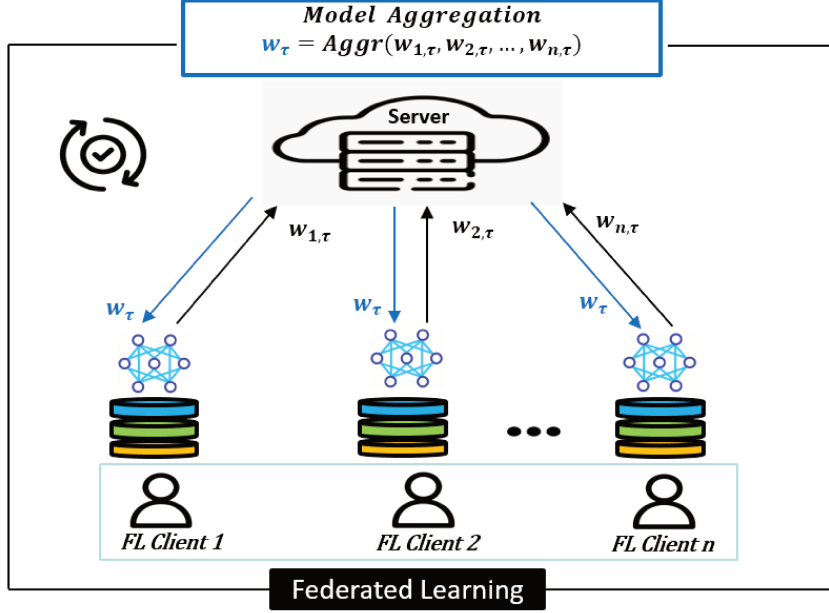


Figure 2.1: Overview of the FL training process.

corresponding model update is defined as

$$\Delta w_{u,\tau} = w_{u,\tau} - w_{\tau-1}. \quad (2.2)$$

The server can then aggregate these local model updates according to

$$w_\tau = w_{\tau-1} + \frac{1}{n} \sum_{u \in \mathcal{U}_\tau} \Delta w_{u,\tau}. \quad (2.3)$$

Equivalently, the server may directly aggregate the local models as

$$w_\tau = \frac{1}{n} \sum_{u \in \mathcal{U}_\tau} w_{u,\tau}. \quad (2.4)$$

2.1.2 FedAvg for Aggregation

In practice, FL algorithms commonly allow each FL client to perform multiple local optimization steps (called local epoch) before communicating its updated model to the server. Consequently, clients may produce different local models $w_{u,\tau}$, which the FL server then aggregates to obtain the global model. A widely used aggregation algorithm for this purpose is FedAvg [12]. Rather than assigning equal weight to every client, FedAvg typically weights

each local model according to the number of training samples available at the corresponding client. Let $d_u = |D_u|$ denote the number of training samples at client u . The global model is then computed as

$$w_\tau = \sum_{u \in \mathcal{U}_\tau} \frac{d_u}{\sum_{v \in \mathcal{U}_\tau} d_v} w_{u,\tau} \quad (2.5)$$

where $w_{u,\tau}$ is the local model obtained by client u after local training. Consequently, clients with larger local datasets have a greater contribution to the global model. If all clients have the same number of training samples, the weights are equal and FedAvg reduces to a simple average of the local models.

2.2 Cryptographic Primitives

In this section, we present the main cryptographic schemes used throughout this thesis, particularly those employed to develop SA solutions for privacy-preserving FL.

2.2.1 Pseudo-Random Generator

A *pseudo-random generator* (PRG) is a deterministic algorithm that expands a short, uniformly random seed into a longer sequence that is computationally indistinguishable from a uniformly random sequence.

Definition 2.2.1 (Pseudo-Random Generator). $\text{PRG}(m, R, b) \rightarrow B$: A pseudo-random generator that expands a seed $b \in \{0, 1\}^\lambda$ into a vector $B \in \mathbb{Z}_R^m$, where λ is the security parameter, m is the number of elements in the output vector, R is the modulus, and $\mathbb{Z}_R$ denotes the ring of integers modulo R . The output vector B contains m elements from $\mathbb{Z}_R$.

2.2.1.1 PRG Security

Let $\mathcal{D}_1$ be the distribution of $A \leftarrow \mathbb{Z}_R^m$ and $\mathcal{D}_2$ the distribution of $B \leftarrow \text{PRG}(m, R, b)$, where $b \leftarrow 0, 1^\lambda$. The PRG is secure if, for a sufficiently large security parameter λ ,

$$\mathcal{D}_1 \approx_c \mathcal{D}_2,$$

where $\approx_c$ denotes computational indistinguishability.

2.2.2 Key Agreement Scheme

A *Key Agreement* (KA) scheme is a cryptographic protocol that enables two parties to establish a common secret key over a public communication channel.

Definition 2.2.2 (Key Agreement Scheme). *A key agreement scheme is parameterized by a security parameter λ and a key space $\mathcal{K}$. It consists of the following algorithms:*

- $\text{KA.Param}(1^\lambda) \rightarrow pp$: *Given the security parameter λ , this algorithm generates the public parameters pp of the scheme.*
- $\text{KA.Gen}(pp) \rightarrow (pk, sk)$: *Given the public parameters pp , this algorithm generates a public-private key pair (pk, sk) .*
- $\text{KA.Agree}(pp, sk, pk_1, pk_2, G) \rightarrow k$: *Given pp , a private key sk , public keys pk_1, pk_2 , and a hash function G , this algorithm derives a shared secret key $k \in \mathcal{K}$ as the common secret established between the two parties.*

2.2.2.1 Correctness

A key agreement scheme is correct if and only if the two parties participating in the protocol derive the same shared key, except with negligible probability. Formally, for every security parameter λ ,

$$\Pr \left[\begin{array}{l} \text{KA.Param}(1^\lambda) \rightarrow pp, \\ \text{KA.Gen}(pp) \rightarrow (pk_1, sk_1), \\ \text{KA.Gen}(pp) \rightarrow (pk_2, sk_2), \\ \text{KA.Agree}(pp, sk_1, pk_1, pk_2, G) \rightarrow k_{1,2}, \\ \text{KA.Agree}(pp, sk_2, pk_2, pk_1, G) \rightarrow k_{2,1} \end{array} : k_{1,2} \neq k_{2,1} \right] \leq \text{negl}(\lambda).$$

Equivalently,

$$\Pr[k_{1,2} = k_{2,1}] \geq 1 - \text{negl}(\lambda).$$

2.2.2.2 Security

A key agreement scheme is secure if the shared key established by the protocol is computationally indistinguishable from a uniformly random key in the key space $\mathcal{K}$, given the public keys of the two parties. Formally, let the following two distributions be defined as

$$\mathcal{D}_1 = \left\{ (pk_1, pk_2, r) \left| \begin{array}{l} \text{KA.Param}(1^\lambda) \rightarrow pp:, \\ \text{KA.Gen}(pp) \rightarrow (pk_1, sk_1):, \\ \text{KA.Gen}(pp) \rightarrow (pk_2, sk_2):, \\ r \leftarrow_R \mathcal{K} \end{array} \right. \right\},$$

and

$$\mathcal{D}_2 = \left\{ (pk_1, pk_2, k) \left| \begin{array}{l} \text{KA.Param}(1^\lambda) \rightarrow pp:, \\ \text{KA.Gen}(pp) \rightarrow (pk_1, sk_1):, \\ \text{KA.Gen}(pp) \rightarrow (pk_2, sk_2):, \\ \text{KA.Agree}(pp, sk_1, pk_1, pk_2, G) \rightarrow k: \end{array} \right. \right\}.$$

The scheme is secure if and only if

$$D_1 \approx_c D_2.$$

2.2.2.3 Instantiation with Diffie–Hellman Key Exchange.

Definition 2.2.3 (Decisional Diffie–Hellman (DDH) Assumption). *Let $\mathbb{G}$ be a cyclic group of prime order p with generator g and $\mathbb{Z}_p$ denote the set of integers modulo p . The DDH assumption states that, given (g, g^a, g^b, g^c) , where $a, b, c \leftarrow_R \mathbb{Z}_p$, it is computationally infeasible to distinguish whether $c = ab \pmod{p}$ or c is chosen uniformly at random from $\mathbb{Z}_p$. Equivalently, the tuples (g, g^a, g^b, g^{ab}) and (g, g^a, g^b, g^c) are computationally indistinguishable.*

Under the Decisional Diffie–Hellman (DDH) assumption, a key agreement scheme can be constructed in the random oracle model using a hash function G . Let $\mathbb{G}$ be a cyclic group of prime order p with generator g , in which the DDH assumption holds. The scheme is defined as follows:

Definition 2.2.4 (Diffie–Hellman Key Agreement Scheme). *The Diffie–Hellman key agreement scheme consists of the following algorithms:*

- **KA.Param** $(1^\lambda) \rightarrow pp$: *Given security parameter λ , this algorithm selects a cyclic group $\mathbb{G}$ of prime order p and generator $g \in \mathbb{G}$. The public parameters are set to $pp = (\mathbb{G}, g, p)$.*
- **KA.Gen** $(pp) \rightarrow (pk, sk)$: *The algorithm samples $a \leftarrow_R \mathbb{Z}_p$ and outputs $(pk, sk) = (g^a, a)$.*
- **KA.Agree** $(pp, sk, pk_1, pk_2, G) \rightarrow k$: *Given private key $sk = a$ and*

the public keys $pk_1 = g^a$ and $pk_2 = g^{a^2}$, the algorithm computes the shared key k as

$$k = G(pk_2^{sk}) = G(g^{aa^2}).$$

2.2.3 Symmetric Encryption

A symmetric encryption scheme parameterized by a security parameter λ , its key space $\mathcal{K}$, message space $\mathcal{M}$, and ciphertext space $\mathcal{C}$ consists of three algorithms: key generation, encryption, and decryption.

Definition 2.2.5 (Symmetric-Key Encryption Scheme). *It consists of the following algorithms:*

- $\text{Gen}(1^\lambda) \rightarrow K$: Given security parameter λ , it generates secret key $K \leftarrow \mathcal{K}$.
- $\text{Enc}(K, m) \rightarrow c$: Given secret key K and a plaintext message $m \in \mathcal{M}$, it produces ciphertext $c \in \mathcal{C}$.
- $\text{Dec}(K, c) \rightarrow m$: Given secret key K and ciphertext $c \in \mathcal{C}$, it recovers the plaintext $m \in \mathcal{M}$, or outputs $\perp$ if the ciphertext is invalid.

2.2.3.1 Correctness

A symmetric encryption scheme is correct if, for every key $K \in \mathcal{K}$ and every message $m \in \mathcal{M}$, an honestly generated ciphertext decrypts to the original message:

$$\forall K \in \mathcal{K}, m \in \mathcal{M}, \quad \Pr[\text{Dec}_K(\text{Enc}_K(m)) \neq m] = 0.$$

2.2.3.2 Security

A symmetric encryption scheme is secure if and only if it satisfies indistinguishability under chosen-plaintext attacks (IND-CPA) [32]. Informally, given the ciphertexts of messages chosen by an adversary, the adversary cannot distinguish which of two messages was encrypted, except with negligible probability.

2.2.3.3 Authentication

A symmetric encryption scheme is authenticated if and only if it satisfies ciphertext integrity (INT-CTXT) [32]. That is, an adversary cannot generate

a new valid ciphertext that is accepted by the decryption algorithm, except with negligible probability.

In this thesis, we will use (Enc, Dec) for authenticated encryption scheme.

2.2.4 Threshold Secret Sharing

A *Threshold Secret Sharing* (SS) scheme enables a secret value to be distributed among n parties such that any sufficiently large subset of the parties can reconstruct the secret, while a smaller subset learns no information about it. A threshold t specifies the minimum number of shares required for reconstruction.

Definition 2.2.6 (Threshold Secret Sharing Scheme). *A threshold secret sharing scheme is parameterized by a security parameter λ , a secret space $\mathcal{I}$, threshold t , and consists of the following algorithms:*

- **SS.Share**($s, t, \mathcal{U} = \{1, \dots, n\}, \mathcal{I}$) $\rightarrow (u, [s]_u)_{u \in \mathcal{U}}$: *Given secret $s \in \mathcal{I}$, threshold t , and number of parties $n = |\mathcal{U}|$, this algorithm generates n shares of the secret, where $[s]_u$ denotes the share assigned to party u .*
- **SS.Recon**(($u, [s]_u$) $_{u \in \mathcal{U}'}$) $\rightarrow s$: *Given set of shares indexed by $\mathcal{U}' \subseteq \mathcal{U}$ such that $|\mathcal{U}'| \geq t$, this algorithm reconstructs and outputs the secret s when at least t valid shares are provided.*

2.2.4.1 Correctness

A threshold secret sharing scheme is correct if and only if any set of at least t shares generated from secret $s \in \mathcal{I}$ reconstructs the original secret with probability 1. Formally,

$$\forall s \in \mathcal{I}, \forall n, t, \forall \mathcal{U}' \subseteq \mathcal{U} \text{ such that } |\mathcal{U}'| \geq t,$$

$$\Pr \left[\begin{array}{c} \text{SS.Share}(s, t, \mathcal{U} = \{1, \dots, n\}, \mathcal{I}) \rightarrow (u, [s]_u)_{u \in \mathcal{U}}, \\ \text{SS.Recon}((u, [s]_u)_{u \in \mathcal{U}'}) \neq s \end{array} \right] = 0.$$

2.2.4.2 Security

A threshold secret sharing scheme is secure if any set of fewer than t shares reveals no information about the secret. Formally, for any two secrets $s, s' \in \mathcal{I}$ and any subset $\mathcal{U}' \subseteq \mathcal{U}$ such that $|\mathcal{U}'| < t$, the distributions of the shares

held by the parties in $\mathcal{I}$ are computationally indistinguishable:

$$\text{SS.Share}(s, t, n, \mathcal{I}) \approx_c \text{SS.Share}(s', t, n, \mathcal{I})$$

Thus, an adversary that obtains fewer than t shares cannot distinguish whether those shares were generated from s or from s' .

2.2.4.3 Instantiation of SS over the Field

A well-known realization of threshold secret sharing is *Shamir's Secret Sharing* (SSS) scheme [33]. SSS is defined over the finite field $\mathbb{Z}_p$ and provides a t -out-of- n threshold secret sharing scheme. The construction is based on a randomly generated polynomial of degree $t - 1$.

Definition 2.2.7 (Shamir's Secret Sharing Scheme). *It consists of the following algorithms:*

- $\text{SSS.Share}(s, t, \mathcal{U} = \{1, \dots, n\}, \mathbb{Z}_p) \rightarrow (u, [s]_u)_{u \in \mathcal{U}}$: Given a secret $s \in \mathbb{Z}_p$, the algorithm samples a polynomial

$$f(x) = s + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1},$$

where $a_1, \dots, a_{t-1} \leftarrow \mathbb{Z}_p$, such that $f(0) = s$. The algorithm then generates the share for each party $u \in \mathcal{U}$ as $[s]_u = f(u)$. It outputs $(u, [s]_u)_{u \in \mathcal{U}}$.

- $\text{SSS.Recon}((u, [s]_u)_{u \in \mathcal{U}'}, \mathbb{Z}_p) \rightarrow s$: Given a set of at least t shares from parties in $\mathcal{U}' \subseteq \mathcal{U}$, the algorithm reconstructs the secret by evaluating the polynomial at $x = 0$ using Lagrange interpolation [34].

$$s = f(0) = \sum_{u \in \mathcal{U}'} \lambda_u [s]_u \pmod{p}, \text{ where } \lambda_u = \prod_{\substack{v \in \mathcal{U}' \\ v \neq u}} \frac{v}{v - u} \pmod{p}.$$

- $\text{SSS.ReconExp}((u, g^{[s]_u})_{u \in \mathcal{U}'}, G) \rightarrow g^s$: Given a set of at least t exponentiated shares from parties in $\mathcal{U}' \subseteq \mathcal{U}$, where $G = \langle g \rangle$ is a cyclic group of prime order p , the algorithm reconstructs the exponentiation of the secret without explicitly recovering s . It applies Lagrange interpolation in the exponent to compute

$$g^s = g^{f(0)} = \prod_{u \in \mathcal{U}'} (g^{[s]_u})^{\lambda_u}$$

Correctness Given a cyclic group G of prime order p with generator g , it holds that

$$\forall s, n, t, \mathcal{U}, \mathcal{U}', g \text{ s.t. } (\mathcal{U} = \{u_1, \dots, u_n\}) \wedge (\mathcal{U}' \subseteq \mathcal{U}) \wedge (|\mathcal{U}'| \geq t) \wedge (s \in \mathbb{Z}_p),$$

$$\Pr \left[r \neq g^s \mid \begin{array}{l} \text{SSS.Share}(s, t, \mathcal{U}, \mathbb{Z}_p) \rightarrow \{(u, [s]_u)\}_{u \in \mathcal{U}}, \\ \text{SSS.ReconExp}(\{(u, g^{[s]_u})\}_{u \in \mathcal{U}'}, G) \rightarrow r \end{array} \right] = 0.$$

Security Given cyclic group G of prime order p with generators $g_1, g_2 \in G$, it holds that

$$\forall s_1, s_2, n, t, \mathcal{U}, \mathcal{U}', g_1, g_2 \text{ s.t. } (\mathcal{U} = \{u_1, \dots, u_n\}) \wedge (\mathcal{U}' \subseteq \mathcal{U}) \wedge (|\mathcal{U}'| \geq t) \wedge (s_1, s_2 \in \mathbb{Z}_p),$$

$$\Pr \left[r \neq g_1^{s_1} g_2^{s_2} \mid \begin{array}{l} \text{SSS.Share}(s_1, t, \mathcal{U}, \mathbb{Z}_p) \rightarrow \{(u, [s_1]_u)\}_{u \in \mathcal{U}}, \\ \text{SSS.Share}(s_2, t, \mathcal{U}, \mathbb{Z}_p) \rightarrow \{(u, [s_2]_u)\}_{u \in \mathcal{U}}, \\ \text{SSS.ReconExp}(\{(u, g_1^{[s_1]_u} g_2^{[s_2]_u})\}_{u \in \mathcal{U}'}, G) \rightarrow r \end{array} \right] = 0.$$

2.2.4.4 Instantiation of SS over the Integers

A variant of Shamir's secret sharing can be defined over the integers rather than over a finite field. The integer secret sharing scheme (ISS) [35] shares a secret integer s in an interval $[-I, I]$ and provides statistical security controlled by a security parameter λ .

Definition 2.2.8 (Integer Secret Sharing Scheme). *It consists of the following algorithms:*

- $\text{ISS.Share}(s, t, \mathcal{U} = \{1, \dots, n\}, I) \rightarrow \{(u, [\Delta s]_u)\}_{u \in \mathcal{U}}$: This algorithm splits secret $s \in [-I, I]$ into n shares $[\Delta s]_u$, assigning each share to a user $u \in \mathcal{U}$, with t being the reconstruction threshold and $\Delta = n!$. The algorithm creates a random polynomial $p(x)$ of degree $t - 1$ where $p(0) = \Delta s$ and each share is $[\Delta s]_u = p(u)$.
- $\text{ISS.Recon}(\{(u, [\Delta s]_u)\}_{u \in \mathcal{U}'}, t) \rightarrow s$: This algorithm reconstructs secret $s \in [-I, I]$ from at least t shares received from users in $\mathcal{U}' \subseteq \mathcal{U}$. Using Lagrange interpolation, the secret is computed as:

$$s = \frac{\sum_{u \in \mathcal{U}'} \mu_u [\Delta s]_u}{\Delta^2}, \quad \text{such that } \mu_u = \frac{\Delta \cdot \prod_{v \in \mathcal{U}' \setminus \{u\}} v}{\prod_{v \in \mathcal{U}' \setminus \{u\}} (v - u)}.$$

μ_u represents the Lagrange coefficient for user u .

2.3 Secure Aggregation

As mentioned in Section 2.1, FL clients keep their datasets locally but share information derived from them, such as model updates or gradients with the FL server. These update can still reveal sensitive information about the training data [13]. Therefore, only keeping the data local does not guarantee privacy. SA addresses this limitation by allowing the server to obtain the aggregated result of client updates without accessing their individual contributions. In general, the objective of SA is to compute the aggregate of multiple parties' inputs while preventing the disclosure of individual inputs beyond what can be inferred from the aggregate.

Over the past two decades, extensive research has focused on developing SA protocols for various applications [19, 20, 21]. These applications include distributed systems such as sensor networks, where multiple sensors collect and report measurements, and distributed databases, where data is stored across multiple machines. With the emergence of FL [12], SA protocols have been adapted to the FL setting to protect clients' model updates from the server while supporting robustness to client dropouts during training [36, 22].

SA involves the following parties:

- **Clients ($\mathcal{U}$):** Parties that hold private inputs x_u and participate in the aggregation protocol.
- **Aggregator ($\mathcal{A}$):** The party that performs the aggregation to obtain the aggregation result without learning the individual client inputs.
- **Trusted Third Party ($\mathcal{TP}$):** An optional trusted entity that may assist with the initialization or setup of the protocol.

SA protocols can be realized using different techniques, including SA based on DP [37, 38], SA based on Trusted Execution Environments (TEEs) [31, 39, 40], and SA based on cryptographic mechanisms [36, 22, 28]. In this thesis, we focus on cryptography-based SA, and more specifically on encryption-based techniques, where dedicated encryption schemes protect clients' individual updates while allowing the server to recover their aggregate only.

Let $\mathcal{U} = \{1, \dots, n\}$ denote the set of n clients, where each client $u \in \mathcal{U}$ holds input $x_{u,\tau}$ at round τ . The SA protocol enables aggregator $\mathcal{A}$ to only obtain the aggregate $x_\tau = \sum_{u=1}^n x_{u,\tau}$, while keeping each individual x_u hidden from the aggregator. Depending on the protocol $\mathcal{TP}$ may also be involved during an initial setup phase.

Definition 2.3.1 (Secure Aggregation Scheme). A secure aggregation (SA) scheme enables a set of clients to jointly compute the aggregation of their private inputs while preventing the aggregator from learning any individual input. The scheme consists of the following entities and algorithms.

- **SA.Setup**($1^\lambda, n$) $\rightarrow$ ($pp, \{ek_u\}_{u \in \mathcal{U}}, ak_0$) : On input security parameter λ and total number of clients n , the algorithm outputs public parameters pp , n encryption keys $\{ek_u\}_{u \in \mathcal{U}}$, and an aggregation key ak_0 for the server where $|\mathcal{U}| = n$.
- **SA.Protect**($pp, \tau, ek_u, x_{u,\tau}$) $\rightarrow y_{u,\tau}$: Given public parameters pp , encryption key ek_u , and client u 's plaintext input $x_{u,\tau}$ at round τ , the algorithm outputs ciphertext $y_{u,\tau}$ corresponding to the input of user u .
- **SA.Agg**($pp, \tau, ak_0, \{y_{u,\tau}\}_{u \in \mathcal{U}}$) $\rightarrow X_\tau$: Given round τ , the server's aggregation key ak_0 , and ciphertexts $\{y_{u,\tau}\}_{u \in \mathcal{U}}$ of clients in $\mathcal{U}$, this algorithm outputs the aggregated plaintext as:

$$X_\tau = \sum_{u \in \mathcal{U}} x_{u,\tau}.$$

2.3.1 Security Requirements

The security of SA is primarily characterized by two properties: *aggregator obliviousness* and *aggregate unforgeability*. Aggregator obliviousness [41] ensures the confidentiality of individual client inputs, while aggregate unforgeability [42] ensures the integrity of the computed aggregate.

2.3.1.1 Aggregator Obliviousness

A secure aggregation scheme satisfies *aggregator obliviousness* if the view of the aggregator during the execution of the protocol does not reveal any information about the individual inputs of honest users beyond what can be inferred from the aggregate and the inputs of corrupted users.

Let $\mathcal{U} = \{1, \dots, n\}$ denote the set of users, and let $\mathcal{C} \subseteq \mathcal{U}$ and $\mathcal{H} = \mathcal{U} \setminus \mathcal{C}$ denote the sets of corrupted and honest users, respectively. Each user $u \in \mathcal{U}$ holds a private input x_u , and the aggregate is

$$X = \sum_{u \in \mathcal{U}} x_u.$$

Let $\text{View}_{\mathcal{A}}$ denote the view of the aggregator $\mathcal{A}$ during the execution of the protocol, including its internal state, public parameters, and all messages received from the users. The aggregator obliviousness property requires that

$$\text{View}_{\mathcal{A}} \approx_c \text{Sim}(pp, \{x_u\}_{u \in \mathcal{C}}, X),$$

where Sim is a probabilistic polynomial-time simulator and $\approx_c$ denotes computational indistinguishability.

Thus, the view of the aggregator can be simulated only using the public parameters, the inputs of corrupted users, and the aggregate X . In particular, it reveals no additional information about the individual inputs $\{x_u\}_{u \in \mathcal{H}}$ of the honest users beyond what is implied by these values.

2.3.1.2 Aggregate Unforgeability

A secure aggregation scheme satisfies *aggregate unforgeability* if an adversary corrupting the aggregator cannot cause an incorrect aggregate to be accepted by the honest users without being detected. Let $\mathcal{C} \subseteq \mathcal{U}$ denote the set of corrupted users and $\mathcal{H} = \mathcal{U} \setminus \mathcal{C}$ the set of honest users. For inputs $\{x_u\}_{u \in \mathcal{U}}$, let

$$X = \sum_{u \in \mathcal{H}} x_u$$

denote the aggregate of the honest users' inputs. Assume that each user input is bounded by a maximum value θ , i.e.,

$$x_u < \theta.$$

Then, any aggregate X' accepted by the honest users must satisfy

$$X \leq X' \leq X + \theta|\mathcal{C}|.$$

Thus, the adversary can affect the aggregate only through the inputs of the corrupted users and cannot arbitrarily change the aggregate without being detected.

2.3.2 Joye-Libert SA Scheme

As a concrete example of a SA construction, we consider the *Joye-Libert* (JL) scheme proposed in [43]. The scheme enables multiple clients to encrypt their private inputs such that the aggregator can compute their aggregation while keeping the individual inputs confidential.

It consists of the following algorithms:

- **JL.Setup** $(1^\lambda, n) \leftarrow (pp, \{ek_u\}_{u \in \mathcal{U}}, ak_0)$. Given security parameter λ and number of clients n , the setup algorithm generates two equal-size large prime numbers p and q and sets $N = pq$. It randomly generates n users' encryption keys $ek_u \xleftarrow{R} \{0, 1\}^{2l}$, where l denotes the bit length of N , and sets the aggregation secret key as $ak_0 = -\sum_{u=1}^n ek_u$. It then defines a cryptographic hash function $H : \mathbb{Z} \rightarrow \mathbb{Z}_{N^2}^*$. The public parameters is given as $pp = (N, H)$.
- **JL.Protect** $(pp, \tau, ek_u, x_{u,\tau}) \leftarrow y_{u,\tau}$. Given the public parameters $pp = (N, H)$, a secret key ek_u , and a private input $x_{u,\tau} \in \mathbb{Z}_N$ at time period τ , the algorithm computes the ciphertext

$$y_{u,\tau} = (1 + x_{u,\tau}N) H(\tau)^{ek_u} \pmod{N^2}.$$

The resulting ciphertext $y_{u,\tau}$ is sent to the aggregator.

- **JL.Agg** $(pp, \tau, ak_0, \{y_{u,\tau}\}_{u \in \mathcal{U}}) \leftarrow X_\tau$. Given the public parameters $pp = (N, H)$, the aggregation secret key ak_0 , the time period τ , and the n ciphertexts received from the users, the algorithm first computes the aggregated ciphertext $y_\tau = \prod_{u=1}^n y_{u,\tau} \pmod{N^2}$. and decrypts the result. It obtains the sum of the private inputs $X_\tau = \sum_{u=1}^n x_{u,\tau}$ as follows:

$$V_\tau = H(\tau)^{ak_0} \cdot y_\tau \quad \text{and} \quad X_\tau = \frac{V_\tau - 1}{N} \pmod{N}.$$

Thus, the JL construction provides a concrete realization of SA in which users' inputs are encrypted before sending them to the aggregator and aggregation is performed directly over the ciphertexts. The aggregator only learns the aggregated result without obtaining the individual plaintext inputs. The scheme provides aggregator obliviousness under the Decision Composite Residuosity (DCR) assumption in the random-oracle model, as established in [44].

2.3.3 Threshold Secure Aggregation

As discussed in Chapter 1, client dropouts may occur during a training round in FL due to unreliable network connections, device failures, or limited client availability. Standard SA protects clients' inputs during aggregation but typically requires all clients to send their protected inputs. When a client drops

out, its cryptographic contribution may therefore prevent the aggregation from being completed.

Threshold Secure Aggregation (TSA) extends SA with a threshold mechanism that enables aggregation even when some clients drop out during a training round. Let $\mathcal{U}_\tau^{on} \subseteq \mathcal{U}$ denote the set of clients that remain online at round τ . During the setup phase, each client obtains protected information associated with the other clients. In each round, online client uses this information to derive a one-time aggregation key for that round. These round-specific aggregation keys allow the server to combine the protected client inputs and recover their aggregate. Thus, clients that drop out or are otherwise absent from $\mathcal{U}_\tau^{on}$ do not prevent the aggregation, as long as the number of remaining online clients satisfies the threshold condition $|\mathcal{U}_\tau^{on}| \geq t$.

Thus, compared with standard SA, TSA additionally incorporates a threshold mechanism that determines the minimum number of online clients required for the aggregation to be completed. The following definition describes the general TSA framework and its main algorithms.

Definition 2.3.2 (Threshold Secure Aggregation Scheme). *It consists of the following algorithms:*

- **TSA.Setup** $(1^\lambda, n, t) \rightarrow (pp, \{ek_u\}_{u \in \mathcal{U}}, \{ak_u\}_{u \in \mathcal{U}})$: On input security parameter λ , total number of users n , and threshold $t \leq n$, the algorithm outputs public parameters pp , n encryption keys $\{ek_u\}_{u \in \mathcal{U}}$, and n aggregation keys $\{ak_u\}_{u \in \mathcal{U}}$, where $|\mathcal{U}| = n$.
- **TSA.Protect** $(pp, \tau, ek_u, x_{u,\tau}) \rightarrow Y_{u,\tau} := (y_{u,\tau}^{(S)}, \{y_{u,\tau}^{(v)}\}_{v \in \mathcal{U}})$: Given public parameters pp , encryption key ek_u and user u 's plaintext input $x_{u,\tau}$ at round τ , the algorithm outputs several ciphertexts: one ciphertext $y_{u,\tau}^{(S)}$ intended for the server, together with a set of ciphertexts $\{y_{u,\tau}^{(v)}\}_{v \in \mathcal{U}}$, where each $y_{u,\tau}^{(v)}$ is intended for a specific client $v \in \mathcal{U}$.
- **TSA.Agg** $(pp, \tau, \{ak_u\}_{u \in \mathcal{U}_\tau^{on}}, \{Y_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}}) \rightarrow x_{agg,\tau}$: Given round τ , ciphertexts $\{Y_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}}$, aggregation keys $\{ak_u\}_{u \in \mathcal{U}_\tau^{on}}$ of users in $\mathcal{U}_\tau^{on}$, this algorithm frequently invokes:
 - **Client.TSA.Agg** $(pp_{TSA}, \tau, ak_u, \{y_{v,\tau}^{(u)}\}_{v \in \mathcal{U}_\tau^{on}}) \rightarrow ak_{u,\tau}$: outputs a one-time aggregation key $ak_{u,\tau}$ for round τ and $u \in \mathcal{U}_\tau^{on}$.
 - **Server.TSA.Agg** $(pp_{TSA}, \tau, \{ak_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}}, \{y_{u,\tau}^{(S)}\}_{u \in \mathcal{U}_\tau^{on}}) \rightarrow x_\tau$: which outputs the aggregated plaintext $x_{agg,\tau} = \sum_{u \in \mathcal{U}_\tau^{on}} x_{u,\tau}$.

Lemma 2.1. *TSA scheme $\text{TSA} = (\text{TSA.Setup}, \text{TSA.Protect}, \text{TSA.Agg})$ is round-based ciphertext homomorphic if there exist some operations $\boxplus, \boxtimes$ such that, for any tags $\tau_1 \neq \tau_2$, any client $v \in \mathcal{U}$, and any values $x_{v,\tau_1}, x_{v,\tau_2}$ we have that the following distributions are computationally indistinguishable:*

$$Y_{v,\tau_1} \boxplus Y_{v,\tau_2} \approx_c \tilde{Y}_{v,\tau_1 \boxtimes \tau_2}$$

where $\text{TSA.Setup}(1^\lambda, n, t, r) \rightarrow (pp, \{ek_u\}_{u \in \mathcal{U}}, \{ak_u\}_{u \in \mathcal{U}})$,
 $\text{TSA.Protect}(pp, \tau_i, ek_v, x_{v,\tau_i}) \rightarrow Y_{v,\tau_i}$ for $i \in \{1, 2\}$, and
 $\text{TSA.Protect}(pp_{\text{TSA}}, \tau_1 \boxtimes \tau_2, ek_v, x_{v,\tau_1} + x_{v,\tau_2}) \rightarrow \tilde{Y}_{v,\tau_1 \boxtimes \tau_2}$.

2.3.3.1 Functionality of TSA

To capture the ideal behavior of TSA, we define the ideal functionality $\mathcal{F}_{\text{TSA}}$. The functionality receives the inputs of the clients participating in a given round and returns their aggregate to the server. It aborts when fewer than the threshold number of clients are available.

Auxiliary Inputs: Security parameter λ and descriptions of algorithms TSA.Setup , TSA.Protect , and TSA.Agg . Upon receiving $(\text{sid}, \tau, \{x_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}})$ where $\mathcal{U}_\tau^{on} \subseteq \mathcal{U}$ denotes the set of clients whose inputs are available in round τ , the functionality proceeds as follows:

- If $|\mathcal{U}_\tau^{on}| < t$, send $(\text{Abort}, \text{sid})$;
- Else, compute $x_\tau = \sum_{u \in \mathcal{U}_\tau^{on}} x_{u,\tau}$, and send (x_τ, sid) to the server $\mathcal{S}$.

Thus, $\mathcal{F}_{\text{TSA}}$ captures the intended functionality of TSA: the server learns the aggregate of the available client inputs only, provided that at least t clients participate in the aggregation. Its security mainly relies on the standard security definition of a multi-party computation protocol (see App. ??).

2.3.3.2 Threshold Joye-Libert Scheme

We instantiate the TSA framework using the *Threshold Joye-Libert* (TJL) scheme introduced in [22]. The TJL construction combines the Joye-Libert SA mechanism with threshold secret sharing to support secure aggregation in the presence of client dropouts. In particular, it allows the server to recover the aggregate of the inputs from the set of online clients while using secret shares to account for clients that are unavailable during a training round.

The TJL scheme consists of the following algorithms:

- **TJL.Setup**($1^\lambda, n, t$) : Given security parameter λ , the total number of clients n and threshold $t \leq n$, the algorithm generates two equal-size large prime numbers p, q and compute the modulus $N = p \cdot q$. Then, it runs **SSS.Setup**($1^\lambda, t, n, \mathbb{Z}_N$) $\rightarrow pp_{SS}$, **ISS.Setup**($1^\lambda, t, n, [-N^2, N^2]$) $\rightarrow pp_{ISS}$. It also sets a cryptographic hash function $H : \mathcal{T} \rightarrow \mathbb{Z}_{N^2}^*$ where $\mathcal{T}$ denotes the round space. Next, the algorithm samples the following two elements:

$$sk_u \leftarrow_R [-N^2, N^2] \text{ for all } u \in \mathcal{U},$$

$$ak_0 = - \sum_{u \in \mathcal{U}} ek_u,$$

and secret shares ek_u using **ISS.Share**($ek_v, t, \mathcal{U}$) $\rightarrow \{u, [\Delta \cdot ek_v]_u\}_{u \in \mathcal{U}}$ for all $v \in \mathcal{U}$. Finally, it sets

$$pp = (n, t, N, H, pp_{SS}, pp_{ISS}, ak_0),$$

$$ak_u = \{[\Delta \cdot ek_v]_u\}_{v \in \mathcal{U}} \text{ for all } u \in \mathcal{U}.$$

- **TJL.Protect**($pp, \tau, ek_u, x_{u,\tau}$) $\rightarrow Y_{u,\tau} := (y_{u,\tau}^{(S)}, \{y_{u,\tau}^{(v)}\}_{v \in \mathcal{U}})$: On inputs round τ , encryption key ek_u of client $u \in \mathcal{U}$, and message $x_{u,\tau}$, the algorithm samples $B_{u,\tau} \leftarrow_R \mathbb{Z}_N$, and then computes

$$y_{u,\tau}^{(S)} = (1 + (B_{u,\tau} + x_{u,\tau}) \cdot N) \cdot H(\tau)^{ek_u} \mod N^2,$$

$$\text{SSS.Share}(B_{u,\tau}, t, \mathcal{U}) \rightarrow \{u, [B_{u,\tau}]_v\}_{v \in \mathcal{U}}.$$

Finally, it sets $y_{u,\tau}^{(v)} = [B_{u,\tau}]_v$ for all $v \in \mathcal{U}$.

- **TJL.Agg**($pp, \tau, \{ak_u\}_{u \in \mathcal{U}_\tau^{on}}, \{Y_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}}$) $\rightarrow x_{agg,\tau}$: Given round τ , ciphertexts $\{Y_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}}$, aggregation keys $\{ak_u\}_{u \in \mathcal{U}_\tau^{on}}$ of users in $\mathcal{U}_\tau^{on}$, this algorithm invokes:
 - **Client.TJL.Agg**($pp, \tau, ak_u, \{y_{v,\tau}^{(u)}\}_{v \in \mathcal{U}_\tau^{on}}$) $\rightarrow ak_{u,\tau}$: On inputs round τ , client u 's aggregation key ak_u and set of ciphertexts $\{y_{v,\tau}^{(u)}\}_{v \in \mathcal{U}_\tau^{on}}$ corresponding to clients $v \in \mathcal{U}_\tau^{on}$, client u first defines the set $\mathcal{U}_\tau^{off} = \mathcal{U} \setminus \mathcal{U}_\tau^{on}$ and then computes their share of 0 encryption

$$[y_\tau^{off}]_u = H(\tau)^{\sum_{v \in \mathcal{U}_\tau^{off}} [\Delta \cdot ek_v]_u}.$$

Then, it sets the temporary aggregation key $ak_{u,\tau} = ([y_\tau^{off}]_u, \{[B_{v,\tau}]_u\}_{v \in \mathcal{U}_\tau^{on}})$ and sends it to the server.

- **Server.TJL.Agg**($pp, \{ak_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}}, \{y_{u,\tau}^{(S)}\}_{u \in \mathcal{U}_\tau^{on}} \rightarrow x_{agg,\tau}$: On inputs the temporary aggregation keys $\{ak_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}}$ and the set of ciphertexts $\{y_{u,\tau}^{(S)}\}_{u \in \mathcal{U}_\tau^{on}}$, the server first computes

$$y_\tau^{off} = \prod_{u \in \mathcal{U}_\tau^{on}} ([y_\tau^{off}]_u)^{\mu_u},$$

where μ_u is as described in Definition 2.2.8. Then, it computes

$$y_\tau = \left(\prod_{u \in \mathcal{U}_\tau^{on}} y_{u,\tau}^{(S)} \right)^{\Delta^2} \cdot y_\tau^{off}.$$

Then, using the aggregation key ak_0 , the server computes

$$V_\tau = H(\tau)^{\Delta^2 \cdot ak_0} \cdot y_\tau,$$

$$C_\tau = \frac{V_\tau - 1}{N \cdot \Delta^2} \mod N.$$

Finally, using the secret shares of $B_{u,\tau}$, it computes

$$\{B_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}} \leftarrow \text{SSS.Combine}(\{v, [B_{u,\tau}]_v\}_{v \in \mathcal{U}_\tau^{on}}),$$

$$x_{agg,\tau} = C_\tau - \sum_{u \in \mathcal{U}_\tau^{on}} B_{u,\tau} \mod N.$$

The TJL scheme provides Aggregator Obliviousness under the decisional composite residuosity (DCR) assumption [44] in the random-oracle model, provided that the number of corrupted users is strictly less than threshold t .

Lemma 2.2. *The TJL threshold SA scheme defined above is round-based ciphertext homomorphic, if SS is a t -out-of- n linear secret sharing scheme over $\mathbb{Z}_N$.*

Proof. Let us choose a client $v \in \mathcal{U}$ at random and let any τ_1, τ_2 and $x_{u,\tau_1}^1, x_{u,\tau_2}^2$. Then, we define the operation $\boxplus$ as the product of $y_{u,\tau}^{(S)}$ modulo N^2 and addition of shares $[B_{v,\tau}]_u$ modulo N , and the operation $\boxtimes$ as the

inverse of the product of hashes. Then, we get

$$\begin{aligned}
\tilde{y}_{v,\tau_1 \boxtimes \tau_2}^{(u)} &:= [B_{v,\tau_1}]_u + [B_{v,\tau_2}]_u \mod N \\
&= [B_{v,\tau_1} + B_{v,\tau_2} \mod N]_u \\
\tilde{y}_{v,\tau_1 \boxtimes \tau_2}^{(S)} &= (1 + (B_{v,\tau_1} + x_{u,\tau_1}) \cdot N) \cdot H(\tau_1)^{ek_v} \\
&\quad \cdot (1 + (B_{v,\tau_2} + x_{v,\tau_2}) \cdot N) \cdot H(\tau_2)^{ek_v} \mod N^2 \\
&= \left(1 + (\tilde{B}_{v,\tau_1 \boxtimes \tau_2} + x_{u,\tau_1} + x_{u,\tau_2}) \cdot N\right) \cdot (H(\tau_1) \cdot H(\tau_2))^{ek_v} \mod N^2 \\
&= \left(1 + (\tilde{B}_{v,\tau_1 \boxtimes \tau_2} + x_{u,\tau_1} + x_{u,\tau_2}) \cdot N\right) \cdot H(\tau_1 \boxtimes \tau_2)^{ek_v} \mod N^2
\end{aligned}$$

where we have defined $\tau_1 \boxtimes \tau_2 = H^{-1}(H(\tau_1) \cdot H(\tau_2))$. It is clear that $Y_{v,\tau_1} \boxplus Y_{v,\tau_2}$ is indistinguishable from $\tilde{Y}_{v,\tau_1 \boxtimes \tau_2}$ (reminder that $Y_{v,\tau} := (y_{v,\tau}^{(S)}, \{y_{v,\tau}^{(u)}\}_{u \in \mathcal{U}})$) since $B_{u,\tau_1} + B_{u,\tau_2} \mod N$ is indistinguishable from $\tilde{B}_{v,\tau_1 \boxtimes \tau_2}$ sampled uniformly at random from $\mathbb{Z}_N$ given the linear homomorphism of $\mathbf{SS}$ over $\mathbb{Z}_N$. $\square$

2.4 Differential Privacy

Differential Privacy (DP) is a mathematical framework for protecting information about individuals while allowing useful computations to be performed on a dataset. It was introduced by Dwork et al. [45] in the context of privacy-preserving statistical analysis. The central idea is to limit the influence that any individual data record can have on the output of a computation.

DP achieves this by requiring the output distributions of a randomized mechanism to remain sufficiently similar for two neighboring datasets i.e., where the datasets differ in the data of at most one individual. As a result, an observer should have limited ability to infer whether a particular individual's data was included in the computation.

2.4.0.1 (ε, δ) -Differential Privacy

DP is commonly parameterized by two parameters, ε and δ , which control the strength of the privacy guarantee.

Definition 2.4.1 ((ε, δ) -Differential Privacy [45, 46]). *A randomized mechanism $\mathcal{M}$ satisfies (ε, δ) -differential privacy, for $\varepsilon, \delta \geq 0$, if, for all pairs of neighboring databases $D, D' \in \mathcal{D}$ and for all measurable sets $\mathcal{O} \subseteq \text{Range}(\mathcal{M})$,*

$$\Pr[\mathcal{M}(D) \in \mathcal{O}] \leq e^\varepsilon \Pr[\mathcal{M}(D') \in \mathcal{O}] + \delta.$$

The parameter ε , commonly referred to as the *privacy budget*, controls the privacy–utility trade-off. A smaller ε provides stronger privacy but generally requires greater perturbation and may reduce utility [46]. The parameter δ allows a small probability that the privacy guarantee may not hold exactly and is typically chosen to be very small.

An important property of DP is that its guarantee does not depend on assumptions about the adversary’s background knowledge or computational capabilities. It therefore provides a worst-case privacy guarantee even when the adversary possesses additional information about the dataset [45].

2.4.0.2 ε -Differential Privacy

(ε, δ) -DP reduces to the special case of ε -DP by setting $\delta = 0$. Which means that there is no additional probability of a privacy violation. In this case, the privacy guarantee depends on the parameter ε only.

Definition 2.4.2 (ε -Differential Privacy [45, 46]). *A randomized mechanism $\mathcal{M}$ satisfies ε -differential privacy, for $\varepsilon \geq 0$, if, for all pairs of neighboring datasets $D, D' \in \mathcal{D}$ and for all measurable sets $\mathcal{O} \subseteq \text{Range}(\mathcal{M})$,*

$$\Pr[\mathcal{M}(D) \in \mathcal{O}] \leq e^\varepsilon \Pr[\mathcal{M}(D') \in \mathcal{O}].$$

The definitions above specify the required privacy guarantees but do not determine the amount of noise required to achieve them. This depends on how much the output of an underlying function f can change when the data of a single individual changes. This property is quantified by the *global sensitivity* of f .

2.4.0.3 Global Sensitivity

Global sensitivity measures the maximum change in the output of a function when evaluated on any pair of neighboring datasets. It therefore determines the amount of perturbation required to achieve a given DP guarantee.

A *query function* is a function that takes a dataset as input and produces the value or vector of values that is to be released. For a query function $f: \mathcal{D} \rightarrow \mathbb{R}^d$, the L_p -global sensitivity is defined as

$$\Delta_p(f) = \max_{D, D' \in \mathcal{D}, D \sim D'} \|f(D) - f(D')\|_p,$$

where $D \sim D'$ indicates that D and D' differ in the data of a single individual, $f(D)$ and $f(D')$ are the outputs of the query function on these datasets, $\|\cdot\|_p$ denotes the L_p norm, and $\Delta_p(f)$ represents the maximum possible change in the output of f over all pairs of neighboring datasets.

Global sensitivity therefore represents the maximum influence that a single individual's data can have on the output of a function. A function with higher sensitivity requires more noise to achieve the same privacy guarantee, while lower sensitivity requires less noise. Thus, global sensitivity is an important parameter for determining the amount of noise used by DP mechanisms.

2.4.1 Differential Privacy Mechanisms

DP is typically achieved by applying a randomized mechanism to the output of a function. The mechanism determines the distribution and magnitude of the perturbation required to satisfy a given privacy guarantee. The required noise depends on the privacy parameters and the sensitivity of the underlying function.

For a vector $x = (x_1, \dots, x_d) \in \mathbb{R}^d$, the L_1 - and L_2 -norms are defined as

$$\|x\|_1 = \sum_{i=1}^d |x_i| \quad \text{and} \quad \|x\|_2 = \left(\sum_{i=1}^d x_i^2 \right)^{1/2}.$$

Accordingly, L_1 -sensitivity measures the maximum change in the output according to the L_1 -norm, while L_2 -sensitivity measures this change according to the L_2 -norm.

Two classical DP mechanisms are the *Laplace mechanism* and the *Gaussian mechanism*. The Laplace mechanism is calibrated using L_1 -sensitivity and provides pure ε -DP, whereas the Gaussian mechanism is calibrated using L_2 -sensitivity and provides an (ε, δ) -DP guarantee under the appropriate calibration.

2.4.1.1 Laplace Mechanism

The *Laplace mechanism* achieves pure ε -DP by adding independently sampled Laplace noise to each coordinate of a numerical query.

Definition 2.4.3 (Laplace Mechanism [47]). *Let $f: \mathcal{D} \rightarrow \mathbb{R}^d$ be a numerical query with L_1 -global sensitivity $\Delta_1(f)$. The Laplace mechanism*

with privacy parameter $\epsilon > 0$ is defined as

$$\mathcal{M}_L(D, f, \epsilon) = f(D) + (Y_1, \dots, Y_d),$$

where the Y_i are random variables drawn independently from a Laplace distribution with zero mean and scale $\Delta_1(f)/\epsilon$.

The noise scale is proportional to the L_1 -sensitivity and inversely proportional to the ϵ . Thus, higher sensitivity requires more noise to maintain the same privacy guarantee, whereas a larger ϵ permits less noise at the cost of weaker privacy. Since Laplace noise is continuous and real-valued, its direct use requires an appropriate representation when the underlying computation is performed over a discrete or modular domain.

2.4.1.2 Gaussian Mechanism

The *Gaussian mechanism* provides an alternative approach by adding normally distributed noise to the output of a numerical query. It is particularly useful when a small additive relaxation δ is permitted in the privacy guarantee.

Definition 2.4.4 (Gaussian Mechanism [47]). *Let $f: \mathcal{D} \rightarrow \mathbb{R}^d$ be a numerical query with L_2 -global sensitivity $\Delta_2(f)$. The Gaussian mechanism is defined as*

$$\mathcal{M}_G(D, f, \epsilon, \delta) = f(D) + (Y_1, \dots, Y_d),$$

where the Y_i are random variables drawn independently from a Gaussian distribution with zero mean and standard deviation $\sigma = \frac{\Delta_2(f)\sqrt{2\ln(1.25/\delta)}}{\epsilon}$.

Unlike the Laplace mechanism, which is calibrated using L_1 -global sensitivity, the Gaussian mechanism is calibrated using L_2 -global sensitivity. The resulting noise is continuous and is therefore naturally suited to real-valued vector computations, including high-dimensional numerical data such as model updates.

2.5 Membership Inference Attack

A Membership Inference Attack (MIA) aims to determine whether a given data record was used during the training of a target machine learning model. Following the approach of Shokri et al. [13], the attack is formulated as a

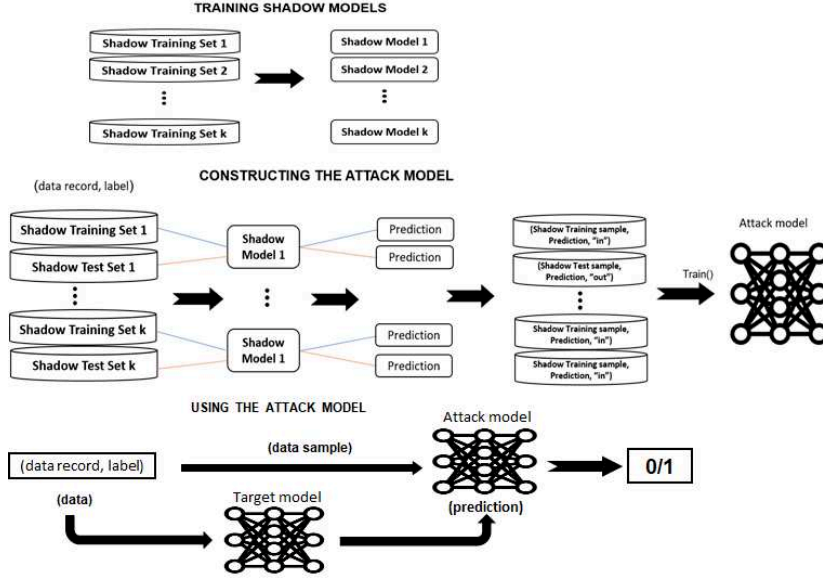


Figure 2.2: Membership Inference Attack.

binary classification problem. The adversary first trains shadow models to reproduce the behavior of the target model and then uses the outputs of these models to train an attack model. The overall procedure is illustrated in Figure 2.2.

Training shadow models. The adversary first constructs k shadow training datasets $D_1^{\text{tr}}, \dots, D_k^{\text{tr}}$ and corresponding shadow test datasets $D_1^{\text{te}}, \dots, D_k^{\text{te}}$. For each $i \in \{1, \dots, k\}$, a shadow model $\mathbb{M}_i$ is trained using D_i^{tr} . Consequently, the adversary knows the membership status of every sample used to train each shadow model: samples from D_i^{tr} are members, whereas samples from D_i^{te} are non-members. As shown in the upper part of Figure 2.2, these shadow models are used to approximate the behavior of the target model.

Constructing the attack model. The adversary next queries each shadow model with samples from both its training and test sets. For a sample (x, y) , the shadow model produces a prediction vector

$$\mathbb{P}_i(x) = \mathbb{M}_i(x),$$

where $\mathbb{P}_i(x)$ contains the predicted probabilities for the output classes. The adversary associates this prediction with the known membership label

$$m_i(x) = \begin{cases} 1, & x \in D_i^{\text{tr}}, \\ 0, & x \in D_i^{\text{te}}. \end{cases}$$

The resulting tuples $(x, y, \mathbb{P}_i(x), m_i(x))$ form the training data for the attack model. In particular, the attack model learns to distinguish the prediction patterns produced for members from those produced for non-members. This corresponds to the middle part of Figure 2.2, where the predictions generated by the shadow models are combined with their known “in” or “out” membership labels to train the attack model.

Using the attack model. After training, the attack model is applied to the target model. Given a target sample (x, y) , the adversary queries the target model and obtains

$$\mathbb{P}_{\text{target}}(x) = \mathbb{M}_{\text{target}}(x).$$

The attack model then takes the target model’s prediction, together with the corresponding label y when used as an attack feature, and outputs

$$\mathcal{A}(\mathbb{P}_{\text{target}}(x), y) \rightarrow \hat{m}(x), \quad \hat{m}(x) \in \{0, 1\}.$$

Here, $\hat{m}(x) = 1$ indicates that the attack model predicts the sample to be a member of the target model’s training set, whereas $\hat{m}(x) = 0$ indicates that it predicts the sample to be a non-member. This process is depicted in the bottom part of Figure 2.2.

2.6 Notation

Table 2.1 summarizes the notation used throughout this thesis.

Table 2.1: Recurring notation used throughout the thesis.

Notation	Explanation
n	Number of clients or users
u	Client index
u'	Unlearn client index
τ	Training round
$\mathcal{U}$	Set of clients
$ \mathcal{U} $	Cardinality of set $\mathcal{U}$
$\mathcal{U}_\tau^{\text{on}}$	Set of online clients during aggregation at round τ
$\mathcal{U}_\tau^{\text{off}}$	Set of dropout clients during aggregation at round τ
$\mathcal{U}_\tau^{\text{forget}}$	Set of online clients during forget at round τ
t	Threshold number of clients required for reconstruction
w_0	Initial global model at round τ
w_τ	Original global model at round τ
$w_{u,\tau}$	Client u local model at round τ
w_u	Unlearned model
$\tilde{w}_\tau$	Ideal global model without the unlearned client at round τ
$\bar{w}_\tau$	Unlearned model without added noise at round τ
$\hat{w}_\tau$	Unlearned model with added noise at round τ
$f_u(w)$	Local loss function of client u
$\nabla f_u(w_{\tau-1})$	Gradient computed by client u at round τ
$\nabla \mathring{f}_u(w_{\tau-1})$	Clipped gradient computed by client u at round τ
α, β	Unlearning privacy parameters
ε	Differential privacy parameter (privacy budget)
δ	Differential privacy relaxation parameter
$\Delta_p(f)$	L_p -global sensitivity of function f
$\ \cdot\ _p$	L_p -norm
D, D'	Pair of neighboring datasets
$\mathcal{D}$	Sampling distribution
$\mathcal{M}$	Randomized mechanism
$\mathcal{O}$	Output event of a randomized mechanism
$\Pr[\cdot]$	Probability of an event
ek_u	Encryption key of client u
$[s]_u$	Share of secret s held by client u
$\boxplus$	Homomorphic addition operation on ciphertexts
$\boxtimes$	Homomorphic operation for protocol rounds

Chapter 3

SAAFL: Secure Aggregation for Label-Aware Federated Learning

This chapter addresses the first challenge, **C1**, by jointly considering **privacy protection**, **client dropout robustness**, and **counter data heterogeneity** in Federated Learning (FL). Existing Secure Aggregation (SA) protocols mainly support conventional aggregation such as Federated Averaging (FedAvg), which may be ineffective for non-IID data. Federated Label-Aware Aggregation (FedLA) [48] addresses this issue by assigning client-specific weights based on their local label distributions. Nevertheless, applying FedLA in SA setting introduces additional challenges, since both the clients' label distributions and model updates must remain private. Moreover, when clients drop out, their client-specific label weights and model contributions may become unavailable, making it difficult to correctly compute the weighted aggregation. To address these challenges jointly, we propose **SAAFL**¹, a privacy-preserving SA protocol for label-aware FL that computes each client's FedLA weight without revealing its local label distribution and securely aggregates the corresponding weighted model updates while handling client dropouts and client selection. As the underlying cryptographic mechanism, we introduce **TJL-A**, a label-aware extension of Threshold Joye-Libert scheme (TJL) that accounts for the contributions of dropouts clients during aggregation.

The work presented in this chapter was accepted and published in 40th International Conference on ICT Systems Security and Privacy Protection – IFIP SEC 2025.

¹SAAFL means simple in Urdu

3.1 Introduction

As discussed in Chapter 2, FedAvg [12] is one of the most widely used aggregation methods in FL. However, recent findings [49, 50] have shown that FedAvg can be less effective when clients hold heterogeneous or non-IID data, particularly when their local datasets exhibit substantially different label distributions. Federated Label-Aware Aggregation (FedLA) [48] addresses this limitation by assigning client-specific aggregation weights according to the label distributions of their local datasets. By taking these client-specific label distributions into account, FedLA improve model aggregation under heterogeneous data.

While existing SA protocols can efficiently support aggregation based on FedAvg, they cannot be directly applied to FedLA. Unlike FedAvg, where the aggregation mainly consists of summing client updates, FedLA requires each update to be multiplied by a client-specific weight derived from its local label distribution. These weights are sensitive and must remain private during the aggregation process. Moreover, client dropouts further complicate the computation, as the absence of a client may also make its corresponding weight and weighted update unavailable. While with FedAvg, dropouts were already addressed by reconstructing an encrypted zero-value [22], such a solution unfortunately falls short in the context of FedLA and may drop the accuracy of the model significantly. Therefore, applying SA to FedLA requires mechanisms that can securely compute the client-specific weights and correctly account for client participation and dropouts without revealing sensitive client information.

In this Chapter, we propose a new FL protocol, named **SAAFL**, that is customized for FedLA. This solution ensures, through a new SA building block, a privacy-preserving clients' weighted model update aggregation along with secure clients' weight computations. Furthermore, at each training round, SAAFL enables the online clients to reconstruct the contributions of dropped clients as weighted updates derived from the model parameters of the previous round, allowing the aggregation to proceed without requiring the dropped clients to participate in the current round.

The main contributions are summarized as follows:

- Design SAAFL, a privacy-preserving SA protocol for label-aware FL that enables the computation of client-specific FedLA weights while keeping clients' label distributions private.
- We introduce TJL-A, a label-aware extension of TJL that provides dropout-resilient SA for weighted model updates and accounts for dropped and non-selected clients.

- We implement SAAFL and experimentally evaluate its accuracy, privacy, computational cost, and communication overhead. The results show that SAAFL achieves model accuracy comparable to FedLA while preserving the privacy of clients’ label information and model updates.

3.2 Background

3.2.1 FedLA Algorithm

As described in [48], FedLA is a FL framework designed to address non-IID data distributions across clients. In conventional Federated Averaging (FedAvg) [12], the global model is obtained by computing a weighted average of clients’ model updates, where the weights are typically determined by the number of local training samples. FedLA instead takes the label distribution of each client’s local dataset into account when determining its contribution to the global model. In particular, FedLA computes a client-specific aggregation weight based on the distribution of labeled samples across clients.

More specifically, consider n clients participating in the training and k labels represented in the overall dataset. Let s_u denote the total number of samples held by client u , s_l denote the total number of samples with label l across all clients, and $s_{u,l}$ denote the number of samples with label l held by client u . The FedLA weight computation consists of three steps.

1. **Client weight per label.** For each label l , FedLA first computes the total number of samples associated with that label across all clients:

$$s_l = \sum_{u=1}^n s_{u,l}. \quad (3.1)$$

The weight of client u with respect to label l is then computed as

$$p_{u,l} = \frac{s_{u,l}}{s_l}. \quad (3.2)$$

Thus, $p_{u,l}$ represents the proportion of samples associated with label l that are held by client u . A larger value of $p_{u,l}$ indicates that client u holds a larger proportion of the samples belonging to label l .

2. **Client weight with respect to all labels.** Next, FedLA combines the client weights computed for individual labels by summing them across all k labels:

$$p_u = \sum_{l=1}^k p_{u,l}. \quad (3.3)$$

The resulting p_u represents the overall contribution of client u across the labels represented in the dataset.

3. **Client update weight.** Finally, FedLA normalizes the client weights across the n participating clients to obtain the aggregation weight used for the client update:

$$p_{\text{FedLA},u} = \frac{p_u}{\sum_{v=1}^n p_v}. \quad (3.4)$$

The resulting $p_{\text{FedLA},u}$ is the FedLA aggregation weight assigned to the model update of client u .

Therefore, unlike FedAvg, where the aggregation weights are typically determined by the number of local training samples, FedLA determines the contribution of each client from its distribution of labeled samples across the overall dataset. The resulting FedLA weights can then be used to compute the global model as a weighted combination of the participating clients' model updates. The complete weight computation is summarized in Algorithm 3.2.1.

Algorithm 3.2.1: FedLA Weights Calculation

Require: Client set $\mathcal{U} = \{u_1, \dots, u_n\}$ and label set $\mathcal{L} = \{l_1, \dots, l_k\}$; sample counts $s_{u,l}$ for all $u \in \mathcal{U}$ and $l \in \mathcal{L}$

Ensure: FedLA aggregation weights $p_{\text{FedLA},u}$ for all $u \in \mathcal{U}$

- 1: **for** each label $l \in \mathcal{L}$ **do**
- 2: Compute total number of samples:
 $s_l \leftarrow \sum_{u \in \mathcal{U}} s_{u,l}$
- 3: **for** each client $u \in \mathcal{U}$ **do**
- 4: Compute client weight per label:
 $p_{u,l} \leftarrow \frac{s_{u,l}}{s_l}$
- 5: **end for**
- 6: **end for**
- 7: **for** each client $u \in \mathcal{U}$ **do**
- 8: Compute client weight with respect to all labels:
 $p_u \leftarrow \sum_{l \in \mathcal{L}} p_{u,l}$
- 9: **end for**
- 10: **for** each client $u \in \mathcal{U}$ **do**
- 11: Compute normalized client update weight:
 $p_{\text{FedLA},u} \leftarrow \frac{p_u}{\sum_{v \in \mathcal{U}} p_v}$

12: end for 13: return $\{p_{\text{FedLA},u}\}_{u \in \mathcal{U}}$
--

3.3 Towards Privacy-preserving FedLA

In this section, we first introduce the threat model considered in this work. We then discuss the main challenges of integrating SA with FedLA. Finally, we provide a high-level overview of our proposed solution and explain how it addresses these challenges.

3.3.1 Threat model

We consider an untrusted FL server that may collude with a subset of clients, while some honest clients may unintentionally fail or drop out during protocol execution. Client failures may occur at different stages of the protocol. The adversary, consisting of the server and its colluding clients, aims to learn information about the private inputs of honest clients beyond what is revealed by the final aggregate.

We consider two types of adversary. In the *passive adversarial model*, the server and colluding clients follow the protocol correctly but attempt to infer private information about honest clients from the messages they receive and the protocol transcript. In the *active adversarial model*, the adversary may deviate from the prescribed protocol and manipulate or inject protocol messages with the objective of obtaining private information about honest clients.

Attacks that intentionally modify the final aggregate or target the availability of the system, such as denial-of-service (DoS) attacks, are outside the scope of this work. We also do not consider client impersonation attacks, as these can be addressed using standard authentication mechanisms, such as public key infrastructure (PKI).

3.3.2 Challenges when Integrating SA with FedLA

Integrating SA into FedLA is not straightforward because FedLA performs a weighted aggregation in which the client-specific weights depend on the clients' label distributions. These weights and label distributions should remain private, while the aggregation must still be correctly computed when clients are dropped or not selected for a training round. Consequently, existing SA schemes designed for conventional aggregation methods such as FedAvg cannot be directly applied to FedLA.

In particular, integrating SA into FedLA raises the following challenges:

- **FedLA-weight privacy:** Unlike the traditional FL approach whereby the FL server only receives private model updates and executes SA, FedLA firstly requires the private computation of each client’s FedLA-weights (which depend on some private inputs from other clients as well). As described in the previous section 3.1, these FedLA-weights should remain confidential to the server [51].
- **Client Dropout in FedLA:** Another challenge that arises with secure FedLA as well as in the standard FL framework is client dropouts. In many real-world scenarios, clients may be intermittently unavailable due to network issues, device power limitations, or other reasons. When clients drop out, their model updates are not received, which can disrupt the training of the model. There already exist many solutions [28, 36, 52, 22, 53] that are resilient to client dropouts in privacy-preserving variants of FL frameworks. In these schemes, online clients mainly reconstruct the encryption of a zero-value for dropped clients [28, 36, 22, 53]. Unfortunately, these solutions fall short in the case of FedLA because clients’ FedLA weights depend on the contributions of all the other clients including the dropped ones. Hence, the reconstruction of the encryption of a zero-value for dropped clients remains insufficient because this simple addition could skew the FedLA aggregation, leading to inaccurate model updates and degraded performance.
- **Client selection in SA for FedLA:** Another challenge in achieving privacy-preserving FedLA through the use of SA consists of addressing client selection at each round. Indeed, in SA, the aggregation key usually depends on all clients’ individual keys, and at each FL round, SA should still be correct while non-selected clients’ keys are not included. Since, in FedLA, the distribution of clients is assumed to be non-IID and this is reflected through the use of FedLA weights, these weights of non-selected clients also need to be omitted while still achieving SA correctly.

3.3.3 Our Solution - Overview

Our design goals address each of the three challenges identified in the previous section.

- **Privacy-Preserving FedLA-weight Computation:** The new solution should incorporate two privacy layers: one to compute each client’s

FedLA-weight $\alpha_{FedLA,u}$ and one to compute global (aggregated) model parameters w_τ during the actual FL round. To compute $p_{FedLA,u}$ in a privacy-preserving manner, we propose that each client u protects the local label distribution $s_{u,l}$ through a standard SA framework and transfers it to the FL server. The server aggregates these received values from all clients to calculate the overall weight per label s_l , which is subsequently returned to each client. Clients can now individually determine their $p_{FedLA,u}$ weight relative to the global label distribution as described in Section 3.2.1, and this, without having access to other clients label weights $s_{v,l}$ in cleartext. These weights should be computed only once in the *setup* step and require only one communication round. Once weights are determined, clients can proceed to the SA of the weighted model parameters.

- **Fault-tolerant FedLA :** Regarding the client dropout in FedLA, because the inputs of each client are not independent of each other due to $\alpha_{FedLA,u}$, to address client dropouts, we propose a simple, yet effective, fault-tolerant SA solution based on a variant of the TJJL scheme proposed in [22]. In this new solution named TJJL-A, each online client sends the difference between their current and previous round weighted inputs. Once the server receives all inputs, the result will correspond to the aggregation of online clients’ weighted parameters together with the previous model parameters multiplied by the remaining weight (corresponding to the dropped clients). Hence dropout clients’ inputs become previous model’s parameters that are properly weighted.
- **Handling client selection in SA for FedLA :** To handle non-selected clients, we apply the same mechanism used for dropped clients. Thus, non-selected clients are treated as dropped clients during the aggregation process. .

3.4 Related Work

We study the related work according to the three challenges that we have identified in Section 3.3.

- Many SA schemes have been proposed for FL [54]: The majority of them unfortunately omit the case with non-IID clients (first challenge) and use the standard FedAvg solution to obtain the aggregated model. Such solutions become ineffective in the non-IID case.

- Regarding client dropouts as in challenge two, existing solutions either (i) rely on online clients to reconstruct offline clients’ inputs [55, 22] or, (ii) define dedicated parties such as *decryptors* in [52], to ensure aggregation correctness, or (iii) consider online clients’ inputs only and define aggregation keys on the fly [53] (this also seems to address client selection). SAAFL, relies on online clients but to achieve a good level of accuracy in the non-IID context, the reconstructed value for dropped clients is not zero but a customized one. Recently, [56] develops a FL scheme that uses a dedicated polynomial integer neural network to address both non-IID distribution and client dropouts.
- Regarding client selection in challenge three, most of the existing solutions, including [56], assume that client selection has already happened and usually omit this important operation, whereas in SAAFL, non-selected clients are considered as dropouts too. Finally, the very recent work in [57] considers the use of DP mechanisms for non-IID clients. This work can be considered as independent as it does not implement SA. A combination of this work with SAAFL can be an interesting future work direction.

3.5 TJL-A: A Label-Aware Threshold Secure Aggregation Scheme

We propose a new variant of the TJL scheme originally proposed in [22] that helps SAAFL take into account the FedLA weights of dropped clients during aggregation. The main difference between the newly proposed TJL-A scheme and TJL is that when FL clients drop, their inputs are not replaced with zero-values anymore. Indeed, the original TJL, explained in Chapter 2 Section 2.3.2, reconstructs the zero-value encrypted with the dropped clients’ individual keys. As also shown in Figure 3.3 in Section 5.6, in the context of a non-IID distribution of FL clients, aggregating a zero-value for dropped clients significantly impacts the accuracy of the actual model. Therefore, we propose to replace this zero-value with a previous round’s model parameters weighted with a specific coefficient. This carefully selected value compensates for the missing contributions of the dropped clients more accurately and ensures the correctness of the aggregation at the same time. Furthermore, the initial contribution of each FL client becomes a weighted differences of their model parameters and the previous global model parameters: $x_{u,\tau} = p_{FedLA,u}w_{u,\tau} - p_{FedLA,u}w_{\tau-1}$. Therefore, at the aggregation phase, the server will reconstruct a weighted average of online clients’ inputs and previous

model parameters for dropped clients.

The TJL-A scheme consists of the following algorithms:

- **TJL – A.Setup**($1^\lambda, n, t$) : Given security parameter λ , the total number of clients n and threshold $t \leq n$, the algorithm generates two equal-size large prime numbers p, q and compute modulus $N = p \cdot q$. Then, it calls **SS.Setup**($1^\lambda, t, n, \mathbb{Z}_\mathbb{N}$) $\rightarrow pp_{SS}$, **ISS.Setup**($1^\lambda, t, n, [-N^2, N^2]$) $\rightarrow pp_{ISS}$. It also sets a cryptographic hash function $H : \mathcal{T} \rightarrow \mathbb{Z}_{N^2}^*$ where $\mathcal{T}$ denotes the round space. Next, the algorithm samples the following two elements:

$$ek_u \leftarrow_R [-N^2, N^2] \text{ for all } u \in \mathcal{U},$$

$$ak_0 = - \sum_{u \in \mathcal{U}} ek_u,$$

and secret shares ek_u using **ISS.Share**($ek_v, t, \mathcal{U}$) $\rightarrow \{u, [\Delta ek_v]_u\}_{u \in \mathcal{U}}$ for all $v \in \mathcal{U}$. Finally, it sets

$$pp = (n, t, N, H, pp_{SS}, pp_{ISS}, ak_0),$$

$$ak_u = \{[\Delta ek_v]_u\}_{v \in \mathcal{U}} \text{ for all } u \in \mathcal{U}.$$

- **TJL – A.Protect**($pp, \tau, ek_u, x_{u,\tau}$) $\rightarrow Y_{u,\tau} := (y_{u,\tau}^{(S)}, \{y_{u,\tau}^{(v)}\}_{v \in \mathcal{U}})$: On inputs round τ the encryption key ek_u of client $u \in \mathcal{U}$, and message $x_{u,\tau}$, the algorithm samples $B_{u,\tau} \leftarrow_R \mathbb{Z}_N$, and then computes

$$y_{u,\tau}^{(S)} = (1 + (B_{u,\tau} + x_{u,\tau}) \cdot N) \cdot H(\tau)^{ek_u} \mod N^2,$$

$$\text{SS.Share}(B_{u,\tau}, t, \mathcal{U}) \rightarrow \{u, [B_{u,\tau}]_v\}_{v \in \mathcal{U}}.$$

Finally, it sets $y_{u,\tau}^{(v)} = [B_{u,\tau}]_v$ for all $v \in \mathcal{U}$.

- **TJL – A.Agg**($pp, \tau, \{ak_u\}_{u \in \mathcal{U}_\tau^{on}}, \{Y_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}})$ $\rightarrow x_{agg,\tau}$: Given round τ , ciphertexts $\{Y_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}}$, aggregation keys $\{ak_u\}_{u \in \mathcal{U}_\tau^{on}}$ of users in $\mathcal{U}_\tau^{on}$, this algorithm invokes:

- **Client.TJL – A.Agg**($pp, \tau, ak_u, \{y_{v,\tau}^{(u)}\}_{v \in \mathcal{U}_\tau^{on}})$ $\rightarrow ak_{u,\tau}$: On input the tag τ , the client u 's aggregation key ak_u , and the set of ciphertexts $\{y_{v,\tau}^{(u)}\}_{v \in \mathcal{U}_\tau^{on}}$ corresponding to the online clients $v \in \mathcal{U}_\tau^{on}$, first defines the set $\mathcal{U}^{off} = \mathcal{U} \setminus \mathcal{U}_\tau^{on}$. It then uses the shares of the secret keys of the offline clients in $\mathcal{U}^{off}$ to compute the

encryption of a non-zero value $x'_{u,\tau}$, which accounts for the cryptographic contributions of the dropped-out clients.

$$[y_\tau^{off}]_u = (1 + Nx'_{u,\tau})^{\Delta^2 \mu_u^{-1}} \cdot H(\tau)^{\sum_{v \in \mathcal{U}_\tau^{off}} [\Delta \cdot ek_v]_u}.$$

Then, it sets the temporary aggregation key $ak_{u,\tau} = ([y_\tau^{off}]_u, \{[B_{v,\tau}]_u\}_{v \in \mathcal{U}_\tau^{on}})$ and sends it to the server.

- **Server.TJL – A.Agg**($pp, \{ak_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}}, \{y_{u,\tau}^{(S)}\}_{u \in \mathcal{U}_\tau^{on}} \rightarrow C_\tau$: On inputs the temporary aggregation keys $\{ak_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}}$ and the set of ciphertexts $\{y_{u,\tau}^{(S)}\}_{u \in \mathcal{U}_\tau^{on}}$, the server first computes

$$y_\tau^{off} = \prod_{u \in \mathcal{U}_\tau^{on}} ([y_\tau^{off}]_u)^{\mu_u},$$

where μ_u is as described in Definition 2.2.8. Then, it computes

$$y_\tau = \left(\prod_{u \in \mathcal{U}_\tau^{on}} y_{u,\tau}^{(S)} \right)^{\Delta^2} \cdot y_\tau^{off}.$$

Then, using the aggregation key ak_0 , the server computes

$$V_\tau = H(\tau)^{\Delta^2 \cdot ak_0} \cdot y_\tau,$$

$$C_\tau = \frac{V_\tau - 1}{N \cdot \Delta^2} \mod N.$$

Finally, using the secret shares of $B_{u,\tau}$, it computes

$$\{B_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}} \leftarrow \text{SS.Combine}(\{v, [B_{u,\tau}]_v\}_{v \in \mathcal{U}_\tau^{on}}),$$

$$x_{agg,\tau} = C_\tau - \sum_{u \in \mathcal{U}_\tau^{on}} B_{u,\tau} \mod N.$$

3.5.1 Correctness

Given a set of clients $\mathcal{U}$, an online client set $\mathcal{U}_\tau^{on} \subseteq \mathcal{U}$, and an offline client set $\mathcal{U}_\tau^{off} = \mathcal{U} \setminus \mathcal{U}_\tau^{on}$, the correctness of TJL-A is defined as follows. For all security parameters λ , number of clients n , threshold t , rounds τ , client inputs $x_{u,\tau}$, and the non-zero value $x'_{u,\tau}$, such that $|\mathcal{U}_\tau^{on}| \geq t$, we have

$$\Pr \left[\begin{array}{l} \text{TJL-A.Setup}(1^\lambda, n, t) \rightarrow (pp, \{ak_u\}_{u \in \mathcal{U}}), \\ \text{TJL-A.Protect}(pp, \tau, ek_u, x_{u,\tau}) \rightarrow Y_{u,\tau}, \\ \text{TJL-A.Agg}(pp, \tau, \{ak_u\}_{u \in \mathcal{U}_\tau^{on}}, \{Y_{u,\tau}\}_{u \in \mathcal{U}_\tau^{on}}) \rightarrow x_{\text{agg},\tau}, \\ x_{\text{agg},\tau} \neq \sum_{u \in \mathcal{U}_\tau^{on}} x_{u,\tau} + \sum_{u \in \mathcal{U}_\tau^{off}} x'_{u,\tau} \end{array} \right] = 0.$$

3.5.2 Security

The TJL-A scheme provides Aggregator Obliviousness security under the DCR assumption in the random oracle model, provided that the number of corrupted clients is less than the threshold t .

Proof. The security of TJL-A mainly relies on the security of the underlying Joye–Libert secure aggregation scheme, which is proven secure under the DCR assumption 2.3.1.1. As in the original scheme, we assume that each client encrypts at most one input for each round. In addition, dropout client does not provide an encrypted input for that round, while the online clients construct at most one encrypted value on behalf of each dropout client using the corresponding secret-key shares and its non-zero replacement value.

The confidentiality of the individual client inputs follows from the DCR assumption underlying the Joye–Libert encryption [58]. Moreover, the secret sharing mechanism prevents a coalition of fewer than t corrupted clients from reconstructing the secret keys of the offline clients or the masking values required to recover their corresponding encrypted contributions. Since the secret sharing scheme over integers is statistically secure, fewer than t corrupted clients cannot reconstruct these secrets.

Consequently, the aggregator obtains only the aggregate of the online clients' inputs together with the non-zero contributions associated with the dropout clients, but cannot recover any individual input or non-zero value. Therefore, TJL-A provides Aggregator Obliviousness in the random oracle model under the DCR assumption, as long as fewer than t clients are corrupted.

3.6 SAAFL: Secure Aggregation for Label-Aware Federated Learning

In this section, we describe SAAFL, which uses TJL-A to compute each client’s FedLA weight in a privacy-preserving manner, along with a traditional fault-tolerant SA scheme for aggregation. SAAFL also makes use of a pseudo-random generator (PRG), key agreement scheme (KA), authenticated encryption (AE), and the Shamir’s secret sharing Scheme (SS) . These cryptographic primitives are described in Chapter 2. Before describing SAAFL, we first define the threat model considered in this work.

3.6.1 Overview

SAAFL consists of two main phases: (1) a Setup Phase, in which FL clients are registered, cryptographic keying material is established, and the FedLA aggregation weights are securely computed; and (2) a Training Phase, in which clients are selected, their model updates are protected, and the weighted model updates are securely aggregated over multiple FL rounds.

3.6.2 Setup Phase

The protocol starts with the Setup Phase which comprises three steps: Registration, Key Setup and FedLA-weight Calculation. Each of these steps is executed only once.

3.6.2.1 Registration

During the Registration step, a trusted dealer TD generates the encryption keys ek_u for each client and aggregation key ak_0 for the server along with relevant public parameters pp required by the clients and server. Each client generates public-private key pair over the pp using KA.Gen and registers itself by sending the public key to the server.

3.6.2.2 Key Setup

At the Key Setup step, once each client receives the list of all registered clients together with their public keys, they establish pairwise keys with each of them and use SS.Share to generate the shares of their individual TJL-A key ek_u .

3.6.2.3 FedLA-weight Calculation

Finally, at the FedLA-weight Calculation step, each FL client u encrypts $s_{u,l}$ of data samples with label l with the TjL-A encryption key ek_u (generated in the previous Key Setup step) and sends them to the FL server. The server performs their aggregation using **TjL-A.Agg** and sends s_l of samples with label l to all FL clients. Then, each client calculates its FedLA-weight $\alpha_{FedLA,u}$ (see Algorithm 3.2.1). Since the FedLA weights are computed only once during the setup phase, we assume that all clients are available at this stage. Consequently, no client dropout needs to be handled during this computation, and TjL-A effectively reduces to the standard SA setting for this step.

SAAFL - Setup Phase

Registration: $|\mathcal{U}| = n$. $|\mathcal{L}| = k$.

– **Trusted Dealer (TD):**

1. $\text{KA.Param}(1^\lambda) \rightarrow pp^{\text{KA}}$ and $\text{TJL-A.Setup}(1^\lambda, n, t) \rightarrow (pp, \{ek_u\}_{u \in \mathcal{U}})$.
2. Set $pp = (n, t, N, H, pp_{\text{SS}}, pp_{\text{ISS}}, ak_0)$
3. Send (pp, ek_u) to each client u and (pp, ak_0) to server.

– **Clients** ($\forall u \in \mathcal{U}$):

1. Receive (pp, ek_u) from TD.
2. $\text{KA.Gen}(pp^{\text{KA}}) \rightarrow (c_u^{\text{PK}}, c_u^{\text{SK}})$: Generate a key pair and send c_u^{PK} to the server.

– **Server:**

1. Receive (pp, ak_0) from TD and c_u^{PK} from each client $u \in \mathcal{U}$.
2. Broadcast to all clients the list $\{u, c_u^{\text{PK}}\}_{u \in \mathcal{U}}$.

Key Setup:

– **Clients** ($\forall u \in \mathcal{U}$):

1. Collect $\{v, c_v^{\text{PK}}\}_{v \in \mathcal{U}}$ of all registered clients.
2. $\forall v \in \mathcal{U} \setminus \{u\}$, $\text{KA.Agree}(pp^{\text{KA}}, c_u^{\text{SK}}, c_v^{\text{PK}}, H) \rightarrow c_{u,v}$: Establish pairwise shared keys.
3. $\text{SS.SKShare}(ek_u, t, \mathcal{U}) \rightarrow \{(v, [ek_u]_v)\}_{v \in \mathcal{U}}$: Generate t -out-of- n shares of ek_u
4. $\forall v \in \mathcal{U} \setminus u$, $\text{AE.Enc}(c_{u,v}, u || v || [ek_u]_v) \rightarrow \epsilon_{u,v}$: Encrypt each share with $c_{u,v}$ and send $\{(u, v, \epsilon_{u,v})\}_{v \in \mathcal{U}}$ to server.

– **Server:**

1. Collect and forward $\{(u, v, \epsilon_{u,v})\}_{v \in \mathcal{U}}$ to registered clients.

FedLA-weight Calculation:

– **Clients** ($\forall u \in \mathcal{U}$):

1. $\text{TJL-A.Protect}(pp, \tau_0, ek_u, s_{u,l}) \rightarrow Y_{u,l}$: Protect the local label distribution $s_{u,l}$ and send $Y_{u,l}$ to the server.

– **Server:**

1. Collect all $Y_{u,l}$ from clients.
2. $\forall l \in \mathcal{L}$, $\text{TJL-A.Agg}(pp, \tau, \{ak_u\}_{u \in \mathcal{U}}, \{Y_{u,l}\}_{u \in \mathcal{U}}) \rightarrow s_l$: Aggregate all $Y_{u,l}$ to obtain the total number s_l of samples with label l .

– **Clients** ($\forall u \in \mathcal{U}$):

1. Compute $p_{\text{FedLA},u}$ (Algorithm 3.2.1).

Figure 3.1: Detailed description of the *Setup Phase* of SAAFL

3.6.3 Training Phase

The training phase consists of the execution of several FL Rounds until reaching a certain level of accuracy. Each FL Round consists of two steps: Protection and Aggregation. At the Protection step, all selected clients encrypt their inputs and send them to the server. Then, at the Aggregation step, clients interact with the server to account for the dropped and non-selected clients so that the server can perform the aggregation correctly and accurately. Each FL round begins with the selection of clients $\mathcal{U}_\tau \subseteq \mathcal{U}$ that will participate in the aggregation, and this is performed based on selection ratio k_τ as in the FedLA [48] and ensures that all parties are aware of which clients will participate in the current FL Rounds τ in advance.

3.6.3.1 Protection

At the *Protection* step, after training the local models, each client generates a random seed $b_{u,\tau}$ to generate a blinding mask $B_{u,\tau}$ for masking its input $x_{u,\tau}$ and then encrypts it using **TJL – A.Protect**. These blinding masks are used in the same way as in [22, 55] to protect the privacy of the clients' inputs in case the server receives the protected values of a given client (usually a dropout clients) twice using the same τ . Each client then secretly shares seed $b_{u,\tau}$ with other clients using **SS.Share** through the server and transmits its protected input $y_{u,\tau}$ masked with $B_{u,\tau}$ to the server.

3.6.3.2 Aggregation

At the *Aggregation* step, the clients receive the list of encrypted shares of blinding mask seeds $b_{u,\tau}$ and deduce the list of online clients $\mathcal{U}_\tau^{\text{on}}$ from those shares. Online clients encrypt the relevant value $x'_{u,\tau}$ for non-selected and dropped clients $\mathcal{U}_\tau^{\text{off}} = \mathcal{U} \setminus \mathcal{U}_\tau^{\text{on}}$ with **Client.TJL – A.Agg** using key-shares of these clients. This is to ensure that the FedLA-weights of non-selected and dropped clients also appear in the final aggregation as discussed in Section 3.3.3. Then, each online client sends to the server the shares of the blinding mask seeds for other online clients and the share of the protected non-zero value $[y_\tau^{\text{off}}]_u$ for the non-selected and dropped clients. The server reconstructs the blinding masks using **SS.Combine** and **PRG**. Finally, the server computes the masked sum C_τ of the weighted inputs of online clients $x_{u,\tau}$ and the approximated value $x'_{u,\tau}$ for non-selected and dropped clients with **Server.TJL – A.Agg** and removes the clients' blinding masks to obtain the aggregated model $x_{\text{agg},\tau}$ for *FL Round* τ .

SAAFL - FL Round (τ)

Client Selection: Clients are selected based on the parameter k_τ and previous round model $w_{\tau-1}$. Let $\mathcal{U}_\tau \subseteq \mathcal{U}$ represent the set of selected clients in the current FL round τ .

Protection:

- **Clients** ($\forall u \in \mathcal{U}_\tau$):
 1. $\mathbb{F} \rightarrow_R b_{u,\tau}$: Sample a seed for PRG.
 2. $\text{PRG}(b_{u,\tau}) \rightarrow B_{u,\tau}$: Extend the seed using **PRG** to produce the blinding mask.
 3. $p_{FedLA,u} w_{u,\tau} - p_{FedLA,u} w_{\tau-1} \rightarrow x_{u,\tau}$: Compute client u 's input.
 4. Executes **TJL-A.Protect**($pp, sk_u, \tau, x_{u,\tau} + B_{u,\tau}$): which mask and protect the input $x_{u,\tau}$ and also run **SS.Share**($b_{u,\tau}, t, \mathcal{U}_\tau$) $\rightarrow (v, [b_{u,\tau}]_v)_{v \in \mathcal{U}_\tau}$, which generates t -out-of- n shares of $b_{u,\tau}$.
 5. $\forall v \in \mathcal{U}_\tau \setminus u$, $\text{AE.Enc}(c_{u,v}, u || v || [b_{u,\tau}]_v) \rightarrow e_{(u,v),\tau}$: Encrypt shares.
 6. If any of the above steps fails, abort. Otherwise, send $Y_{u,\tau} = (y_{u,\tau}, \{e_{(u,v),\tau}\}_{v \in \mathcal{U}_\tau})$ to the server.
- **Server:**
 1. Collect $(\{e_{(u,v),\tau}\}_{v \in \mathcal{U}_\tau}, y_{u,\tau})$ from remaining clients.
 2. Define the set of online clients $\mathcal{U}_\tau^{\text{on}} \subseteq \mathcal{U}_\tau$ based on received tuples. If $|\mathcal{U}_\tau^{\text{on}}| < t$, then abort. Otherwise, send $\{(u, v, e_{(u,v),\tau})\}_{u,v \in \mathcal{U}_\tau^{\text{on}}}$ to online clients.

Aggregation:

- **Clients** ($\forall u \in \mathcal{U}_\tau^{\text{on}}$):
 1. Receive $\{(v, u, e_{(v,u),\tau})\}_{v \in \mathcal{U}_\tau^{\text{on}}}$ from the server and deduce $\mathcal{U}_\tau^{\text{on}}$.
 2. Execute **Client.TJL - A.Agg**($pp, \tau, \{[\Delta ek_v]_u\}_{v \in \mathcal{U}_\tau^{\text{off}}}, \{Y_{u,\tau}\}_{v \in \mathcal{U}_\tau^{\text{on}}}$: which protects $x'_{u,\tau} = \frac{1}{|\mathcal{U}_\tau^{\text{on}}|} w_{\tau-1}$ with client u 's key shares of non-selected and dropped clients and outputs $[y_\tau^{\text{off}}]_u$.
 3. $\forall v \in \mathcal{U}_\tau^{\text{on}} \setminus \{u\}$, $\text{AE.Dec}(c_{u,v}, e_{(v,u),\tau}) \rightarrow v' || u' || [b_{v,\tau}]_u$: Decrypt secret shares. Assert that $v' = v$ and $u' = u$.
 4. If any of above steps fails, abort. Otherwise, send $ak_{u,\tau} = ([y_\tau^{\text{off}}]_u), \{[b_{v,\tau}]_u\}_{v \in \mathcal{U}_\tau^{\text{on}}}$ to the server.
- **Server**
 1. Collect $(\{[b_{v,\tau}]_u\}_{v \in \mathcal{U}_\tau^{\text{on}}}, [y_\tau^{\text{off}}]_u)$ from online clients.
 2. Execute **Server.TJL - A.Agg**($pp, \{ak_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{on}}}, \{y_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{on}}}) \rightarrow C_\tau$ as shown in 3.5.
 3. $\forall u \in \mathcal{U}_\tau^{\text{on}}$, **SS.Combine**($\{[b_{u,\tau}]_v\}_{v \in \mathcal{U}_\tau^{\text{on}}}, t) \rightarrow b_{u,\tau}$: Reconstruct blinding mask seeds.
 4. $\forall u \in \mathcal{U}_\tau^{\text{on}}$, $\text{PRG}(b_{u,\tau}) \rightarrow B_{u,\tau}$: Reconstruct the blinding masks.
 5. $x_{agg,\tau} = C_\tau - \sum_{v \in \mathcal{U}_\tau^{\text{on}}} B_{u,\tau} = \sum_{u \in \mathcal{U}_\tau^{\text{on}}} x_{u,\tau} + \sum_{u \in \mathcal{U}_\tau^{\text{on}}} x'_{u,\tau}$: Calculate the final result by removing the blinding masks.

Figure 3.2: Detailed description of the FL Round in SAAFL

3.6.4 Security

SAAFL guarantee the privacy of the clients' inputs against the passive and active adversaries. We sketch the proofs following [22].

3.6.4.1 Security in the Passive Model

In the honest-but-curious model, the server follows the protocol but colludes with up to $n - t$ clients. Let C represent the corrupted clients, and $\mathcal{C} = C \cup \mathcal{S}$ where $\mathcal{S}$ is the server. The view of $\mathcal{C}$ is computationally indistinguishable from a simulated view if $|C| = n - t < t$. Therefore, $t > n/2$, allowing recovery from up to $n/2 - 1$ client failures. This relies on the security of the underlying cryptographic primitives. KA ensures that entities in $\mathcal{C}$ cannot distinguish the honest clients' pairwise keys from random values. The TJL-A scheme ensures that protected inputs and honest clients' shares appear random if entities in $\mathcal{C}$ have fewer than t shares. SS (and ISS) guarantees that entities in $\mathcal{C}$ cannot distinguish the shares from random values if they have no more than $t - 1$ shares.

3.6.4.2 Security in the Active Model

In the active model, the server can manipulate its inputs. The server only distributes clients' public keys and encrypted shares, which it cannot alter due to the AE scheme. The server's power is limited to withholding shares, potentially misleading clients about which are online. Importantly, the server can provide different views to clients, convincing some to send a protected approximation $[y_\tau^{off}]_u$ for client u , while others send shares of the blinding mask $b_{u,\tau}$. If the server obtains both $[y_\tau^{off}]_u$ and $b_{u,\tau}$ for the same client u in round τ , it can unmask the input. Colluding with $n - t$ corrupted clients, the server can gain $n - t$ shares of $[y_\tau^{off}]_u$ and $b_{u,\tau}$. For the remaining t honest clients, it can manipulate half to send shares of $[y_\tau^{off}]_u$ and the other half shares of $b_{u,\tau}$, learning up to $n - t + t/2$ shares. Hence, $n - t + t/2 < t$, implying $t > 2n/3$, allowing recovery from up to $n/3 - 1$ client failures.

3.7 Experimental Evaluation

In this section, we experimentally evaluate the proposed SAAFL framework and the underlying TJL-A protocol. The evaluation focuses on the effectiveness of the proposed approach in supporting secure and privacy-preserving federated learning with label-aware aggregation. We compare SAAFL with relevant baseline approaches under different data distributions and system

settings. In particular, we examine the impact of data heterogeneity, privacy protection, and system parameters on the model performance and communication and computational costs. The experimental results demonstrate the practicality of the proposed approach and highlight the trade-offs between privacy, utility, and efficiency.

3.7.1 Experimental Setup

We have implemented SAAFL² and to evaluate its performance, experiments are conducted on a machine equipped with an AMD EPYC 7272 processor, 128 GB of RAM, and 4 NVIDIA GeForce RTX 3090 graphics cards. The underlying cryptographic primitives consist of the same building blocks and libraries as in [22]. We use the EMNIST-balanced dataset [59] with $k = 47$ labels, each of which represented by 2400 samples. This is one of the two datasets utilized in [48] to evaluate the original FedLA. We also employ the same CNN model of [48] which consists of two convolutional layers, one dropout layer, and two linear layers. The model is configured with a batch size of 256 and a learning rate of 0.01.

To evaluate the accuracy of the resulting model, similar to [48], the EMNIST-balanced dataset is distributed among $n = |\mathcal{U}| = 10$ clients according to two parameters: the number of unique labels that each non-IID client has, denoted as $unique_c$ and $noniid_s$ which is the proportion of non-IID clients. We vary $unique_c$ across $\{1, 2, 3, 4, 6\}$ and $noniid_s$ across $\{0.1, 0.3, 0.5, 0.7\}$. Each IID client holds 2400 samples while each non-IID client holds all samples of the labels to which it is assigned: e.g., if $unique_c = 2$, each non-IID client holds $2 \times 2400 = 4800$ samples. At each FL round, we define the selection ratio $k_\tau \in \{0.7, 0.8, 0.9\}$. Each setting is evaluated by running experiments over 100 FL rounds each with 10 local epochs. In our protocol, non-selected clients are treated as dropped. We set the TJL-A threshold to $t = 6$ and allow a maximum dropout rate f of 0.1, 0.2, or 0.3 when k_τ is 0.7, 0.8, or 0.9, respectively. We further evaluate the performance of SAAFL in terms of computation and communication while considering 100 users and $k_\tau = 0.8$, $f = 0.2$

3.7.2 Experimental Results

First, we compare testing accuracy of SAAFL under a non-IID setting with that of FedAvg, original FedLA, and FedLA-FTSA [22] which executes TJL

²The code is available in <https://github.com/Aftab201/SAAFL.git>.

with dropout clients' inputs set to zero. We evaluate worst-case scenario for SA, i.e., a maximum dropout rate that each method can handle.

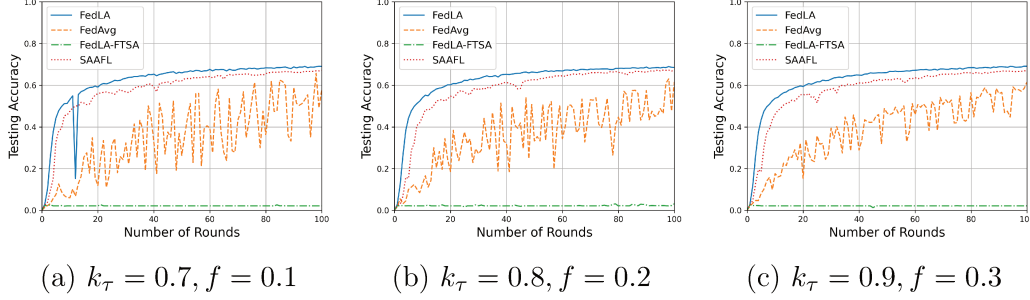


Figure 3.3: Testing accuracy for FedLA, FedAvg, FedLA-FTSA, and SAAFL with $unique_c = 1$, $noniid_s = 0.7$.

Since FedLA has been proved to have superior accuracy to FedAvg in data distribution in which the non-IID clients are the majority [48], we choose to discuss the results of the experiments in which $unique_c = 1$ and $noniid_s = 0.7$. As shown in Figure 3.3 and as expected, FedAvg suffers more unstable performance than FedLA and SAAFL because of the highly non-IID data distribution. We also observe that FedLA-FTSA yields worst performance mainly because dropped clients' inputs are considered as zero and this does not help achieve aggregation correctness in FedLA. Meanwhile, SAAFL achieves almost as good accuracy level as FedLA thanks to the proposed TJL-A scheme which sets dropped clients' inputs as model parameters of the previous round weighted with specific coefficients. Furthermore, in Figure 3.3a, FedLA has significant accuracy drop in one early round in which all selected clients have non-IID data distribution. This may happen when proportion of non-IID clients $noniid_s$ is equal to the selection ratio k_τ . On the contrary, SAAFL avoids this issue by adding to FedLA-weighted average $\sum_{u \in \mathcal{U}_\tau^{qn}} p_{FedLA, u} w_{u, \tau}$ an approximated value $\left(1 - \sum_{u \in \mathcal{U}_\tau^{qn}} p_{FedLA, u}\right) w_{\tau-1}$ that compensates for the impact of non-IID clients.

We finally evaluate the performance of SAAFL in terms of computation time and communication cost. We set the number of clients to 100. As shown in Table 3.1, with $k_\tau = 0.8$, and $f = 0.2$, each client takes 249 ms to run the Key Setup whereas the server takes 1.55 ms. The calculation of FedLA-weights, takes 38 ms at the client and 1085 ms at the server. Finally, at each FL Round, each client takes 6463 ms for the SA of model updates and the server, 46475 ms. The data transfer at each client in Key Setup, FedLA-weight Calculation, and FL Round are measured as 14.27 KB, 14.64 KB, and 24.88 KB, respectively.

Table 3.1: Computation and Communication Cost of SAAFL with $n = 100$, $k_\tau = 0.8$, $f = 0.2$.

SAAFL	Server	Client	BW
Key Setup	1.55 ms	249 ms	14.27 KB
FedLA-weight Calculation	1085 ms	38 ms	14.64 KB
FL Round	46457 ms	6463 ms	24.88 KB

When compared with the solution in [22], SAAFL incurs the same complexity, namely: $O(n + m)$ for the client and $O(n^2 + nm)$ for the server both for communication and storage, and $O(n^2 + m)$ for the client and $O(n^2 + nm)$ for the server in terms of computation. (n and m being the number of clients and the number of model parameters, respectively).

3.8 Conclusion

To address **C1: Privacy, Dropout Robustness, and Data Heterogeneity (R1 + R2 + R3)**, We have developed a new privacy-preserving FL framework named SAAFL that is suitable to environments where clients datasets are non-IID and runs in two communication rounds, only. SAAFL is based on a customized SA protocol **TJL-A** where the dropped clients' shares depend on the previous round's parameters. Thanks to **TJL-A** and as shown by our experimental results, SAAFL exhibits an acceptable level of accuracy.

Chapter 4

Privacy-preserving Federated Unlearning

This chapter addresses the second challenge, **C2**, by jointly considering **privacy protection**, **client dropout robustness**, and **legal compliance**, in particular the 'right to be forgotten', in Federated Learning (FL). Federated Unlearning (FU) aims to remove the influence of a client's data from a trained global model thereby supporting the right to be forgotten under the GDPR [14]. Secure Aggregation (SA), while protecting individual client updates from the server, also hides the contributions needed to remove a specific client's influence from the global model. To address this challenge, we propose a privacy-preserving FU (PPFU) approach that combines an improved FU method with a novel extension of SA that supports unlearning even in the presence of client dropouts.

The work presented in this chapter is currently under review.

4.1 Introduction

FU aims to remove the influence of a client's data from a trained global model without requiring complete retraining. To achieve this, the unlearning mechanism needs to identify the client's contributions made during training, including the rounds in which it participated. Exposing such information to the server reveals information about the client's training history and therefore conflicts with the privacy requirements of FL, as discussed in Chapter 1.

SA provides privacy protection for clients' updates but creates an additional challenge for privacy-preserving FU: once the updates have been securely aggregated, the server cannot directly isolate and remove the contribution of a specific client. Instead, removing a client's contribution may

require the remaining clients to reconstruct an aggregate that excludes the client requesting unlearning. This process becomes more challenging when some of these clients become unavailable, as the information required for reconstruction may be incomplete, preventing the unlearning process from being completed. Consequently, supporting FU together with SA framework requires a mechanism that can remove a client’s contribution without accessing its individual update while remaining robust to client dropouts.

To address these challenges jointly, we extend SA with a new **Forget** functionality that enables the server to remove a client’s contribution from a previously computed aggregation without revealing individual inputs or additional information beyond the updated aggregate. We further combine this functionality with DP mechanism [30] to limit information leakage from the updated aggregate. To build a complete privacy-preserving FU solution, we improve the existing FU approach, named FedRecovery [1] and combine it with our SA scheme supporting the **Forget** functionality. The improved FedRecovery enhances the accuracy of the unlearned model by reducing the required DP noise through adaptive gradient clipping.

Our contributions can be summarized as follows:

- We first identify that the most relevant baseline for our FU use case is **FedRecovery** proposed in [1]. FedRecovery erases the impact of a client by removing a weighted sum of gradient residuals from the model and adjusts DP noises to make the unlearned model and retrained model statistically indistinguishable. We then propose an improved version of FedRecovery, called **FedRecovery+**, which improves the accuracy of the unlearned model by significantly reducing the size of the DP noise added.
- We extend the paradigm of SA with a **Forget** functionality to allow for forgetting the input of some client without leaking any additional information than the new aggregate, which is then blurred by some arbitrarily chosen DP noise. We show that this extension is generic and can be applied to any SA scheme.
- Finally, we build a new privacy-preserving FU scheme, **PPFU** for short, instantiated by combining FedRecovery+ and the Forget functionality of SA. The solution is provably secure against an honest-but-curious adversary.
- We implement and assess our solution through extensive benchmarks, by evaluating it in terms of computation time, unlearning quality and accuracy of the model and comparing it with baseline solutions.

4.2 Related Work

FU can target different types of information that contributed to the training of a global model. Recent survey [60] distinguish three main unlearning objectives:

- Sample unlearning, which removes the influence of specific data samples;
- Class unlearning, which removes the influence of data associated with a particular class;
- Client unlearning, which removes the influence of all data contributed by a specific client.

In this work, we focus on client-level FU, where the objective is to remove the contribution of an entire client’s local dataset from the global model without retraining the model from scratch. This setting is particularly relevant to privacy-related requests in FL, such as a client’s request to withdraw its contribution from the learned model. The following presents the related work on client-level FU, covering approaches based on historical update recalibration, gradient sharing, knowledge distillation, Bayesian federated learning, clustering, and random model perturbation.

Re-calibration of Historical Updates. The first approach consists of the re-calibration of historical updates [61, 62, 63, 64] where the server identifies and removes the target client’s updates from historical rounds and then "re-plays" the training without them. Such an approach requires the server either to track and identify which update is coming from which client or to store/cache individual client contributions over time. Therefore, client privacy is inherently violated.

Gradient Sharing or Inspection. The second category of solutions involves the to-forget client and requires gradient sharing or inspection [65, 66, 67]. Additional information from the gradients themselves is computed by the client itself and further shared with all parties and using this to unlearn the client’s data. Hence, client’s contributions are disclosed.

Knowledge Distillation. Another approach is that of knowledge distillation [68, 69], where a new dataset or labels are synthesized from the original private data and used during the unlearning phase. Despite the fact that the data remains local in this approach, client privacy can also be harmed since

the resulting model updates are still shared, carrying implicit information about the forgotten data.

Bayesian Federated Learning. Other works use the framework of Bayesian federated learning [70, 71], which represents model knowledge as a probability distribution over parameters rather than a single point estimate. These methods still require clients to share variational parameters or particle updates, which reflect their full local dataset and can expose private information.

Clustering-based Unlearning. Finally some solutions use the clustering method [72], where clients are grouped into clusters based on their data distributions. When a client requests unlearning, only the members of its specific cluster participate in the retraining process, while others remain unaffected. The server must still receive and aggregate model updates from the clients within the target cluster. The goal of such solutions is to enhance performance while omitting privacy concerns.

Random Model Perturbation. While the previously mentioned solutions consider model accuracy as the main goal, another group of schemes use random model perturbation to ensure unlearning, inspired by DP [73, 1, 74]. Their goal is to ensure that the unlearned model is statistically close to an ideal model trained from scratch without the leaving client’s data. We observe that these works either require the server to store and identify each client’s past updates, which introduces a privacy risk before DP is applied (in the case of [73, 1]), or rely on the presence of tight gradient clusters [74], such that the added noise significantly perturbs the global model parameters, leading to weaker utility and approximate unlearning guarantees.

Our Approach. To address the privacy risks present in existing FU approaches, we propose a framework in which the server only processes encrypted client data throughout both training and unlearning. Unlike prior work, where the server may access plaintext gradients, or intermediate model states, our method ensures that the server never sees the unencrypted client information. During the unlearning phase, the server performs the required computations on encrypted updates, only, and generates the final unlearned model without learning or inferring anything about the removed data. This design guarantees that both the learning and unlearning phases protect client privacy while maintaining model correctness/accuracy.

4.3 Problem Setting

This section establishes the setting for client-level FU considered in this chapter. We first define the unlearning objective and formally define client-level FU. We then present the design goals and threat model that guide the development of the proposed privacy-preserving unlearning approach. Next, we introduce FedRecovery [1], which serves as the basis for our unlearning mechanism, and describe its limitations in terms of model utility. Finally, we present our improved version of FedRecovery named as FedRecovery+, which is designed to enhance the accuracy of the unlearned model while reducing the amount of DP noise required during the unlearning.

4.3.1 Federated Unlearning

FU [61] is a collaborative learning framework born from FL, in which after training the model for a certain number of rounds, unlearn client u' wishes to withdraw from the model, not only refusing to participate to any future training rounds but also requesting the removal of the influence of its data from the actual model.

More in details, we consider the definition given in [1]. Let $\mathcal{M}_L(\mathcal{U})$ be a FL algorithm trained over data from clients in $\mathcal{U}_\tau$ and $\mathcal{M}_U(w, \Omega, u', \alpha, \beta)$ be a FU algorithm over the global model w , unlearning data from unlearned client u' with Ω denoting the cached information gathered during the training process and unlearning parameters α, β .

Definition 4.3.1 (Client-level (α, β) -Federated Unlearning). *An unlearning algorithm $\mathcal{M}_U$ satisfies (α, β) -federated unlearning with respect to the learning algorithm $\mathcal{M}_L$, if for any possible set of output $W \subseteq \mathbb{R}^d$ and any client $u' \in \mathcal{U}_\tau$,*

$$\begin{aligned} \Pr[\mathcal{M}_L(\mathcal{U}_\tau \setminus \{u'\}) \in W] &\leq e^\alpha \cdot \Pr[\mathcal{M}_U(w, u', \Omega, \alpha, \beta) \in W] + \beta, \\ \Pr[\mathcal{M}_U(w, u', \Omega, \alpha, \beta) \in W] &\leq e^\alpha \cdot \Pr[\mathcal{M}_L(\mathcal{U}_\tau \setminus \{u'\}) \in W] + \beta. \end{aligned}$$

In other words, this is equivalent to asking the distributions coming from $\mathcal{M}_U$ and $\mathcal{M}_L$ without client u' to be (α, β) -indistinguishable [75]. This definition is heavily inspired by differential privacy [30], in which the indistinguishability of distributions coming from datasets differing in one record is used to guarantee the privacy of individual records. However, in the unlearning definition, the indistinguishability of distributions does not give any

guarantee on the privacy of the gradients of any client, but rather to ensure that the unlearned model has correctly removed the influence of the unlearned client’s data.

4.3.2 Divisible Distribution

Divisible Distributions [76] are probability distributions that can be decomposed into multiple independent and identically distributed random variables such that the sum of samples is indistinguishable from a sample from the original distribution. It is formally defined as follows.

Definition 4.3.2 (Divisible Distribution). *Let $\mathcal{D}$ be a probability distribution. We say it is (infinitely) divisible if for any $k \in \mathbb{Z}_{>0}$, there exists some distribution $\mathcal{D}'$ (which we call k -th division of $\mathcal{D}$) such that $\mathcal{D} \sim \sum_{i=1}^k \mathcal{D}'$. A distribution can only be infinitely divisible if it has an infinite support. If it has some finite support of size p (e.g., a distribution over $\mathbb{Z}_p$), then it can only be divisible for $k \leq p$.*

This concept has been widely used in the context of local differential privacy [77], in which the most used distribution are the normal distribution $\mathcal{N}(0, \sigma^2)$ for $\sigma > 0$ (whose k -division is also a normal distribution $\mathcal{N}(0, \sigma^2/k)$) or the two-sided geometric distribution $\text{Geo}(p)$.

4.3.3 Design Goals and Threat Model

The objective of this work is to build a privacy-preserving FU protocol with the following design goals:

- **Feature-wise:** The protocol should allow for efficient FL training, as well as allow for the efficient unlearning of any one client involved at any point during the training.
- **Privacy-wise:** The protocol should ensure that the updates or gradients can only be seen by the client computing them, and the server is only able to see the global averaged models trained by the federated setting.

4.3.3.1 Threat Model

We consider an honest-but-curious FL server that follows the protocol correctly but may attempt to infer information from the data and messages it receives. The FL clients are assumed to be honest, with an honest majority

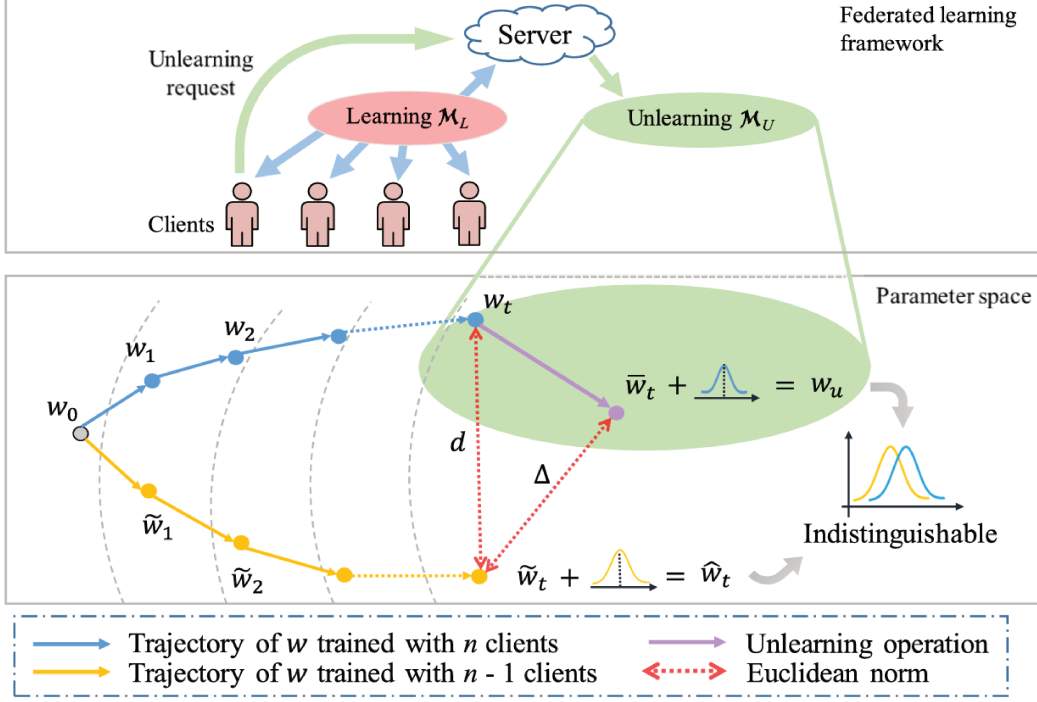


Figure 4.1: Figure explaining the workings of FedRecovery, taken from [1].

among them. We further assume secure communication channels between clients and the server, ensuring that all exchanged messages are encrypted and protected from external adversaries.

4.3.4 FedRecovery

As a starting point for our unlearning mechanism, we use FedRecovery [1]. At a high-level, some noise is added to the global model w trained by all clients, so as to blur the contribution of the unlearned client (up to some unlearning parameters α, β), making the model indistinguishable from an ideal model $\tilde{w}$ trained without the *unlearned* client. To achieve this, the server first precomputes a model $\bar{w}$ removing most of the influence of the client in such a way that the distance between itself and the ideal model $\tilde{w}$ can be easily bounded, so that some carefully calibrated noise can be applied. This setup can be seen in Fig. 4.1.

More in details, at a given round τ , $\bar{w}_\tau$ is computed as:

$$\bar{w}_\tau = w_\tau - \sum_{k=1}^{\tau-1} p_k \cdot \delta_k \quad (4.1)$$

where δ_k is the gradient residual that contains the information submitted by client v on round k , $0 < p_k < 1$ denotes a weighting parameter for δ_k in round τ , and ∇F denotes the average of the gradients among all clients. These terms are computed as:

$$\delta_k := \frac{\eta}{n} \left(\frac{1}{n-1} \sum_{u \in \mathcal{U}_\tau/v} \nabla f_u(w_k) - \nabla f_v(w_k) \right) \quad (4.2)$$

$$p_k := \frac{\|\nabla F(w_k)\|^2}{\sum_{j=1}^{\tau-1} \|\nabla F(w_j)\|^2} \quad (4.3)$$

$$\nabla F(w_k) := \frac{1}{n} \sum_{u \in \mathcal{U}} \nabla f_u(w_k) = \frac{w_k - w_{k+1}}{\eta} \quad (4.4)$$

Finally, some Gaussian noise [47] is added to obtain the unlearned model $w_u = \bar{w}_\tau + z$ where $z \leftarrow \mathcal{N}(0, \sigma^2 \mathbb{I}_d)$ with:

$$\sigma := \frac{1}{\sqrt{2}} \cdot \frac{\|\bar{w}_\tau - \tilde{w}_\tau\|}{\sqrt{\log(1/\beta) + \alpha} - \sqrt{\log(1/\beta)}}, \quad (4.5)$$

where the (α, β) -federated unlearning follows directly from the results in [78], as long as the distance $\|\bar{w}_\tau - \tilde{w}_\tau\|$ can be upper-bounded. In [1], by leveraging L -smoothness of the objective function, as well as the gradients of the objective function being bounded, they achieve the bound

$$\begin{aligned} \|\bar{w}_\tau - \tilde{w}_\tau\| &\leq \left\| \frac{\eta}{n-1} \sum_{k=1}^{\tau-1} \sum_{u \neq u'} \nabla f_u(w_k) - \nabla f_u(\tilde{w}_k) \right\| \\ &\quad + \left\| \sum_{k=1}^{\tau-1} (1 - p_k) \cdot \delta_k \right\|, \end{aligned} \quad (4.6)$$

which in the particular case of batch gradient descent becomes

$$\|\bar{w}_\tau - \tilde{w}_\tau\| \leq 2\sqrt{\tau\eta \cdot F(w_0)}. \quad (4.7)$$

Here the objective function $F(w)$ is defined as $F(w_k) := \frac{1}{n} \sum_{u \in \mathcal{U}} f_u(w_k)$. Since in 4.7 the objective function applies on the starting model w_0 , this bound tends to be very large, resulting in some noise with large standard deviation: adding the noise essentially removes all previous information on the gradients. Authors in [1] claim to start training from an already pre-trained model, without giving any information as to how much this pre-training is¹.

¹To the best of our estimations, to achieve the results reported in [1], the training

4.3.5 FedRecovery+

In this section we propose a new version of the unlearning mechanism FedRecovery, called **FedRecovery+**. The objective is to significantly reduce the size of the noise added to $\bar{w}_\tau$, while maintaining the same privacy guarantees. Indeed, one of the main limitations of the work presented in [1] is the tightness of the bound of $\|\bar{w}_\tau - \tilde{w}_\tau\|$ which directly impacts the standard deviation σ of the noise added to guarantee the unlearning property. We propose three improvements:

- Optimize the residual weights.
- Modify the training through gradient clipping to achieve more fine-grained bound.
- Define round dependent clipping parameters.

4.3.5.1 Tuning the Residual Weights

To start improving Equation 4.6, we first take a look into the weights p_k . In the original unlearning algorithm FedRecovery (see Section 4.3.4), the “residuals” δ_k are multiplied by some weights $0 < p_k < 1$, which become smaller when k increases. The intuition is that the more the model is trained, the less the particular contribution of client u' has impact on the difference between w_k and $\tilde{w}_k$. The exact way these coefficients are computed in [1] (see Equation 4.3) is by computing the weight of the norm of the update (considering all clients) at round k within *all* rounds up to the unlearning round τ . Because of this, after many rounds, the residuals for a client that is to be “unlearned” become very small. This means that the unlearning process becomes less effective over time.

We instead propose to set these to 1 for any round. This way, each round’s residual has the same importance and the contribution from the client we want to unlearn stays strong even in later rounds. In the following Lemma 4.1, we show that taking all weights equal to 1 yields a smaller distance between the unlearned model $\bar{w}_\tau$ and the ideal model $\tilde{w}_\tau$ than any other set of $\{p_k\}_{k \in [\tau-1]}$ with $0 < p_k \leq 1$.

Lemma 4.1. *Let $\tau \in \mathbb{Z}_{>0}$, the unlearned model $\bar{w}_\tau^{p_k}$ be defined as in Equation 4.1 with any set of $\{p_k\}_{k \in [\tau-1]}$ with $0 < p_k < 1$, the ideal model $\tilde{w}_\tau$ be the*

should start on an already converged model as well as running for a very low amount of rounds.

model retrained for τ rounds without the contribution of client u' . Then, we have:

$$\sup \|\bar{w}_\tau^1 - \tilde{w}_\tau\| \leq \sup \|\bar{w}_\tau^{p_k} - \tilde{w}_\tau\|$$

and the equality is only satisfied if either $p_k = 1$ for all $k \in [\tau - 1]$ or $\|\delta_k\| = 0$ for all $k \in [\tau - 1]$.

Proof. Given the definitions of $\bar{w}_\tau^{p_k}$ and $\tilde{w}_\tau$ and following the proof of [1, Theorem 2] we get that

$$\begin{aligned} \bar{w}_\tau^{p_k} - \tilde{w}_\tau &= \frac{\eta}{n-1} \sum_{k=1}^{\tau} \sum_{u \in \mathcal{U} \setminus u'} (\nabla f_u(w_{k-1}) - \nabla f_u(\tilde{w}_{k-1})) \\ &\quad - \sum_{k=1}^{\tau} (1 - p_{k-1}) \delta_{k-1}, \end{aligned}$$

We denote $\Lambda := \eta/(n-1) \sum_{k=1}^{\tau} \sum_{u \in \mathcal{U} \setminus u'} (\nabla f_u(w_{k-1}) - \nabla f_u(\tilde{w}_{k-1}))$. Then, given [1, Equations (26)–(28)] we have that

$$\sup \|\bar{w}_\tau^{p_k} - \tilde{w}_\tau\| = \|\Lambda\| + \sum_{k=1}^{\tau} (1 - p_{k-1}) \|\delta_{k-1}\|.$$

Finally, adding everything together we get

$$\frac{\sup \|\bar{w}_\tau^1 - \tilde{w}_\tau\|}{\sup \|\bar{w}_\tau^{p_k} - \tilde{w}_\tau\|} = \frac{\|\Lambda\|}{\|\Lambda\| + \sum_{k=1}^{\tau} (1 - p_{k-1}) \|\delta_{k-1}\|} \leq 1,$$

where the equality is only satisfied if either $p_k = 1$ for all $k \in [\tau - 1]$ or $\|\delta_k\| = 0$ for all $k \in [\tau - 1]$. $\square$

Using Lemma 4.1, Equation 4.6 can be simplified by removing the second term, yielding the following:

$$\|\bar{w}_\tau - \tilde{w}_\tau\| \leq \left\| \frac{\eta}{n-1} \sum_{k=1}^{\tau-1} \sum_{u \neq u'} \nabla f_u(w_k) - \nabla f_u(\tilde{w}_k) \right\|. \quad (4.8)$$

4.3.5.2 Clipping the Gradient Updates

To further improve Equation 4.8, we take a closer look to it, and notice that once the terms involving the residuals δ_k are removed by setting all the weights to 1, the only elements left are the distance at each step between

the gradients applied to either the real model w_k or the ideal model $\tilde{w}_k$. Therefore, we strive to more accurately bound this distance per client and per round.

To tackle this, we adopt a different approach to FedRecovery during the learning phase, inspired by the DP-SGD algorithm [79]. This algorithm, which applies DP to each computed gradient, controls the size of the noise added by clipping the updates to some small quantity that allows for the noise to remain small. Similarly, we propose to clip the gradient updates at each round with some parameter C to more accurately control the bound in Equation 4.8 depending on C . More formally, given some clipping parameter C , we define the new clipped version of the gradient updates as

$$\nabla \mathring{f}_u(\cdot) := \min(1, C/\|\nabla f_u(\cdot)\|) \cdot \nabla f_u(\cdot), \quad (4.9)$$

which in turn ensures that $\|\nabla \mathring{f}(\cdot)\| \leq C$. Then, by swapping in the clipped gradients during training and computing the residuals, we obtain:

$$\|\bar{w}_\tau - \tilde{w}_\tau\| = 2 \cdot \eta \cdot \sum_{k=1}^{\tau} C. \quad (4.10)$$

The objective now is finding an appropriate clipping parameter C such that Equation 4.10 becomes better than Equation 4.7, while taking into account that clipping has an impact in the model convergence. Proof of Equation 4.10 is give as following;

Proof.

$$\begin{aligned} \|\bar{w}_\tau - \tilde{w}_\tau\| &\leq \left\| \frac{\eta}{n-1} \sum_{k=1}^{\tau} \sum_{u \in \mathcal{U} \setminus u'} \nabla \mathring{f}_u(w_{k-1}) - \nabla \mathring{f}_u(\tilde{w}_{k-1}) \right\| \\ &\leq \eta \cdot \sum_{k=1}^{\tau} \left\| \nabla \mathring{f}_u(w_{k-1}) - \nabla \mathring{f}_u(\tilde{w}_{k-1}) \right\| \\ &\leq 2 \cdot \eta \cdot \sum_{k=1}^{\tau} C \end{aligned}$$

where the first inequality comes from Equation 4.8, the second step comes from applying the triangular inequality and the final step comes from applying that both clipped updates are at most of size C and at worst in completely opposite directions. $\square$

4.3.6 A Round-Dependent Clipping Parameter

When considering the trade-off between lower clipping parameter and better model convergence, we make the following observation. In gradient descent-based approaches, the updates usually become smaller as the training advances. As such, with a constant clipping parameter C , the first updates are greatly reduced, while later on hardly any update gets clipped.

In the FU scenario, where the noise is added *only during unlearning*, the clipping budget may be too important. In other words, a lot of norm bound is wasted in the later rounds, while it could be used in former rounds to increase the efficiency of the training, incurring a faster convergence. This becomes in contradiction with the original DP-SGD algorithm [79], since noise is added at each update, and consequently increasing the already large clipping budget.

To address this problem, we propose to define a variable clipping parameter per round, denoted as C_τ , which adds up to some global clipping budget C_{tot} . Accordingly,

$$\|\bar{w}_\tau - \tilde{w}_\tau\| \leq 2 \cdot \eta \cdot \sum_{k=1}^{\tau} C_k \quad (4.11)$$

Finally, the round-based clipping parameters are defined in a parametrizable way such that they decrease with the training round, and, at the unlearning round τ , they add up to some previously fixed clipping budget C_{tot} . Since the unlearning round is not known in advance, we propose to estimate a bounding of it by some τ_{max} with some assigned minimum clipping parameter C_{min} .

We propose a polynomial way to distribute the clipping budget depending on some round τ_{flip} at which the clipping parameter flips from being more than the average among all rounds to being less than the average among all rounds. Then, this round τ_{flip} can be matched (as best as possible) to the round of convergence, so as to maximize the usefulness of the clipping budget.

Let $\ell_1, \ell_2 \in \mathbb{R}_{>0}$ be shape parameters, $\tau_{\text{max}} \in \mathbb{Z}_{>0}$ be the maximum efficient unlearning round, τ_{flip} be the flipping round such that $0 < \tau_{\text{flip}} < \tau_{\text{max}}$, $C_{\text{tot}} \in \mathbb{R}_{>0}$ be the clipping budget, and C_{min} be the minimum allowed round clipping budget such that $0 < C_{\text{min}} < C_{\text{tot}}/\tau_{\text{max}}$. Additionally, we define the following parameters:

$$a_1 := \frac{\frac{C_{\text{tot}}}{\tau_{\text{max}}} - C_{\text{min}}}{(\tau_{\text{max}} - \tau_{\text{flip}})^{\frac{1}{\ell_1}}} \text{ and } a_2 := a_1 \cdot \frac{\sum_{i=\tau_{\text{flip}}+1}^{\tau_{\text{max}}} |i - \tau_{\text{flip}}|^{\frac{1}{\ell_1}}}{\sum_{i=1}^{\tau_{\text{flip}}-1} |i - \tau_{\text{flip}}|^{\frac{1}{\ell_2}}} \quad (4.12)$$

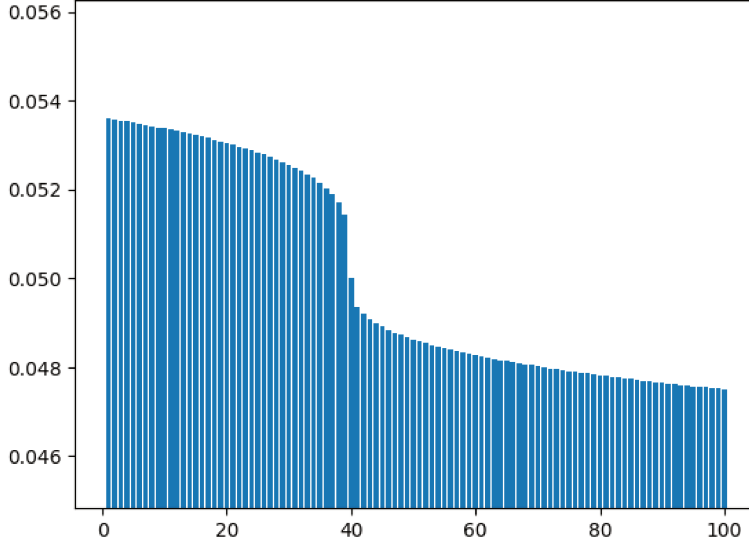


Figure 4.2: Computation of parameters C_k with parameters $\ell_1 = 3, \ell_2 = 4, \tau_{\max} = 100, \tau_{\text{flip}} = 40, C_{\text{tot}} = 5, C_{\min} = 0.0475$.

Then, for any integer $0 < k \leq \tau_{\max}$, the *clipping parameter at round k* is defined as:

$$C_k := \begin{cases} \frac{C_{\text{tot}}}{\tau_{\max}} + a_2 \cdot |k - \tau_{\text{flip}}|^{\frac{1}{\ell_2}} & \text{if } k < \tau_{\text{flip}}, \\ \frac{C_{\text{tot}}}{\tau_{\max}} & \text{if } k = \tau_{\text{flip}}, \\ \frac{C_{\text{tot}}}{\tau_{\max}} - a_1 \cdot |k - \tau_{\text{flip}}|^{\frac{1}{\ell_1}} & \text{if } k > \tau_{\text{flip}}. \end{cases} \quad (4.13)$$

Remark 1. *Unlearning may happen after $\tau_{\max}$, if for any round after $\tau_{\max}$, we keep clipping at $C_{\min}$. The only difference is that the sum over all rounds of the clipping parameters will be higher than the clipping budget. Therefore, we can refer $\tau_{\max}$ as the maximum efficient unlearning round.*

Remark 2. *The different parameters involved in the clipping definition have different impacts on the shape of the curve of the round-based clipping parameters, as seen in Figure 4.2. This means that each application may have different optimal parameters. The shape parameters ℓ_i , for $i \in \{1, 2\}$, define the curve: the higher they are the “boxier” the curve becomes. Furthermore if they are smaller than 1, then the curves change curvature (from convex to concave and vice versa). The round parameters work intuitively: a higher $\tau_{\max}$ will imply more parameters being computed, while τ_{flip} will denote where the parameters go from being bigger than average to lower than average. Finally, the budget parameters will impact the size of each parameter: the higher*

C_{tot} is the more budget there is to allocate (implying higher individual parameters), and the closer C_{min} is to $C_{\text{tot}}/\tau_{\text{max}}$, the flatter the distribution will be.

As such, we want to show that the parameters do indeed satisfy the intended conditions, thus the following Lemma.

Lemma 4.2. *Let C_k be as defined as in Section 4.3.6. Then, we have $C_{\tau_{\text{max}}} = C_{\text{min}}$ and $\sum_{k=1}^{\tau_{\text{max}}} C_k = C_{\text{tot}}$.*

Putting it all together, we finally obtain the bound:

$$\|\bar{w}_{\tau_{\text{max}}} - \tilde{w}_{\tau_{\text{max}}}\| \leq 2 \cdot \eta \cdot C_{\text{tot}}. \quad (4.14)$$

Proof. Let us first verify $C_{\tau_{\text{max}}}$.

$$\begin{aligned} C_{\tau_{\text{max}}} &= \frac{C_{\text{tot}}}{\tau_{\text{max}}} - a_1 \cdot |\tau_{\text{max}} - \tau_{\text{flip}}|^{\frac{1}{\ell_1}}, \\ &= \frac{C_{\text{tot}}}{\tau_{\text{max}}} - \frac{\frac{C_{\text{tot}}}{\tau_{\text{max}}} - C_{\text{min}}}{(\tau_{\text{max}} - \tau_{\text{flip}})^{\frac{1}{\ell_1}}} \cdot |\tau_{\text{max}} - \tau_{\text{flip}}|^{\frac{1}{\ell_1}}, \\ &= C_{\text{min}}. \end{aligned}$$

Now let us verify the clipping budget is satisfied.

$$\begin{aligned} \sum_{k=1}^{\tau_{\text{max}}} C_k &= \sum_{i=1}^{\tau_{\text{flip}}-1} C_i + C_{\tau_{\text{flip}}} + \sum_{i=\tau_{\text{flip}}+1}^{\tau_{\text{max}}} C_i, \\ &= \sum_{i=1}^{\tau_{\text{flip}}-1} \frac{C_{\text{tot}}}{\tau_{\text{max}}} + a_2 |i - \tau_{\text{flip}}|^{\frac{1}{\ell_2}} + \frac{C_{\text{tot}}}{\tau_{\text{max}}} \\ &\quad + \sum_{i=\tau_{\text{flip}}+1}^{\tau_{\text{max}}} \frac{C_{\text{tot}}}{\tau_{\text{max}}} - a_1 |i - \tau_{\text{flip}}|^{\frac{1}{\ell_1}}, \\ &= C_{\text{tot}} + \sum_{i=1}^{\tau_{\text{flip}}-1} a_2 |i - \tau_{\text{flip}}|^{\frac{1}{\ell_2}} - \sum_{i=\tau_{\text{flip}}+1}^{\tau_{\text{max}}} a_1 |i - \tau_{\text{flip}}|^{\frac{1}{\ell_1}}, \end{aligned}$$

which will be equal to C only if the summation involving a_2 is canceled out

by the one involving a_1 . In other words,

$$\begin{aligned}
\sum_{i=1}^{\tau_{\text{flip}}-1} a_2 \cdot |i - \tau_{\text{flip}}|^{\frac{1}{\ell_2}} &= a_2 \cdot \sum_{i=1}^{\tau_{\text{flip}}-1} |i - \tau_{\text{flip}}|^{\frac{1}{\ell_2}}, \\
&= a_1 \cdot \frac{\sum_{i=\tau_{\text{flip}}+1}^{\tau_{\text{max}}} |i - \tau_{\text{flip}}|^{\frac{1}{\ell_1}}}{\sum_{i=1}^{\tau_{\text{flip}}-1} |i - \tau_{\text{flip}}|^{\frac{1}{\ell_2}}} \cdot \sum_{i=1}^{\tau_{\text{flip}}-1} |i - \tau_{\text{flip}}|^{\frac{1}{\ell_2}}, \\
&= \sum_{i=\tau_{\text{flip}}+1}^{\tau_{\text{max}}} a_1 \cdot |i - \tau_{\text{flip}}|^{\frac{1}{\ell_1}}.
\end{aligned}$$

□

4.3.7 Final Design Considerations

4.3.7.1 Controlling High Noise

In addition to the previously proposed optimizations to reduce the noise as much as possible to increase the accuracy of the model while not sacrificing privacy protection, we propose a last clipping of the unlearned model using some pre-defined bound γ . Indeed, despite the important decrease on the standard deviation of the noise, the normal distribution still has a chance of outputting a very high noise, especially with lower unlearning parameter α . This may refrain the unlearned model from converging, making any further training by the remaining clients unsuccessful. The new pre-defined bound is defined by the problem and architecture to prevent such an unfortunate event. For example, in the case of training convolutional neural networks for handwritten recognition whereby the parameters are usually in the interval $[-1, 1]$, we set $\gamma = 1$. This additional clipping does not harm the unlearning guarantee, since the property of post-processing for DP [80, Proposition 2.4] extends to our unlearning definition.

4.3.7.2 Final FedRecover+

As defined in Figure 4.3, at a training round τ , each client sends to the server its gradients clipped with parameter C_k computed as in Section 4.3.6. Then, at an unlearning round τ , the residuals are computed following Equation 4.2 with the clipped gradients. After that, the unlearned model is computed as $w_u := w_\tau - \sum_{k=1}^{\tau-1} \delta_k + z$ for some sampled noise $z \leftarrow \mathcal{N}(0, \sigma^2 \mathbb{I}_d)$ for σ computed by combining Equations 4.5 and 4.14. Finally, the unlearned model is clipped to remove excess noise using the predefined parameter γ .

FedRecovery+

Parameters: $\alpha, \beta \in \mathbb{R}_{>0}$ unlearning parameters, $\eta \in \mathbb{R}_{>0}$ learning rate, $\ell_1, \ell_2 \in \mathbb{R}_{>0}$ shape parameters, $\tau_{\max} \in \mathbb{Z}_{>0}$ maximum clipping round, $0 < \tau_{\text{flip}} < \tau_{\max}$ flipping round, $C_{\text{tot}} \in \mathbb{R}_{>0}$ clipping budget, $0 < C_{\min} < C_{\text{tot}}/\tau_{\max}$ minimum clipping parameter, and $\gamma \in \mathbb{R}_{>0}$ model domain parameter.

Learning round τ :

Client $u \in \mathcal{U}$:

- Compute clipping parameter C_τ as in Definition ??.
- Compute clipped gradient

$$\nabla \mathring{f}_u(w_{\tau-1}) := \min(1, C_\tau / \|\nabla f_u(w_{\tau-1})\|) \cdot \nabla f_u(w_{\tau-1}),$$

and send it to the server.

Server:

- Compute $w_\tau = w_{\tau-1} - \eta/n \sum_{u \in \mathcal{U}_\tau} \nabla \mathring{f}_u(w_{\tau-1})$.

Unlearning round τ (unlearning client u'):

Server:

- For $k \in [\tau]$, compute the residuals

$$\delta_k = \eta/n \left((1/n - 1) \sum_{u \in \mathcal{U}_\tau/v} \nabla \mathring{f}_u(w_k) - \nabla \mathring{f}_v(w_k) \right).$$

- Sample noise $z \leftarrow \mathcal{N}(0, \sigma^2 \mathbb{I}_d)$, for

$$\sigma := \frac{1}{\sqrt{2}} \cdot \frac{2 \cdot \eta \cdot \sum_{k=1}^{\tau} C_k}{\sqrt{\log(1/\beta) + \alpha} - \sqrt{\log(1/\beta)}}$$

- Compute unlearned model $w_u := w_\tau - \sum_{k=1}^{\tau-1} \delta_k + z$.
- Output clipped unlearned model $\mathring{w}_u := \min(1, \gamma/\|w_u\|) \cdot w_u$.

Figure 4.3: Description of FedRecovery+

Theorem 1. *The unlearning algorithm FedRecovery+ described in Figure 4.3 satisfies (α, β) -unlearning.*

Proof. Let us start analyzing w_u at round $\tau_{\max}$. By Propositions 3 and 6 in [78], we know that adding the distribution $\mathcal{D} \sim \mathcal{N}(0, \sigma^2 \cdot \mathbb{I}_d)$ to values $\bar{w}_{\tau_{\max}}$ or $\tilde{w}_{\tau_{\max}}$ implies that for any $\beta > 0$ the two distribution are (α', β) -

indistinguishable for

$$\alpha' = \frac{\|\bar{w}_{\tau_{\max}} - \tilde{w}_{\tau_{\max}}\|^2}{2\sigma^2} + \frac{2 \cdot \|\bar{w}_{\tau_{\max}} - \tilde{w}_{\tau_{\max}}\|}{\sqrt{2} \cdot \sigma} \cdot \sqrt{\log\left(\frac{1}{\beta}\right)},$$

and by then applying Equation 4.14 we get

$$\alpha' \leq \frac{(2 \cdot \eta \cdot C_{\text{tot}})^2}{2 \cdot \sigma^2} + \frac{2 \cdot (2 \cdot \eta \cdot C_{\text{tot}})}{\sqrt{2} \cdot \sigma} \cdot \sqrt{\log\left(\frac{1}{\beta}\right)},$$

and by swapping the σ used in Figure 4.3 we get

$$\begin{aligned} \alpha' &\leq \left(\sqrt{\log\left(\frac{1}{\beta}\right)} + \alpha - \sqrt{\log\left(\frac{1}{\beta}\right)} \right)^2 \\ &\quad + 2 \left(\sqrt{\log\left(\frac{1}{\beta}\right)} + \alpha - \sqrt{\log\left(\frac{1}{\beta}\right)} \right) \cdot \sqrt{\log\left(\frac{1}{\beta}\right)}, \end{aligned}$$

which by developping becomes

$$\begin{aligned} \alpha' &\leq \log\left(\frac{1}{\beta}\right) + \alpha + \log\left(\frac{1}{\beta}\right) - 2 \cdot \log\left(\frac{1}{\beta}\right), \\ &\leq \alpha. \end{aligned}$$

This means that the distributions $\bar{w}_{\tau_{\max}} + \mathcal{D}$ and $\tilde{w}_{\tau_{\max}} + \mathcal{D}$ are (α, β) -indistinguishable. Let us then define the clipping function $f(x) := \min(1, \gamma/\|x\|) \cdot x$. By the postprocessing property (Proposition 2.4, [80]), we know that if two distributions are (α, β) -indistinguishable, then applying the same function to both functions maintains this property. Therefore, by rewriting the property, we get that the output of $\mathcal{M}_u$ (the clipped unlearned model $\hat{w}_u$) satisfies (α, β) -machine unlearning with respect to the machine learning algorithm that outputs $\tilde{w}_u := \min(1, \gamma/\|\tilde{w}_{\tau_{\max}} + z\|) \cdot (\tilde{w}_{\tau_{\max}} + z)$ for some $z \leftarrow \mathcal{D}$, as we wanted to see. $\square$

4.4 Threshold Secure Aggregation with Forget Functionality

In this section, we propose an extension to the paradigm of Threshold Secure Aggregation (TSA) that supports posterior forgetting of one client's input while retaining the privacy guarantees of the inputs.

4.4.1 Our TSA Extension

A key limitation of traditional TSA schemes (Chapter 2) is that they are essentially one-shot and immutable: once the inputs are aggregated, they cannot be separated or removed (usually for security reasons due to their design). Nevertheless, the ability to remove specific inputs (the ones from the leaving client u') is required to implement FU. The key requirement is therefore to support the removal of specific inputs from the aggregate without ever exposing them in plaintext. It is important to note that if the unlearned value is disclosed, it directly leaks u' 's input as well as some information about the remaining clients' data.

To address this challenge, we propose an extension of TSA, named as TSA-forget, which includes a new algorithm **TSA.Forget** along with the existing algorithms **TSA.Setup**, **TSA.Protect** and **TSA.Agg**. This new algorithm mirrors **TSA.Agg** using a forget key assigned to each client and divided into a client-side part and a server-side part. It allows the aggregator (server) and online clients to efficiently and securely remove the input of u' from a previously computed aggregate while maintaining strong privacy guarantees, i.e., without disclosing any individual input, including the one from u' .

4.4.2 Ideal Functionality of TSA with Forget Functionality

The ideal functionality of the proposed TSA-forget is shown in Figure 4.4. It captures the expected behavior of the protocol in an ideal setting and consists of two sub-functionalities: $\mathcal{F}_{agg}$, which performs the aggregation operation, and $\mathcal{F}_{forget}$, which handles the removal of a client's contribution during unlearning.

4.4.2.1 Instantiation of the Forget Functionality

The additions to a TSA (Section 2.2.4) protocol to instantiate the new forget functionality are depicted in 4.4.1. First, **TSA.Setup** is updated to generate forget key fk_u for every client $u \in \mathcal{U}$. To do so, it secret shares the aggregation key of every client among the others. The algorithms **TSA.Protect** and **TSA.Agg** remain the same. Then, **TSA.Forget** invokes two algorithms. Firstly, **Client.TSA.Forget** which takes as input a divisible distribution $\mathcal{D}$ and samples $\mathcal{D}'$ from it, i.e., its k -division with parameter $k = |\mathcal{U}_\tau^{forget}|$, here $\mathcal{U}_\tau^{forget}$ is the set of online clients during forget, adds this noise to $x_{agg,\tau}/k$, and calls **TSA.Protect** to encrypt this noisy aggregate with the user's encryption key ek_u . It then calls **Client.TSA.Agg** in order to obtain the one-time aggrega-

Auxiliary Inputs: security parameter λ , description of TSA.Setup, TSA.Protect, TSA.Agg and TSA.Forget algorithms.

There are two sub-functionalities:

Functionality $\mathcal{F}_{agg}$, as $\mathcal{F}_{TSA}$ given in Section 2.3.3.1, outputs x_τ .

Functionality $\mathcal{F}_{forget}$: Upon receiving $(\text{sid}, \tau, x_\tau, x_{u',\tau})$:

- if $|\mathcal{U}_\tau^{forget}| < t$ or if $v \notin \mathcal{U}_\tau^{on}$, send $(\text{Abort}, \text{sid})$;
- else, sample $g \leftarrow \mathcal{D}$ and $x'_\tau = x_\tau - x_{u',\tau} + g$, and send (x'_τ, sid) to server $\mathcal{S}$.

Figure 4.4: Ideal functionality $\mathcal{F}_{TSA}$ with Forget Functionality.

tion key $ak'_{u,\tau}$ for the server, to decrypt the noisy output. Accordingly, it outputs the one-time forget key $fk_{u,\tau}$ defined as $fk_{u,\tau} = (ak'_{u,\tau}, y_{u',\tau}^{(u)}, [ak'_u]_u)$. Secondly, given the one-time forget keys received from all users $u \in \mathcal{U}_\tau^{forget}$, **Server.TSA.Forget** first reconstructs the aggregation key of u' using the shares $\{[ak_u]_u\}_{u \in \mathcal{U}_\tau^{forget}}$. Then, it calls **Client.TSA.Agg** which outputs the one-time aggregation key $ak'_{u',\tau}$ corresponding to u' 's input. Finally, the one-time aggregation key $ak'_{u,\tau}$ of user u and the one-time aggregation key $ak'_{u',\tau}$ of user u' are used for **Server.TSA.Agg** to obtain the aggregation of the previously computed noisy values and the ciphertext from user u' .

Definition 4.4.1 (Instantiation of TSA with Forget Functionality). *It consists of the following algorithms:*

- **TSA.Setup** $(1^\lambda, n, t) \rightarrow (pp, \{ek_u, ak_u, fk_u\}_{u \in \mathcal{U}})$: Given security parameter λ , total number of users n and threshold $t \leq n$, it starts as **TSA.Setup** $(1^\lambda, n, t)$ and adds a family of divisible distributions $\mathcal{G}$ to the public parameters. Then, it invokes **SS.Share** $(ak_u, r, \mathcal{U})$ to obtain one t -out-of- n share of ak_u for each user in $\mathcal{U}$. The last output is the forget key for each user, defined as: $fk_u = (ek_u, ak_u, \{[ak_v]_u\}_{v \in \mathcal{U}})$.
- **TSA.Forget** $(pp, \tau, \{fk_u\}_{u \in \mathcal{U}_\tau^{forget}}, Y_{u',\tau}, x_{agg,\tau}, \mathcal{D}) \rightarrow x'_{agg,\tau}$: Given the forget keys $\{fk_u\}_{u \in \mathcal{U}_\tau^{forget}}$, where $\mathcal{U}_\tau^{forget} \subseteq \mathcal{U}$ and $|\mathcal{U}_\tau^{forget}| \geq t$, ciphertext $Y_{u',\tau}$ at round τ belonging to the to-forget user $u' \in \mathcal{U}_\tau^{on}$, the aggregate at round τ $x_{agg,\tau}$, and a k -divisible probability distribution $\mathcal{D}$ where $k = |\mathcal{U}_\tau^{forget}|$, this algorithm calls the following:
 - **Client.TSA.Forget** $(pp, \tau, fk_u, y_{u',\tau}^{(u)}, x_\tau, \mathcal{D}) \rightarrow fk_{u,\tau}$: For ev-

ery client $u \in \mathcal{U}_\tau^{\text{forget}}$, it samples $z_u \leftarrow \mathcal{D}'$, where $\mathcal{D}'$ is the k -division of distribution $\mathcal{D}$ where $k = |\mathcal{U}_\tau^{\text{forget}}|$, and then calls $\text{TSA.Protect}(pp, \tau, ek_u, -(\frac{x_{agg, \tau}}{k} + z_u)) \rightarrow Y'_{u, \tau} := (y'^{(S)}_{u, \tau}, \{y'^{(v)}_{u, \tau}\}_{v \in \mathcal{U}})$, forwarding $y'^{(S)}_{u, \tau}$ to the server and $y'^{(v)}_{u, \tau}$ to the corresponding user $v \in \mathcal{U}$. Then, each client $u \in \mathcal{U}_\tau^{\text{forget}}$ computes the one-time aggregation key for these noisy ciphertexts as $\text{Client.TSA.Agg}(pp, \tau, ak_u, \{y'^{(u)}_{u', \tau} \cup \{y'^{(u)}_{v, \tau}\}_{v \in \mathcal{U}_\tau^{\text{forget}}}\}) \rightarrow ak'_{u, \tau}$. Finally, it sets the one-time forget-key as $fk_{u, \tau} = (ak'_{u, \tau}, y'^{(u')}_{u, \tau}, [ak_{u'}]_u)$.

- $\text{Server.TSA.Forget}(pp, \tau, \{fk_{u, \tau}\}_{u \in \mathcal{U}_\tau^{\text{forget}}}, y'^{(S)}_{u', \tau}) \rightarrow x'_{agg, \tau}$: The server first reconstructs $ak_{u'}$ by executing $\text{SS.Combine}(\{(u, [ak_{u'}]_u)\}_{u \in \mathcal{U}_\tau^{\text{forget}}}, t)$. Then, it computes $\text{Client.TSA.Agg}(pp, \tau, ak_{u'}, \{y'^{(u')}_{u', \tau} \cup \{y'^{(u')}_{u, \tau}\}_{u \in \mathcal{U}_\tau^{\text{forget}}}\}) \rightarrow ak'_{u', \tau}$. Finally, it executes $-\text{Server.TSA.Agg}(pp, \tau, \{ak'_{u', \tau} \cup \{ak'_{u, \tau}\}_{u \in \mathcal{U}_\tau^{\text{forget}}}\}, \{y'^{(S)}_{u', \tau} \cup \{y'^{(S)}_{u, \tau}\}_{u \in \mathcal{U}_\tau^{\text{forget}}}\}) \rightarrow x'_{agg, \tau}$.

Remark 3. One observation is that when Server.TSA.Forget is called, the server gains access to the aggregation key of the to-forget client $ak_{u'}$. Note that this does not compromise the overall security as u' is being removed from future sets $\mathcal{U}_\tau^{\text{on}}$, and will no longer participate in future aggregation processes. Given access to this key does not disclose the inputs of u' .

Theorem 2. The protocol extension described in Figure 4.4.1 correctly and securely instantiates the ideal functionality $\mathcal{F}_{\text{TSA}}$ against at most t corrupted clients and a corrupted server as long as the threshold aggregation protocol $\Pi_{\text{TSA}} := (\text{TSA.Setup}, \text{TSA.Protect}, \text{TSA.Agg})$ correctly and securely instantiates the ideal functionality $\mathcal{F}_{\text{Agg}}$ against at most t corrupted clients and a corrupted server and the secret sharing scheme $\text{SS} = (\text{SS.Setup}, \text{SS.Share}, \text{SS.Combine})$ holds correctness and secrecy for threshold t .

Proof. Regarding correctness, let us examine the correctness of forgetting. Given the correctness of SS , we know that ak_v is correctly formed. Then, thanks to the correctness of the aggregation in TSA , we know that:

$$\begin{aligned} x'_{agg, \tau} &= - \left(x_{u', \tau} + \sum_{v \in \mathcal{U}_\tau^{\text{forget}}} - \left(\frac{x_{agg, \tau}}{k} + z_{v, \tau} \right) \right), \\ &= x_{agg, \tau} - x_{u', \tau} + \sum_{v \in \mathcal{U}_\tau^{\text{forget}}} z_{v, \tau}, \end{aligned}$$

and by the k -divisibility of $\mathcal{D}$, we have $x'_{agg,\tau} \approx_c x_{agg,\tau} - x_{u,\tau} + \mathcal{D}$.

Regarding security, first recap how exactly the simulator from TSA is defined.

- $(pp, \{\widetilde{ek}_u, \widetilde{ak}_u\}_{u \in \mathcal{C}}) \leftarrow \text{Sim}_{\mathcal{C}, \text{Server}}^{\text{TSA.SetUp}}(1^\lambda, n, t)$: On input the security parameter λ , the total number of clients n and the threshold t , it outputs some public parameters pp , and simulated encryption keys $\widetilde{ek}_u$ and simulated aggregation keys $\widetilde{ak}_u$ for every corrupted client $u \in \mathcal{C}$.
- $\{\widetilde{Y}_{u,\tau} := (\widetilde{y}_{u,\tau}^{(S)}, \widetilde{y}_{u,\tau}^{(v)})\}_{u \in \mathcal{U}_\tau^{\text{on}} \setminus \mathcal{C}, v \in \mathcal{C}}, \text{st}_{\text{TSA}} \leftarrow \text{Sim}_{\mathcal{C}, \text{Server}}^{\text{TSA.Protect}}(pp, \tau, \mathcal{U}_\tau^{\text{on}}, \{x_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{on}} \cap \mathcal{C}}, \mathcal{F}_{\text{Agg},\tau})$: On input some public parameters pp , a tag τ , the set of clients participating to the specific aggregation $\mathcal{S}_{x,\tau}$, the inputs of the corrupted clients for the specific aggregation $\{x_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{on}} \cap \mathcal{C}}$ and the output of the ideal functionality $\mathcal{F}_{\text{Agg},\tau}$, it outputs a simulated ciphertext from every honest player to the server and corrupted player $\{\widetilde{Y}_{u,\tau} := (\widetilde{y}_{u,\tau}^{(S)}, \widetilde{y}_{u,\tau}^{(v)})\}_{u \in \mathcal{U}_\tau^{\text{on}} \setminus \mathcal{C}, v \in \mathcal{C}}$ as well as a state recording the simulated value encoded by each of these ciphertexts $\text{st}_{\text{TSA}} := \{\widetilde{x}_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{on}} \setminus \mathcal{C}}$.
- $\{\widetilde{ak}_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{on}} \setminus \mathcal{C}} \leftarrow \text{Sim}_{\mathcal{C}, \text{Server}}^{\text{Client.TSA.Agg}}(pp, \tau, \mathcal{U}_\tau^{\text{on}}, \text{st}_{\text{TSA}}, \{x_{u,\tau}\}_{u \in \mathcal{C}}, \{y_{u,\tau}^{(v)}\}_{u \in \mathcal{C}, v \in \mathcal{U}_\tau^{\text{on}} \setminus \mathcal{C}}, \mathcal{F}_{\text{Agg},\tau})$: On input some public parameters pp , a tag τ , the set of clients participating to the specific aggregation $\mathcal{U}_\tau^{\text{on}}$, the state from the protect simulator st_{TSA} , the input (for protect) of the corrupted clients $\{x_{u,\tau}\}_{u \in \mathcal{C}}$, the parts of the ciphertexts generated by the corrupted clients for the honest clients $\{y_{u,\tau}^{(v)}\}_{u \in \mathcal{C}, v \in \mathcal{U}_\tau^{\text{on}} \setminus \mathcal{C}}$ and the output of the ideal functionality $\mathcal{F}_{\text{Agg},\tau}$, it outputs a simulated temporary aggregation key from every honest player $\{\widetilde{ak}_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{on}} \setminus \mathcal{C}}$.

Given this simulator, the correctness property for it states that $\mathcal{F}_{\text{Agg},\tau} = \text{Server.TSA.Agg}(\{ak_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{on}} \cap \mathcal{C}}, \{\widetilde{ak}_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{on}} \setminus \mathcal{C}}, \{y_{u,\tau}^{(S)}\}_{u \in \mathcal{U}_\tau^{\text{on}} \cap \mathcal{C}}, \{\widetilde{y}_{u,\tau}^{(S)}\}_{u \in \mathcal{S}_x \setminus \mathcal{C}})$.

Now, let us prove that there exists a PPT simulator $\text{Sim}_{\mathcal{C}, \text{Server}}^{\text{TSA.Forget}} := (\text{Sim}_{\mathcal{C}, \text{Server}}^{\text{TSA.SetUp}}, \text{Sim}_{\mathcal{C}, \text{Server}}^{\text{Client.TSA.Forget}})$ indistinguishable from the real execution of the protocol, through a series of hybrid games H_0, H_1, H_2, H_3 .

Hybrid H_0 : This is the real world execution.

Hybrid H_1 : In this hybrid we simulate the shares of the aggregation key of all the honest clients sent to the corrupted clients $\{[\widetilde{ak}_v]_u\}_{v \in \mathcal{U} \setminus \mathcal{C}, u \in \mathcal{C}}$ as follows. The simulator samples $\widetilde{ak}_v$ at random for every honest client $v \in \mathcal{U} \setminus \mathcal{C}$, and computes $\{[\widetilde{ak}_v]_u\}_{u \in \mathcal{U}} \leftarrow \text{SS.Share}(pp_{\text{SS}}, \widetilde{ak}_v, t, \mathcal{U})$. Note that the simulator keeps track of the actual ak_v (generated by the execution of TSA.SetUp) for $v \in \mathcal{U} \setminus \mathcal{C}$ as well as the simulated pieces $\{[\widetilde{ak}_v]_u\}_{v \in \mathcal{U} \setminus \mathcal{C}, u \in \mathcal{C}}$. Then, to maintain the congruence, during the Client.TSA.Forget phase, when v is the

client being forgotten, using ak_v as well as the simulated pieces from the corrupted clients participating in the forgetting $\{\widetilde{ak_v}_u\}_{u \in \mathcal{U}_\tau^{\text{forget}} \cap \mathcal{C}}$ computes $\{\widetilde{ak_v}_u\}_{u \in \mathcal{U}_\tau^{\text{forget}} \setminus \mathcal{C}}$ such that $ak_v \leftarrow \text{SS.Combine}(pp_{\text{SS}}, \{\widetilde{ak_v}_u\}_{u \in \mathcal{U}_\tau^{\text{forget}}, t})$. The view of the adversary in this hybrid is indistinguishable from H_0 due to the secrecy of the secret sharing scheme.

Hybrid H_2 : In this hybrid, we swap all the uses of the protocol Π_{TSA} by their respective simulators. In more details,

- during the setup phase, the simulator will call $(pp, \{\widetilde{ek}_u, \widetilde{ak}_u\}_{u \in \mathcal{C}}) \leftarrow \text{Sim}_{\mathcal{C}, \text{Server}}^{\text{TSA.Setup}}(1^\lambda, n, t)$,
- during the protect phase, the simulator will call $\{\widetilde{Y}_{u,\tau} := (\widetilde{y}_{u,\tau}^{(S)}, \widetilde{y}_{u,\tau}^{(v)})\}_{u \in \mathcal{U}_\tau^{\text{on}} \setminus \mathcal{C}, v \in \mathcal{C}}, \text{st}_{\text{TSA}} \leftarrow \text{Sim}_{\mathcal{C}, \text{Server}}^{\text{TSA.Protect}}(pp, \tau, \mathcal{U}_\tau^{\text{on}}, \{x_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{on}} \cap \mathcal{C}}, \mathcal{F}_{\text{Agg}, \tau})$, and
- during the aggregation phase, the simulator will call $\{\widetilde{ak}_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{on}} \setminus \mathcal{C}} \leftarrow \text{Sim}_{\mathcal{C}, \text{Server}}^{\text{Client.TSA.Agg}}(pp, \tau, \mathcal{U}_\tau^{\text{on}}, \text{st}_{\text{TSA}}, \{x_{u,\tau}\}_{u \in \mathcal{C}}, \{y_{u,\tau}^{(v)}\}_{u \in \mathcal{C}, v \in \mathcal{U}_\tau^{\text{on}} \setminus \mathcal{C}}, \mathcal{F}_{\text{Agg}, \tau})$.

Then, during the forget phase for client u' , let us assume the simulator has access to the noisy inputs of the corrupted clients $\{x'_{u,\tau}\}_{u \in \mathcal{C}} := \{-(x_{\text{agg}, \tau}/k) + z_{u,\tau}\}_{u \in \mathcal{C}}$. First, the simulator will call $\{\widetilde{Y}'_{u,\tau} := (\widetilde{y}'_{u,\tau}^{(S)}, \widetilde{y}'_{u,\tau}^{(v)})\}_{u \in (\mathcal{U}_\tau^{\text{forget}} \cup u') \setminus \mathcal{C}, v \in \mathcal{C}}, \text{st}'_{\text{TSA}} \leftarrow \text{Sim}_{\mathcal{C}, \text{Server}}^{\text{TSA.Protect}}(pp, \tau, \mathcal{U}_\tau^{\text{forget}} \cup \{u'\}, \{x'_{u,\tau}\}_{u \in \mathcal{U}_\tau^{\text{forget}} \cap \mathcal{C}}, \mathcal{F}_{\text{Forget}, \tau})$, to simulate the aggregation performed for the ciphertexts given by $\mathcal{U}_\tau^{\text{forget}} \cup u'$, and will output simulated outputs from all non corrupted clients involved (i.e. $u \in (\mathcal{U}_\tau^{\text{forget}} \cup u') \setminus \mathcal{C}$), specifically the parts corresponding to the corrupted clients (i.e. $w \in \mathcal{C}$). Note that the simulator generates a new ciphertext for the forgotten client u' , for which we want to reuse the ciphertext generated during the protect phase instead. Despite this, due to st_{TSA} and st'_{TSA} , we know that $\widetilde{y}'_{u',\tau}^{(S)}, \{\widetilde{y}'_{u',\tau}^{(v)}\}_{v \in \mathcal{C}}$ encodes some value $\widetilde{x}'_{u',\tau}$ and $\widetilde{y}'_{u',\tau}^{(S)}, \{\widetilde{y}'_{u',\tau}^{(v)}\}_{v \in \mathcal{C}}$ encodes some value $\widetilde{x}'_{u',\tau}$. As such, we know that the set $\{\widetilde{y}'_{u',\tau}^{(S)}, \widetilde{y}'_{u,\tau}^{(S)}, \widetilde{y}'_{u',\tau}^{(v)}, \widetilde{y}'_{u,\tau}^{(v)}\}_{u \in \mathcal{U}_\tau^{\text{forget}} \setminus \mathcal{C}, v \in \mathcal{C}}$ together with the ciphertexts generated by the corrupted clients encode the value $\mathcal{F}_{\text{Forget}, \tau} - (\widetilde{x}'_{u',\tau} - \widetilde{x}_{u',\tau})$. Therefore, to maintain coherence, upon receiving from the corrupted clients $\{y'_{v,\tau}^{(u)}\}_{v \in \mathcal{C}, u \in (\mathcal{U}_\tau^{\text{forget}} \cup u') \setminus \mathcal{C}}$, the simulator calls $\{\widetilde{ak}'_{u,\tau}\}_{u \in (\mathcal{U}_\tau^{\text{forget}} \cup u') \setminus \mathcal{C}} \leftarrow \text{Sim}_{\mathcal{C}, \text{Server}}^{\text{Client.TSA.Agg}}(pp, \tau, \mathcal{U}_\tau^{\text{forget}} \cup u', \text{st}'_{\text{TSA}}, \{x'_{u,\tau}\}_{u \in \mathcal{C}}, \{y'_{v,\tau}^{(u)}\}_{v \in \mathcal{C}, u \in (\mathcal{U}_\tau^{\text{forget}} \cup u') \setminus \mathcal{C}}, \mathcal{F}_{\text{Forget}, \tau} - (\widetilde{x}'_{u',\tau} - \widetilde{x}_{u',\tau}))$. Note that, once again, the simulator has generated a temporary aggregation key for client u' even if it should be done by the server.

Finally, the simulator computes $ak_{u'}$ such that $\widetilde{ak}'_{u',\tau} \leftarrow \text{Client.TSA.Agg}(pp_{\text{TSA}}, \tau, ak_{u'}, \widetilde{y}'_{u',\tau}^{(u')}, \{\widetilde{y}'_{u,\tau}^{(u')}\}_{u \in \mathcal{U}_\tau^{\text{forget}}})$ and simulates secret shares of this value. The view of the adversary in this hybrid is indistinguishable from H_1 due to the security of Π_{TSA} and the secrecy of SS .

Hybrid H_3 : In this hybrid, we finish the simulation by simulating the noisy inputs of the corrupted clients. The simulator simply samples $\{\tilde{x}_{u,\tau}\}'_{u \in \mathcal{C}} := \{-(x_{agg,\tau}/k) + z'_{u,\tau}\}_{u \in \mathcal{C}}$ with $z'_{u,\tau} \leftarrow D'$ and runs the TSA simulator over this. The view of the adversary in this hybrid is indistinguishable from H_2 since the inputs are sampled from the same distribution.

Observe that this last hybrid is the simulator for TSA-Forget. This concludes the proof. $\square$

4.5 Privacy-Preserving Federated Unlearning

In this section, we describe the newly proposed privacy-preserving FU solution which consists of combining TSA with forget capabilities and FedRecovery+.

4.5.1 Our PPFU Construction

Our PPFU protocol consists of two phases: (1) the *Setup Phase* whereby clients are registered and each of them receives its keying material and (2) the *Online Phase* during which aggregation takes place across several FL rounds and at most one unlearning occurs. It is assumed that a secure channel exist between the aggregator (server) and each client and a virtual one for each pair of clients (u, v) (through authenticated key agreement), as instantiated in [81, 22].

Under our threat model, the PPFU scheme is designed such that the server does not have access to individual client gradients in cleartext. Therefore, the residuals δ_k must be computed using only the gradients of the *to-forget* client u' together with publicly available information. Specifically, we obtain the following lemma.

Lemma 4.3. *For any $k \in [\tau - 1]$, we have:*

$$\delta_k = \frac{w_k - w_{k+1} - \eta \cdot \nabla \overset{\circ}{f}_{u'}(w_k)}{n - 1}$$

Proof. By using the definition of $\nabla F(w_k)$ in Equation 4.4:

$$\begin{aligned}
\delta_k &= \frac{\eta}{n} \left(\frac{1}{n-1} \sum_{u \in \mathcal{U}/u'} \nabla f_u(w_k) - \nabla f_{u'}(w_k) \right), \\
&= \frac{\eta}{n} \left(\frac{1}{n-1} (n \cdot \nabla F(w_k) - \nabla f_{u'}(w_k)) - \nabla f_{u'}(w_k) \right), \\
&= \frac{\eta}{n} \left(\frac{n}{n-1} \nabla F(w_k) - \left(\frac{1}{n-1} + 1 \right) \nabla f_{u'}(w_k) \right), \\
&= \frac{\eta}{n} \left(\frac{n}{n-1} \cdot \frac{w_k - w_{k+1}}{\eta} - \frac{n}{n-1} \nabla f_{u'}(w_k) \right), \\
&= \frac{w_k - w_{k+1} - \eta \cdot \nabla f_{u'}(w_k)}{n-1},
\end{aligned}$$

□

Lemma 4.3 implies that the whole unlearning procedure can be performed by client u' , solely, which may be of interest for some use-cases. In the next section, we describe a solution for the case where the unlearning work is offloaded to the server, which may be of interest for cases where clients are resource-constrained and/or client u' is less trusted and can, for example, poison the data.

4.5.1.1 Setup Phase

The protocol starts with the setup phase which comprises two steps: *Registration* and *Key Setup*. Each of these steps is executed only once. At the registration step, on request by client $u \in \mathcal{U}$, a trusted dealer (TD) calls `TSA.Setup` (from 4.4.1) to generate the encryption key ek_u , the aggregation key ak_u and the forget key fk_u , along with the relevant public parameters pp . We recall that an authenticated channel is defined for each pair of clients and between each client and the server as depicted in Figure 3.1.

4.5.1.2 Online Phase

The online phase consists of two sub-protocols: *Learning Round* executed as much as needed to train a model of quality and *Unlearning Round* called only once. It is shown in Figure 4.6.

Learning Round. Each learning round consists of two steps: *Protection* and *Aggregation*. Similar to the original TSA protocol, during the protection

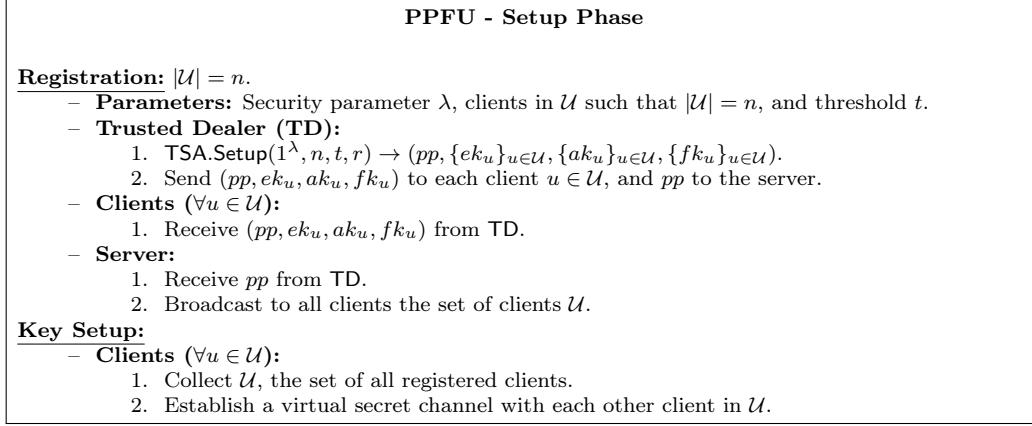


Figure 4.5: Description of the *Setup Phase* of PPFU

step, clients encrypt their inputs (which are their clipped gradients) and send them to the server. Then, during the aggregation step, clients interact with the server to account for any dropped ones, so that the server can perform the aggregation correctly.

More specifically, during protection at round τ , each client u trains the current model on its local dataset to obtain a new local update $\nabla f_u(w_{\tau-1})$, and clips it to C_τ giving $\nabla \mathring{f}_u(w_{\tau-1})$. Client u then encrypts this update using the algorithm **TSA.Protect**, which produces a set of ciphertexts $y_{u,\tau} = (y_{u,\tau}^{(S)}, \{y_{u,\tau}^{(v)}\}_{v \in \mathcal{U}})$, where $y_{u,\tau}^{(S)}$ is destined to the server and each $y_{u,\tau}^{(v)}$ is destined to each other client v . Each client securely transmits these ciphertexts to each other corresponding party.

During aggregation at round τ , each client u receives encrypted ciphertexts $\{y_{v,\tau}^{(u)}\}_{v \in \mathcal{U}_\tau^{\text{on}}}$ corresponding to every other online client v , and deduces the set of online clients $\mathcal{U}_\tau^{\text{on}}$ from it. It then generates its one-time aggregation key $ak_{u,\tau}$ using the algorithm **Client.TSA.Agg**, and forwards it to the server. Finally, the server computes the aggregate $\nabla \mathring{F}(w_{\tau-1})$ from the inputs $\nabla \mathring{f}_u(w_\tau)$ of online clients, with the help of their one-time aggregation keys $ak_{u,\tau}$ and using the algorithm **Server.TSA.Agg**. Finally, it computes the new global model $w_\tau = w_{\tau-1} - \eta/|\mathcal{U}_\tau^{\text{on}}| \cdot \nabla \mathring{F}(w_{\tau-1})$.

Unlearning Round. During an unlearning round τ , on receiving an unlearning request from client u' , the server computes the value $\tilde{x}_{agg} = (1/(n-1)) \sum_{k=1}^{\tau-1} (w_k - w_{k+1})$ based on the global models computed for all rounds up to τ and homomorphically computes the sum of the ciphertexts of the unlearned client in all previous rounds $\tilde{Y}_{u', \boxtimes_{k=1}^{\tau-1} k} := \boxplus_{k=1}^{\tau-1} \eta/(n-1) \cdot Y_{u',k}$. Both $\tilde{x}_{agg}$ and $\tilde{Y}_{u', \boxtimes_{k=1}^{\tau-1} k}$ are forwarded to the online clients. Then, each online

client u (excluding the *to-forget* client u') executes **Client.TSA.Forget** on inputs $\tilde{y}_{u', \boxtimes_{k=1}^{\tau-1} k}^{(u)}$, the aggregate $\tilde{x}_{agg}$, and the distribution $\mathcal{D} \sim \mathcal{N}(0, \sigma^2 \mathbb{I}_d)$ for σ obtained combining Equations 4.5 and 4.14. Note that $\mathcal{D}$ is t -divisible with the t -division being $\mathcal{D}' \sim \mathcal{N}(0, (\sigma^2/t) \mathbb{I}_d)$. Based on these values, the client obtains a one-time forget key $fk_{u, \tau}$ which is forwarded to the server. Upon receiving these forget keys from t clients, the server executes **Server.TSA.Forget** to obtain x'_{agg} , computes the unlearned model $w_u := w_\tau - x'_{agg}$ and finally outputs the clipped unlearned model $w_u := \min(1, \gamma/\|w_\tau - x'_{agg}\|) \cdot (w_\tau - x'_{agg})$.

Theorem 3. *The PPFU protocol described in Figures 3.1 and 4.6 correctly instantiates both a FL round as well as a FU round as long as TSA is a correct round-based ciphertext-homomorphic scheme.*

Proof. With respect to the correctness of the federated learning rounds, correctness of PPFU comes directly from the correctness of TSA with Forget functionality.

Regarding the correctness of unlearning, the ciphertext homomorphism property of **TSA-Forget** implies that $\tilde{y}_{u', \boxtimes_{k=1}^{\tau-1} k}$ is a valid ciphertext of $(\eta \cdot p_k)/(n-1) \sum_{k=1}^{\tau-1} \nabla f(w_k)$, and as such, given the correctness of TSA with Forget functionality we get

$$\begin{aligned}
w_u &= w_\tau - x'_{agg}, \\
&= w_\tau - \left(\frac{1}{n-1} \sum_{k=1}^{\tau-1} (w_k - w_{k+1}) + \frac{\eta}{n-1} \sum_{k=1}^{\tau-1} \nabla f(w_k) + z \right), \\
&= w_\tau - \sum_{k=1}^{\tau-1} \frac{w_k - w_{k+1} - \eta \cdot \nabla f_{w'}(w_k)}{n-1} - z, \\
&= w_\tau - \sum_{k=1}^{\tau-1} \delta_k - z,
\end{aligned}$$

with $z \leftarrow \mathcal{D}$. Finally, the fact that the distribution $\mathcal{D}$ is symmetric with respect to 0 and unlearning from FedRecovery+ is correct (Theorem 1), results in a correct unlearning protocol. $\square$

4.6 Experimental Evaluation

In this section, we evaluate the performance of the proposed PPFU framework. We first assess the effectiveness of the unlearning process by evaluating model accuracy when unlearned, retrained, and trained from scratch without

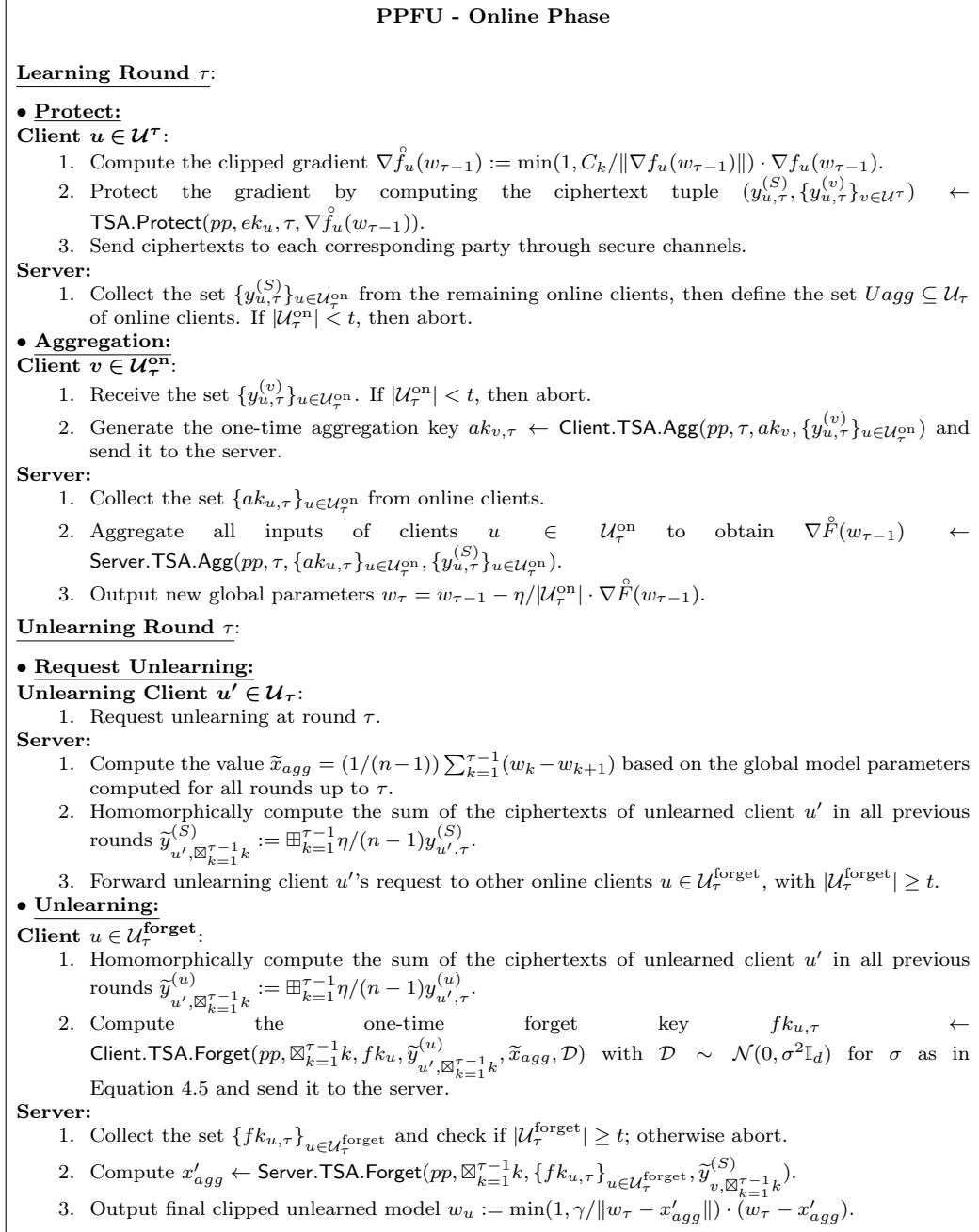


Figure 4.6: Description of the PPFU Online Phase.

the unlearned client. We further compare our results with the previous Fe-dRecovery framework [1]. We also evaluate the quality of the unlearning with respect to its strength against Membership Inference Attacks (MIAs) [13]. Finally, we measure the performance of our solution in terms computation

and communication costs.

4.6.1 Experimental Setup

We implement our framework using a machine equipped with an AMD EPYC 7272 processor, 128 GB RAM, and 4 NVIDIA GeForce RTX 3090 GPUs. The underlying cryptographic primitives use the same building blocks and libraries as in [22]. The implementation was carried out in Python 3.10 using PyTorch². Our framework is evaluated with three distinct datasets widely used in FL benchmarks: the MNIST dataset [82], the larger CIFAR-10 dataset [83] and the smaller USPS dataset [84]. The FL model consists of a Convolutional Neural Network (CNN) model with standard architecture consisting of two convolution layers followed by two fully connected layers for the MNIST and USPS datasets and three fully connected layers for the CIFAR-10 dataset. The model is trained across 10 clients under both independent and identically distributed (IID) and non-independent and identically distributed (non-IID) data settings, and using mini-batch SGD [85] with a batch size of 128. The datasets are distributed using Dirichlet values of 10 and 1 for IID and non-IID settings, respectively. The learning rate is set to 0.001 for the MNIST dataset, and 0.01 for both CIFAR-10 and USPS datasets. For the computation of round-based clipping, the parameters were chosen as described in Table 4.1. These parameters have been chosen empirically to achieve the best accuracy and utility trade-off.

Table 4.1: Parameters for clipping used in evaluation.

	ℓ_1	ℓ_2	$\tau_{\max}$	τ_{flip}	C_{tot}	$C_{\min}$
MNIST	10	1	100	30	1	0.003
CIFAR-10	10	5	200	120	15	0.0375
USPS	10	1	100	30	0.5	0.0005

For the unlearning analysis, the unlearning budget takes $\alpha = 1$ and $\alpha = 5$ as representatives for high and low budget respectively, with a fixed $\beta = 10^{-5}$, since it is the standard value for DP applications [86].

Note on the datasets We evaluate our approach on MNIST, CIFAR-10, and USPS which are not privacy-sensitive. Nevertheless, these datasets provide diverse and reproducible benchmarks for evaluating the effectiveness of our PPFU proposal across different data distributions and model architectures. The unlearning mechanism operates on encrypted model updates and

²Code will be released as open-source upon acceptance.

gradients of the clients rather than on dataset-specific properties. Therefore, the core findings can be easily transferred to privacy-sensitive FL scenarios, such as healthcare or financial applications, where removing a client’s contribution is required to satisfy "right-to-be-forget" and other privacy requirements.

4.6.2 Unlearning Analysis

4.6.2.1 Effectiveness of the PPFU framework

For the sake of comparison, we consider four settings: (i) the original model w trained with 10 clients; (ii) the model $\tilde{w}$ retrained from scratch with 9 clients and without the input of the unlearned client u ; (iii) the unlearned model w_u at round τ with our solution integrating FedRecovery+; (iv) the unlearned model w_u retrained with additional rounds to improve accuracy. Table 4.2 presents the evaluated numbers for these four cases with all the above mentioned datasets, unlearning budgets $\alpha = 1$ and $\alpha = 5$ and training rounds τ .

MNIST. On MNIST, the scratch model $\tilde{w}$ achieves around 88-90% accuracy across both IID and non-IID settings, with slightly lower performance under non-IID due to increased data heterogeneity. After unlearning, the model w_u drops to roughly 80-89%, especially when privacy is tight ($\alpha = 1$) or data is non-IID. This is due to the inherent loss of information due to randomly perturbing the model. However, retraining for just 12-25 extra rounds pushes the accuracy above 91%. With 25–100 more rounds, it reaches 94–98% and even beats the scratch model $\tilde{w}$ (as shown in green). In many cases, retraining for only half the original number of rounds (e.g., +50 rounds when the model was trained for $\tau = 100$ rounds) is enough to match or exceed the baseline, demonstrating substantial computational savings.

CIFAR-10. CIFAR-10 is more challenging due to its more complex images and classification task. The baseline accuracy is around 60–65%, but after unlearning, w_u drops to 8–25%. This drop is especially severe with $\alpha = 1$, where the added noise causes a larger drop in model accuracy. Retraining improves the accuracy, but recovery is slower than for MNIST. With $\alpha = 5$, 50-200 extra rounds recover accuracy to 62-70%, often matching or exceeding accuracy from $\tilde{w}$. With $\alpha = 1$, even 200 rounds only reach 64–69%. This shows that more complex datasets may require more retraining to recover from stronger privacy noise.

USPS. USPS behaves similarly to MNIST, with a smaller difference between IID and non-IID performance. The baseline $\tilde{w}$ is around 86-87%, while w_u drops to 42-86%. The drop is smaller with larger α values and IID data.

Table 4.2: Accuracy for models trained from scratch ($\tilde{w}$), unlearned with Fe-dRecovery+ (w_u) and fine-tuned with extra learning rounds after unlearning (Retrain). Cells are colored in green when the model is more accurate than $\tilde{w}$.

		τ	extra rounds	Accuracy			
				IID		Non-IID	
				$\alpha = 1$	$\alpha = 5$	$\alpha = 1$	$\alpha = 5$
MNIST Dataset	w	25	–	0.885		0.854	
	$\widetilde{w}$		+25	0.883		0.841	
	w_u		+1	0.862	0.881	0.831	0.854
	Retrain		+12	0.930	0.930	0.919	0.918
			+25	0.950	0.949	0.944	0.943
	w	50	–	0.893		0.872	
	$\widetilde{w}$		+50	0.891		0.866	
	w_u		+1	0.878	0.891	0.858	0.871
	Retrain		+25	0.952	0.952	0.944	0.943
			+50	0.973	0.972	0.967	0.968
	w	100	–	0.898		0.883	
	$\widetilde{w}$		+100	0.896		0.877	
	w_u		+1	0.853	0.895	0.809	0.879
	Retrain		+50	0.972	0.973	0.968	0.968
			+100	0.982	0.982	0.979	0.979
CIFAR-10 Dataset	w	50	–	0.609		0.593	
	$\widetilde{w}$		+50	0.610		0.607	
	w_u		+1	0.109	0.248	0.088	0.228
	Retrain		+25	0.508	0.632	0.500	0.602
			+50	0.569	0.661	0.557	0.628
	w	100	–	0.654		0.637	
	$\widetilde{w}$		+100	0.653		0.653	
	w_u		+1	0.105	0.109	0.096	0.125
	Retrain		+50	0.578	0.647	0.545	0.586
			+100	0.639	0.686	0.604	0.657
	w	200	–	0.652		0.652	
	$\widetilde{w}$		+200	0.650		0.655	
	w_u		+1	0.093	0.106	0.101	0.100
	Retrain		+100	0.638	0.652	0.593	0.645
			+200	0.688	0.699	0.646	0.672
USPS Dataset	w	25	–	0.868		0.853	
	$\widetilde{w}$		+25	0.864		0.855	
	w_u		+1	0.468	0.860	0.451	0.849
	Retrain		+12	0.941	0.941	0.940	0.940
			+25	0.947	0.950	0.945	0.946
	w	50	–	0.870		0.864	
	$\widetilde{w}$		+50	0.869		0.865	
	w_u		+1	0.629	0.849	0.518	0.829
	Retrain		+25	0.950	0.952	0.945	0.948
			+50	0.953	0.955	0.950	0.951
	w	100	–	0.871		0.862	
	$\widetilde{w}$		+100	0.870		0.864	
	w_u		+1	0.420	0.845	0.457	0.836
	Retrain		+50	0.952	0.953	0.951	0.950
			+100	0.953	0.954	0.952	0.954

Retraining is highly effective, 12-25 extra rounds can increase accuracy above 94%, while 50-100 rounds achieve around 95–96%, consistently exceeding the baseline. Similar to MNIST, half of the original training rounds (e.g., +50 rounds when $\tau = 100$) are enough to achieve near-optimal performance.

Across all datasets a larger unlearning budget $\alpha = 5$ results in better accuracy and faster recovery, as less noise is added during unlearning. In contrast, a smaller unlearning budget $\alpha = 1$ provides stronger unlearning guarantees but may require more retraining rounds. Overall, for MNIST and USPS datasets, retraining for around half of the original training rounds often achieves accuracy comparable to or better than training from scratch. CIFAR-10 shows similar recovery under IID data and larger unlearning budget $\alpha = 5$, while more challenging settings, such as non-IID data and smaller $\alpha = 1$, may require additional retraining rounds or better clipping parameters.

4.6.2.2 Comparison with FedRecovery

In addition to PPFU, we implemented FedRecovery [1] for comparative evaluation. Table 4.3 compares FedRecovery [1] and FedRecovery+ across the MNIST, CIFAR-10 and USPS datasets. The original models for both methods were trained for 100 rounds on MNIST and USPS, and 200 rounds on CIFAR-10. After unlearning, each model was retrained for the same number of rounds. While FedRecovery+ quickly recovers the original model’s accuracy, FedRecovery often fails to converge after unlearning. This improvement is mainly due to the proposed modifications to σ and the clipping parameter C described in Section 4.3.5. However, clipping slows model convergence during the initial training phase before unlearning. The significantly large σ pushes the unlearned model by FedRecovery outside the convergence basin of the problem, i.e., any retraining will not bring the model closer to convergence. Furthermore, we see that retraining the unlearned model for the same number of rounds as the original training achieves higher accuracy than the original model.

4.6.2.3 Unlearning quality with MIA

We evaluate whether the proposed PPFU framework successfully removes the influence of the unlearned client’s data from the model, by implementing a MIA and evaluating its success. We follow the strategy from [61] and implement the MIA from [87]. The main evaluation metric is attack precision which measures the proportion of unlearned client data predicted as members of the training set.

Table 4.3: Comparison for FedRecovery and FedRecovery+.

			Acc. w	σ	Acc. Retrain
MNIST	FedRecovery	$\alpha = 1$	0.982	4.703	0.098
		$\alpha = 5$		1.102	0.098
	FedRecovery+	$\alpha = 1$	0.898	0.010	0.982
		$\alpha = 5$		0.003	0.982
CIFAR-10	FedRecovery	$\alpha = 1$	0.701	21.041	0.098
		$\alpha = 5$		4.527	0.100
	FedRecovery+	$\alpha = 1$	0.651	0.980	0.688
		$\alpha = 5$		0.211	0.699
USPS	FedRecovery	$\alpha = 1$	0.954	14.889	0.179
		$\alpha = 5$		3.204	0.179
	FedRecovery+	$\alpha = 1$	0.871	0.049	0.953
		$\alpha = 5$		0.011	0.954

Figure 4.7 shows the results of the MIA evaluation under different privacy budgets $\alpha = 1$ and $\alpha = 5$. Values closer to 0.5 indicate stronger protection against MIA. The original model achieves a large attack precision of approximately 0.85 across the three datasets, demonstrating that the attacker can effectively infer whether a sample was part of the training dataset or not. After applying the proposed unlearning method, this value significantly decreases to approximately 0.49–0.52, which is close to random guessing (i.e., 0.50). This indicates that the unlearned model successfully removes the influence of the unlearned client’s data, and hence provides strong privacy protection. Furthermore, the values seem identical across both α settings (i.e., $\alpha \in \{1, 5\}$), demonstrating that retraining does not leak any further information from the unlearned client. Note that the lack of significant change in these values with different α is explained by the fact that the bound in Equation 4.14 is not tight.

Overall, these results demonstrate the quality of the unlearning within our proposed PPFU framework while maintaining constant MIA performance across different α values.

4.6.3 PPFU Performance

To evaluate the performance of our solution, we first measure the runtime of TSA with forget functionality (for each functionality) and further extend our study with a full evaluation of PPFU executed with MNIST, 10 clients and 100 rounds.

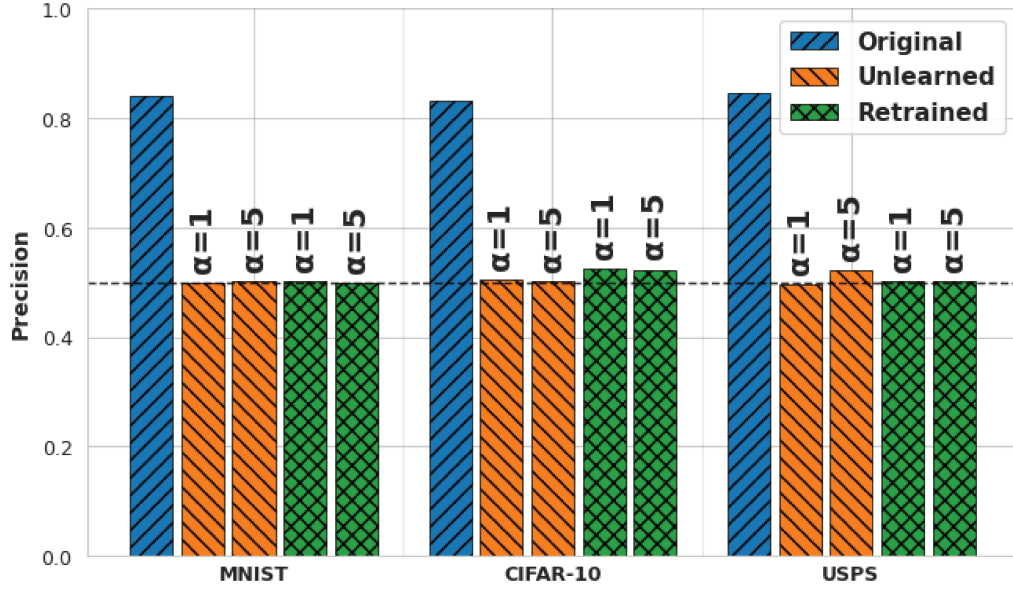


Figure 4.7: Performance of MIA.

4.6.3.1 TSA with Forget Functionality

The results of the construction of TSA with the forget functionality are shown in Table 4.4a. As expected, the runtime values for **Setup**, **Protect** and **Agg** phases directly inherit from the original (round-based homomorphic) TSA construction. In this case, it is TJJL [22], for which **Protect** is more time consuming for the client and **Agg** for the server. More interestingly, we notice that **Forget** takes approximately the same time as **Protect** at the client side and **Agg** at the server side, as expected due to the construction. Regarding communication overhead, the **Forget** phase requires transmitting approximately 4.265 MB per online client, which is marginally higher than the **Protect** phase with 4.261 MB, and substantially larger than the **Agg** and **Setup** phases with 0.001 MB and 0.026 MB, respectively. This increase is expected, as the client must transmit additional share reconstruction data to the server during the **Forget** process. Overall, these results demonstrate that the **Forget** functionality is efficiently integrated, with costs that remain consistent with the cryptographic operations and communication patterns of the underlying TSA scheme.

Table 4.4: Benchmarks for TSA with Forget functionality and PPFU Frameworks with parameter size of 421642.

(a) TSA with Forget functionality Benchmarks

	Setup	Protect	Agg	Forget
Server time (s)	-	-	42.060	50.571
Client time (s)	0.003	39.680	0.008	39.810
Comm. Cost (MB) (per online client)	0.026	4.261	0.001	4.265

(b) PPFU Benchmarks

	Learning Round	Unlearning Round
Server time (s)	42.060	50.671
Client time (s)	42.860	39.920
Comm. Cost (MB) (per online client)	4.262	4.265

4.6.3.2 PPFU

The runtime of the proposed PPFU framework during the learning and unlearning phases is depicted in Table 4.4b. The client-side learning cost includes both the local training and encryption operations, while the server-side learning cost mainly corresponds to aggregating model updates from all participating clients during training. The unlearning cost, as derived from PPFU (see Figure 4.6), corresponds to the execution of **Forget** and homomorphic operations at both sides.

Firstly, we observe that the cost of unlearning is very close to the cost of learning, and therefore does not incur significant cost. These measurements correspond to the cost for one round only. Note that, when compared with the training of the model from scratch (without the unlearned client), the overhead will be very significant. We have shown in Table 4.2 that half of the original number of rounds is sufficient to reach the accuracy of the model trained from scratch. As a result, PPFU can recover a model with accuracy comparable to retraining from scratch while requiring around $2\times$ lower computational cost. From a communication perspective, the unlearning round requires 4.265 MB per online client, which is marginally higher than the learning round with 4.262 MB. This modest increase reflects the additional data exchanged during the unlearning round, namely the share reconstruction information sent by the client. Overall, these benchmarks demonstrate that the PPFU framework introduces only a minimal overhead

in both computation and communication when transitioning from learning to unlearning, making it a practical solution for FL scenarios requiring efficient data removal capabilities.

4.7 Conclusion and Future Work

In this work, we study the design of a privacy-preserving FU protocol. We first propose FedRecovery+ which enhances the learning phase with gradient clipping to obtain a more accurate unlearned model. We further extend TSA with the forget functionality to allow the removal of the client’s input without leaking any information. Both these contributions may be of independent interest. Then, these two techniques are combined to build the new PPFU solution which is further evaluated through the analysis of the trade-off between the proposed unlearning model and a model that is retrained from scratch without the input of the unlearning client, both in terms of model accuracy, computation time, and privacy leakage.

These contributions open the door to interesting future avenues of research. One of these avenues would be to find a theoretical and systematic process to choose the hyperparameters for clipping depending on the problem and training data properties, or to extend FedRecovery+ to allow for more than one unlearning round (with different clients to unlearn). Another one would be to extend TSA with forget functionality to the committee member setting, recently proposed in [29], which allows for a lower threshold of aggregation keys than that of ciphertexts, thus significantly reducing the aggregation (and therefore forgetting) cost.

Chapter 5

Privacy-preserving and Fairness-aware Secure Aggregation in Federated Learning

This chapter addresses the third challenge, **C3**, by jointly considering **privacy protection** and **fairness** in Federated Learning (FL). As discussed in Chapter 1, fairness-aware FL may require the aggregation process to account for client-specific information, such as sensitive attributes (e.g., gender, age, or ethnicity) in local datasets. This information can be used to derive fairness-aware aggregation weights, which determine how individual client updates contribute to the global model while accounting for the representation of different sensitive groups. Since such information is sensitive, it must remain private. However, protecting it through Secure Aggregation (SA) prevents the server from accessing the individual statistics required to compute these weights. To address these challenges, this chapter proposes a privacy-preserving and fairness-aware FL approach that securely derives fairness-aware aggregation weights from private client statistics without revealing the underlying information.

The work presented in this chapter was accepted and published at the Privacy in Statistical Databases 2026 conference - PSD 2026.

5.1 Introduction

As discussed in Chapter 1, fairness is an important requirement for trustworthy FL. In particular, achieving fair model performance may require the

aggregation process to account for client-specific information, such as the distribution of sensitive attributes in local datasets. Such information, however, can itself be sensitive. For example, statistics describing sensitive attributes, such as gender, age, or ethnicity, may reveal information about the demographic composition of a client’s local dataset [88].

In standard FedAvg [12], client updates are weighted according to the number of local training samples. Consequently, clients with larger datasets may have a greater influence on the global model, which can contribute to disparities in model performance across clients or groups. Fairness-aware aggregation can mitigate this issue by assigning client-specific aggregation weights based on the distribution of sensitive attributes in their local datasets. Computing these weights therefore requires access to client-level statistical information that should not be disclosed to the server.

While SA [81] protects the confidentiality of individual model updates by revealing only their aggregate to the server, it does not directly support the computation of data-dependent fairness-aware aggregation weights. The server requires client-level statistics, such as the number of samples associated with different sensitive groups (e.g., the number of female and male samples in a client’s local dataset), to derive these weights. Revealing such statistics compromises their privacy, whereas keeping them private prevents the server from directly using them to determine the aggregation weights. Handling client dropouts introduces additional challenges, as some clients may become unavailable before contributing to the aggregation. Their absence can affect the computation of fairness-aware aggregation weights, particularly when these weights depend on statistics collected across participating clients.

One solution named PrivFairFL-Pre [88], addresses the challenge of computing fairness-aware aggregation weights in a privacy-preserving manner. It uses Multi-Party Computation (MPC) to compute a fairness-based weighted average of model updates without directly revealing the underlying client statistics. However, PrivFairFL-Pre has limitations in terms of fairness, real-world deployment, and computational performance. In this Chapter, we study this solution more in depth and identify that both the fairness-weight calculation and the underlying privacy-enhancing technology can be improved. Our solution, named SaFer-FL, combines SA with an improved fairness-aware weight computation. It also provides robustness against client dropouts by allowing the aggregation process to handle clients that become unavailable during training. To further protect the aggregated model against global membership inference attacks (MIA) [13], SaFer-FL enables clients to apply a Differential Privacy (DP) mechanism. Each client adds noise to its local model update before protecting it with SA. Consequently, the server obtains only a noisy aggregated global model, providing an additional layer

of privacy protection. Our experimental evaluation shows that SaFer-FL improves fairness while preserving the privacy of both client updates and the statistical information used to derive the fairness-aware aggregation weights.

The main contributions of this chapter are:

- We propose an improved fairness-aware aggregation weight computation that more accurately accounts for differences in clients’ local data distributions.
- We integrate the proposed fairness-aware weight computation with a dropout-robust SA protocol, enabling fairness-aware model aggregation while protecting individual client updates and supporting client dropouts.
- We incorporate a dedicated DP mechanism to provide an additional layer of protection against inference attacks on the aggregated global model.
- We implement *SaFer-FL* and evaluate its fairness, model utility, privacy, computational and communication overhead.

5.2 Related Work

Fairness has emerged as an important requirement in FL alongside privacy and robustness. In heterogeneous FL settings, differences in the distribution of data across clients can lead to disparities in model performance across clients or sensitive groups [89]. Fairness-aware FL therefore considers client-specific characteristics when determining how model updates contribute to the global model.

One approach to improving fairness is to modify the aggregation objective so that clients experiencing poorer model performance receive greater consideration. In this direction, [89] proposes q -FFL, which formulates FL fairness using an objective inspired by α -fair resource allocation. Instead of weighting client updates solely according to their local dataset sizes, q -FFL assigns greater importance to clients with higher losses, aiming to reduce disparities in model performance across clients.

Another approach is to address disparities through personalized models rather than modifying only the global aggregation weights. Following this direction, [90] proposes DITTO, a personalized FL framework that learns a personalized model for each client alongside a global model. It uses a regularized local objective to balance personalization and the global model

and evaluates its ability to improve fairness across clients while providing robustness against poisoning attacks. Unlike our setting, these approaches operate on model updates that are accessible to the aggregation server and do not provide cryptographic protection for the aggregation process.

Privacy mechanisms can also have a direct impact on fairness, particularly when noise is introduced during model training. The work in [91] investigates this interaction and shows that applying DP-SGD can increase existing disparities in model performance across different groups, particularly when some groups are under-represented.

Building on the observation that privacy and fairness can interact in complex ways, broader analyses have examined how these objectives can be jointly addressed. The survey by [92] systematically studies the relationship between DP and fairness, identifying settings in which the two objectives may complement or conflict with each other and discussing approaches for mitigating potential adverse effects.

Among existing privacy-preserving fairness approaches, one work is particularly close to our setting because it considers the private computation of fairness-aware aggregation. *PreFairFL* [88] uses Multi-Party Computation (MPC) to compute a fairness-based weighted average of client model updates without directly revealing the underlying client statistics. However, its reliance on MPC introduces additional communication and coordination overhead, as the computation involves multiple computing parties rather than a single aggregation server. In addition, the fairness-weight computation can be improved to better account for the distribution of sensitive attributes across clients.

To the best of our knowledge, no existing protocol simultaneously addresses SA, DP, and fairness within a single unified framework, which is the gap our work aims to fill.

5.3 Background

5.3.1 Fairness in Machine Learning

Machine learning (ML) models learn from the past and when that past is unequal, they tend to reproduce, and sometimes amplify, the same inequalities at scale. In high-stakes domains such as credit, hiring, and criminal justice, this means decisions can systematically disadvantage demographic groups defined by sensitive attributes such as gender, race, or age. Group fairness formalises this concern by requiring the model’s predictions and error rates to be similar across these groups. Although group-fairness criteria

can be defined for different types of ML models, this work focuses on binary classification, where the ground-truth label and model prediction take binary values. The following definitions therefore consider binary classifiers and binary sensitive groups, while the underlying fairness criteria can also be extended to more general settings.

Let $\mathcal{R}$ denote the set of groups defined by a sensitive attribute r , let $l \in \{0, 1\}$ denote the ground-truth label, and let $\hat{l} \in \{0, 1\}$ denote the model prediction. Two commonly used group-fairness criteria are Equalized Odds and Equal Opportunity.

Definition 5.3.1 (Equalized Odds). *A binary classifier satisfies Equalized Odds with respect to the sensitive attribute if, for every pair of groups $r_i, r_j \in \mathcal{R}$ with $i \neq j$,*

$$\begin{aligned}\Pr(\hat{l} = 1 \mid l = 1, r = r_i) &= \Pr(\hat{l} = 1 \mid l = 1, r = r_j), \\ \Pr(\hat{l} = 1 \mid l = 0, r = r_i) &= \Pr(\hat{l} = 1 \mid l = 0, r = r_j).\end{aligned}$$

The first condition requires the same true-positive rate (TPR) across groups, while the second requires the same false-positive rate (FPR). For example, if gender defines the groups, with r_i = female and r_j = male, a model satisfies Equalized Odds if female and male individuals have the same probability of being correctly classified as positive when $l = 1$, and the same probability of being incorrectly classified as positive when $l = 0$.

Definition 5.3.2 (Equal Opportunity). *A binary classifier satisfies Equal Opportunity with respect to the sensitive attribute if, for every pair of groups $r_i, r_j \in \mathcal{R}$ with $i \neq j$,*

$$\Pr(\hat{l} = 1 \mid l = 1, r = r_i) = \Pr(\hat{l} = 1 \mid l = 1, r = r_j).$$

Thus, Equal Opportunity requires only the TPR to be equal across groups and does not impose a constraint on their FPR. For example, if gender defines the groups, Equal Opportunity requires that female and male who truly belong to the positive class ($l = 1$) have the same probability of being correctly predicted as positive.

5.3.2 PrivFairFL-Pre

PrivFairFL-Pre [88] is a privacy-preserving bias mitigation technique designed for FL environments. The method aims to reduce demographic bias before model training by reweighting training samples according to the distribution of sensitive attributes and class labels across all participating clients. Unlike traditional centralized fairness methods, *PrivFairFL-Pre* achieves this without requiring clients to reveal their raw data or sensitive attributes to a central server.

The framework combines FL and MPC. Each client locally computes counts of samples belonging to different sensitive groups and label categories, and secret-shares these statistics before sending them to the MPC servers. The MPC servers aggregate counts and compute fairness-aware sample weights while never accessing the underlying sensitive information in plaintext. Formally, let each client u hold dataset $D_u = \{(s_i, l_i, r_i)\}$, where

- s_i denotes the feature vector of sample i .
- $l_i \in \{0, 1\}$ is the corresponding class label (binary classification).
- $r_i \in \{p, un\}$ (protected vs unprotected) represents the sensitive attribute of that sample.

Each client computes local joint counts:

$$C_u(r, l) = \sum_{(s_i, l_i, r_i) \in D_u} \mathbf{1}[r_i = r, l_i = l].$$

These local statistics are secret-shared and sent to the MPC servers. The servers securely aggregate them to obtain the global counts:

$$C(r, l) = \sum_{u=1}^n C_u(r, l).$$

The total global count is computed as:

$$N = \sum_{r \in \{p, un\}} \sum_{l \in \{0, 1\}} C(r, l).$$

Finally, fairness-aware weights are defined as:

$$p(r, l) = \frac{N}{4 C(r, l)}.$$

These weights are broadcast to all clients and used during federated training.

5.3.3 Symmetric Geometric Discrete Laplace Mechanism

A common approach to protect model updates against inference attacks is to apply local DP by adding random noise before aggregation. However, integrating conventional DP mechanisms into SA can be challenging because standard mechanisms such as Laplace and Gaussian noises are defined over the real numbers, whereas many cryptographic SA schemes, such as the one in [22], operate over an integer domain $\mathbb{Z}_q$ using modular addition. Directly representing real-valued noise in this domain requires quantization or approximation, which can introduce additional errors and complicate the analysis of the resulting privacy guarantees. An alternative is to use a discrete noise distribution that is naturally defined over the same integer domain as the cryptographic protocol. The Symmetric Geometric Discrete Laplace (SGDL) [93] distribution is an integer-valued analogue of the Laplace distribution and is therefore well suited to this setting. For a privacy budget ϵ and ℓ_1 -sensitivity Δ , an SGDL random variable $X \in \mathbb{Z}$ is parameterized by $p = \exp(-\epsilon/\Delta)$ and has probability mass function

$$\Pr[X = k] = \frac{1-p}{1+p} p^{|k|}, \quad k \in \mathbb{Z}.$$

When added to an integer-valued query with bounded ℓ_1 -sensitivity, this mechanism provides ϵ -DP while avoiding the need to represent real-valued noise. Moreover, SGDL is infinitely divisible, meaning that a single SGDL noise sample can be decomposed into k independent shares whose sum follows the same SGDL distribution. In particular, each share can be sampled as

$$\eta_i \sim \text{NB}(1/k, p) - \text{NB}(1/k, p),$$

for $i = 1, \dots, k$, such that the sum of the shares reconstructs the SGDL noise. This property makes SGDL suitable for distributed privacy mechanisms, where multiple parties can independently generate noise shares and their sum provides the required aggregate noise without relying on a trusted curator.

5.4 Privacy-preserving Fairness Aware FL

As discussed in the previous section 5.3, *PrivFairFL-Pre* [88] provides a privacy-preserving approach for fairness-aware FL. This section first presents the threat model considered in this work. We then discuss the key limitations of *PrivFairFL-Pre* and finally provide an overview of our proposed approach, explaining how it addresses each of these limitations.

5.4.1 Threat Model

We consider a threat model in which the FL server is *honest-but-curious*: it follows the prescribed protocol correctly but may attempt to infer sensitive information from the data and messages it receives. All FL clients are assumed to behave honestly throughout the training process. We further assume secure communication channels between clients and the server, ensuring that exchanged messages are encrypted and protected against external adversaries.

5.4.2 Limitations

5.4.2.1 Imbalanced Weight Normalization

The *PrivFairFL-Pre* weighting mechanism computes the normalization factor as $N = \sum_{r \in \{p, un\}} \sum_{l \in \{0, 1\}} C(r, l)$ which aggregates all samples without considering the relative size of each sensitive group. This assumes that protected and unprotected groups are equally represented across the data distribution. Consequently, the normalization term is dominated by the majority group, while minority groups receive disproportionately large weights $p(r, l) = \frac{N}{4 \cdot C(r, l)}$. Although minority groups obtain higher weights, the optimization process can still remain biased toward the majority group due to its overwhelming presence in training and client datasets. Therefore, the reweighting strategy does not fully account for cross-group population imbalance, which degrades fairness performance.

5.4.2.2 MPC-Based Solution

PrivFairFL-Pre relies on MPC to aggregate local counts: clients secret-share their values, which are then jointly processed by multiple servers to obtain the global counts. Although this approach protects the privacy of individual client statistics, it introduces a dependency on a multi-server MPC infrastructure that requires coordination among multiple servers throughout the computation. This makes deployment more complex and introduces additional communication and computational overhead, as clients must distribute secret shares to multiple servers and the servers must interact to perform the secure computation. This overhead can make the MPC-based aggregation a potential performance bottleneck in FL deployments.

5.4.3 Our Approach

Our solution addresses each of the problems identified in the previous section.

5.4.3.1 Cross-group Normalization Rule

We improve the normalization by making sure that each group’s sample count is adjusted using the *client representation* of the other group. This helps prevent the larger group from dominating the final value. Let $N_p = C(p, 0) + C(p, 1)$ and $N_{un} = C(un, 0) + C(un, 1)$ denote the global sample counts of the privileged and unprivileged groups, and let C_p and C_{un} denote the global counts of privileged and unprivileged *clients*. We define the client fractions as $\alpha = \frac{C_p}{C_p + C_{un}}$ and $\beta = \frac{C_{un}}{C_p + C_{un}}$. The improved normalization term is computed as $N_{\text{new}} = \alpha N_{un} + \beta N_p$. This way, both groups are fairly balanced when computing the final weights, so that no single group dominates the training process and fairness is improved.

5.4.3.2 SA-Based Solution

To address the overhead and deployment complexity of the MPC-based solution used in *PrivFairFL-Pre*, our approach uses SA to securely aggregate both the client statistics required for fairness-aware weighting and the local model updates. Each client locally computes its required statistics and sends them through SA, allowing the server to obtain only the aggregated global statistics without learning individual client contributions. These aggregated statistics are then used to compute the fairness metrics and derive the corresponding fairness-aware weights. Each client multiplies its local model update by its computed weight, and the resulting weighted updates are securely aggregated using SA.

Although SA protects individual client contributions from the server, the resulting global model may still reveal information about the clients’ training data [13], particularly when an honest-but-curious server observes changes in the aggregate across rounds. To address this limitation, each client additionally applies a dedicated local DP mechanism to its model update before applying the fairness-aware weight and submitting it for SA. DP therefore provides an additional layer of protection for the resulting global model against inference attacks.

5.5 SaFer-FL

In this section, we present our proposed solution, SaFer-FL, which leverages SA (Section 2.3) to privately compute fairness-aware aggregation weights from clients’ sensitive-attribute statistics and to train the global federated model in a privacy-preserving manner. SaFer-FL consists of three phases: (1) Setup Phase whereby FL clients are registered and each party receives its

keying material (2) Fairness-weight calculation Phase whereby the fairness-weights are calculated in a privacy-preserving way and (3) Training phase whereby model parameters are aggregated and updated in several FL Rounds.

5.5.1 Setup Phase

The Setup Phase comprises two steps: Registration and Key Setup. Each of these steps is executed only once. At the Registration step, on request by client $u \in \mathcal{U}$, a trusted dealer TD calls **SA.Setup** and generates the encryption keys ek_u for each client and aggregation key ak_0 for the server along with relevant public parameters pp required by the clients and server.

5.5.2 Fairness-weight Calculation Phase

During Fairness-weight Calculation phase, each FL client u first locally counts how many of its training samples fall into each of the four buckets defined by the sensitive attribute $r \in \{un, p\}$ and class label $l \in \{0, 1\}$ i.e., binary classification. Specifically, each client computes a six-element vector as

$$C_u = [C(un, 0)_u, C(un, 1)_u, C(p, 0)_u, C(p, 1)_u, \mathbb{I}[T(u) = 0], \mathbb{I}[T(u) = 1]]$$

- $C(un, 0)_u$ and $C(un, 1)_u$ denote the counts of unprotected samples with labels 0 and 1, respectively.
- $C(p, 0)_u$ and $C(p, 1)_u$ denote the counts of protected samples with labels 0 and 1, respectively.
- $T(u) \in \{0, 1\}$ indicates the group membership of client u , where $T(u) = 1$ denotes the protected group and $T(u) = 0$ denotes the unprotected group.

Each FL client u then protects C_u with the **SA** encryption key ek_u (generated in the previous Key Setup step) using **SA.Protect** and sends the resulting protected C_u to the FL server. The server performs their aggregation using **SA.Agg** and computes the global sums as

$$C = \sum_{u \in \mathcal{U}} C_u = [C(un, 0), C(un, 1), C(p, 0), C(p, 1), C_{un}, C_p]$$

- $C_{un} = \sum_i \mathbb{I}[T(i) = 0]$ represents the total number of unprotected clients.
- $C_p = \sum_i \mathbb{I}[T(i) = 1]$ represents the total number of protected clients.

From these aggregated global counts, the server computes fairness-weights that assign higher importance to unprotected groups. Specifically, the server calculates the total group sample counts as $N_p = C(p, 0) + C(p, 1)$ for the protected group and $N_{un} = C(un, 0) + C(un, 1)$ for the unprotected group. The client fractions are then computed as

$$\alpha_{un} = \frac{C_p}{C_p + C_{un}}$$

$$\alpha_p = \frac{C_{un}}{C_p + C_{un}}$$

The server, next, computes an improved normalization term $N_{\text{new}} = \alpha_{un} \cdot N_{un} + \alpha_p \cdot N_p$, which balances the two group sizes by weighting each by the proportion of the other. Finally, the server computes the fairness-weights $p(r, l)$ as

$$p(r, l) = \frac{N_{\text{new}}}{4 \cdot C(r, l)}$$

and broadcasts these weights back to all clients for use in their local training loss functions. This phase, like the Setup Phase, is executed only once, before the actual training of the FL model.

5.5.3 Training Phase

During the training phase, each client computes its local model update $w_{u,\tau}$ and scales it by its client-specific fairness-aware aggregation weight $p(r, l)_u$. The resulting weighted update is used as the input to the SA protocol: $x_{u,\tau} = p(r, l)_u \cdot w_{u,\tau}$. After computing the weighted update, the client adds DP noise calibrated to the desired privacy budget (ϵ, Δ) using the SGDL mechanism defined in Section 5.3.3. The noisy weighted updates $\tilde{x}_{u,\tau}$ are then protected using **SA.Protect** with the encryption key ek_u and forwarded the resulting protected updates $y_{u,\tau}$ to the server. The server aggregates the protected updates using **SA.Agg** with aggregation key ak_0 . Consequently, the server obtains only the aggregated values $x_{add,\tau}$, which already incorporate the added noise.

The proposed protocol can also instantiate the scheme given in Chapter 3 Section 3.2.1, to handle client dropouts in FL. This allows the aggregation process to remain correct even when some clients become unavailable during a training round.

SaFer-FL

Setup Phase:

Registration: Security parameter λ and clients in $\mathcal{U}$ such that $|\mathcal{U}| = n$.

Trusted Dealer (TD):

- 1) Run $\text{SA.Setup}(1^\lambda, n) \rightarrow (pp, \{ek_u\}_{u \in \mathcal{U}}, ak_0)$.
- 2) Send (pp, ek_u) to each client $u \in \mathcal{U}$, and (pp, ak_0) to the server.

Client $u \in \mathcal{U}$:

- 1) Receive (pp, ek_u) from TD.

Server:

- 1) Receive (pp, ak_0) from TD.
- 2) Broadcast to all clients the set of clients $\mathcal{U}$.

Key Setup:

Client $u \in \mathcal{U}$:

- 1) Collect $\mathcal{U}$ the set of all registered clients.
- 2) Establish a virtual secret channel with each other client in $\mathcal{U}$.

Fairness-weight Calculation:

Clients ($\forall u \in \mathcal{U}$):

- 1) Compute local count vector $C_u = [C(un, 0)_u, C(un, 1)_u, C(p, 0)_u, C(p, 1)_u, \mathbb{I}[T(u) = 0], \mathbb{I}[T(u) = 1]]$.
- 2) $\text{SA.Protect}(pp, \tau_0, ek_u, C_u) \rightarrow y_{u, \tau_0}$: Protect each local count vector and send y_{u, τ_0} to server.

Server:

- 1) Collect all y_{u, τ_0} from clients.
- 2) $\text{SA.Agg}(pp, \tau_0, ak_0, \{y_{u, \tau_0}\}_{u \in \mathcal{U}}) \rightarrow C$: computes the global count as $C = \sum_{u \in \mathcal{U}} C_u = [C(un, 0), C(un, 1), C(p, 0), C(p, 1), C_{un}, C_p]$
- 3) Computes the privileged and unprivileged client fractions as $\alpha_{un} = \frac{C_p}{C_{un} + C_p}$ and $\alpha_p = \frac{C_{un}}{C_{un} + C_p}$, where C_p and C_{un} are the global counts of privileged and unprivileged clients.
- 4) Computes the normalization term $N_{\text{new}} = \alpha \cdot N_{\text{un}} + \beta \cdot N_p$, with $N_p = C(p, 0) + C(p, 1)$ and $N_{\text{un}} = C(un, 0) + C(un, 1)$ the global sample counts of the protected and unprotected groups.
- 5) Computes the fairness weights as: $p(r, l) = \frac{N_{\text{new}}}{4 \cdot C(r, l)}$ and broadcasts these weights to all clients in $\mathcal{U}$.

Clients ($\forall u \in \mathcal{U}$):

- 1) Store $p(r, l)$ as fairness-aware weight for local training.

Training Round:

Clients ($\forall u \in \mathcal{U}$):

- 1) Compute the local model update $w_{u, \tau}$ starting from the previous global model $w_{\tau-1}$, and scale it by the fairness-aware aggregation weight $p(r, l)_u$.

$$x_{u, \tau} = p(r, l)_u w_{u, \tau},$$

- 2) $\tilde{x}_{u, \tau} \leftarrow x_{u, \tau} + \text{SGDL}(n = |\mathcal{U}|, \epsilon, \Delta_f)$, where SGDL is the distributed Laplace mechanism as described in section 5.3.3.

- 3) $\text{SA.Protect}(pp, ek_u, \tau, \tilde{x}_{u, \tau}) \rightarrow y_{u, \tau}$.

- 4) Send $y_{u, \tau}$ to the server.

Server:

- 1) Collect $y_{u, \tau}$ from all $u \in \mathcal{U}$.
- 5) Aggregate: $\text{SA.Agg}(pp, ak_0, \tau, \{y_{u, \tau}\}_{u \in \mathcal{U}}) \rightarrow x_\tau$.

Figure 5.1: Detailed description of SaFer-FL algorithm.

5.6 Experiments

5.6.1 Experimental Setting

We evaluate SaFer-FL on two tabular datasets

- MovieLens-1M (movie-rating prediction) [94]
- COMPAS (recidivism prediction) [15]

In both datasets gender is being considered as the protected attribute. Every client trains a logistic-regression classifier. Unless stated otherwise we use a learning rate of 0.03, 300 global rounds, $E = 2$ local epochs per round, and full client participation (frac = 1.0, i.e. $|\mathcal{U}^r| = n$ in every round). The number of FL clients is set to $n = 75$ in MovieLens-1M with a local batch size of 75 and $n = 20$ in the COMPAS dataset with a local batch size of 20. All other hyperparameters are kept identical across the two datasets.

5.6.2 Pre-processing of MovieLens-1M and COMPAS

5.6.2.1 MovieLens-1M

We construct the dataset from the raw MovieLens-1M files (`ratings.dat`, `users.dat`, `movies.dat`), where each example is a single (user, movie) rating. The feature vector concatenates user attributes, including gender, categorical age group, and a one-hot encoding of occupation, with movie attributes, including the number of years between the rating and the movie release, and a multi-hot encoding of movie genres. The label $l_i, i \in \{0, 1\}$ is binary: a rating strictly greater than 3 is positive (1), otherwise negative (0). MovieLens is naturally suited to the federated setting, where each user is a client u holding their own local dataset $\mathcal{D}_u$ containing their movie ratings. We select $n = 75$ users from the full dataset. The selected users include approximately 32% females, and the proportion of positive labels is approximately 0.58%, matching the corresponding proportions in the full dataset, thereby preserving the demographic and label distributions of the full dataset.

5.6.2.2 COMPAS

We construct the dataset from the full COMPAS release published by ProPublica (`compas-scores.csv`), where each example corresponds to a single defendant. The feature vector includes gender together with demographics, criminal-history, case-timing, and COMPAS-score features. The binary label l_i is `is_recid` (1 if recidivism is observed during the follow-up window,

0 otherwise); we use the full COMPAS file rather than the two-year subset to retain more samples per client, which better reflects a realistic federated deployment. Unlike MovieLens, COMPAS is not naturally federated because each row represents an individual (defendant) rather than a user with a personal dataset. We therefore divide the data among $n = 20$ synthetic clients using a label-wise Dirichlet split with Dirichlet factor $\zeta = 1.0$. The Dirichlet factor controls the degree of label heterogeneity across clients: a larger value produces more similar label distributions across clients, resulting in a setting closer to IID, while a smaller value produces greater differences in label distributions and a more heterogeneous non-IID setting. We use $\zeta = 1.0$ to create label heterogeneity across the synthetic clients, providing a more realistic federated setting.

In both datasets the protected attribute is gender, encoded as Female = 0 and Male = 1, which induces the four label-group cells $(F, 0), (F, 1), (M, 0), (M, 1)$ used by both the SaFer-FL weights and the fairness metrics defined below. After the dataset-specific construction described next, each client’s dataset $\mathcal{D}_u$ is split 80%/20% into local train and test sets with a fixed seed.

5.6.3 Evaluation Metrics

We measure utility with global test-set accuracy: after each round, the server evaluates the current global model on the union of all clients’ 20% held-out test partitions. As mentioned in Section 5.3 group fairness is reported with two standard metrics computed across the gender groups:

$$\begin{aligned}\Delta\text{EOP} &= |\text{TPR}_F - \text{TPR}_M|, \\ \Delta\text{EODD} &= \frac{1}{2}(|\text{TPR}_F - \text{TPR}_M| + |\text{FPR}_F - \text{FPR}_M|).\end{aligned}$$

Lower values of ΔEOP and ΔEODD indicate fair behavior across groups.

5.6.4 Experimental Results

In order to conduct a comparative experimental study, we have implemented the standard FedAvg [12] protocol (denoted as FL), the prior solution proposed in [88] (PrivFairFL-Pre), and our solution SaFer-FL. Table 5.1 reports the model accuracy and the two fairness metrics on MovieLens-1M and COMPAS; we could not reproduce PrivFairFL-Pre on COMPAS because the authors did not study it; hence, we put dashes in that row. SaFer-FL is evaluated without DP and with distributed SGDL noise at $\epsilon \in \{1, 5, 10\}$. We observe that without DP, SaFer-FL nearly matches the non-private FL baseline in accuracy on both datasets while outperforming PrivFairFL-Pre

Table 5.1: Utility and fairness on MovieLens-1M and COMPAS for the federated baselines and our method (SaFer-FL), with and without differential privacy. Accuracy is in percent; ΔEODD and ΔEOP are fairness gaps (lower is fairer).

Method	MovieLens-1M			COMPAS		
	Acc.	ΔEODD	ΔEOP	Acc.	ΔEODD	ΔEOP
FL	62.39%	0.141	0.091	70.79%	0.076	0.121
PrivFairFL-Pre	58.52%	0.051	0.042	—	—	—
SaFer-FL (without DP)	61.97%	0.014	0.0003	70.38%	0.018	0.025
SaFer-FL (DP, $\epsilon = 1$)	58.84%	0.042	0.001	67.98%	0.043	0.070
SaFer-FL (DP, $\epsilon = 5$)	59.84%	0.038	0.036	68.84%	0.018	0.029
SaFer-FL (DP, $\epsilon = 10$)	61.18%	0.032	0.020	69.38%	0.023	0.035

on MovieLens-1M and substantially improving both fairness gaps. Adding DP noise yields the expected privacy–utility trade-off on accuracy: a smaller ϵ injects more noise and lowers accuracy, while accuracy recovers toward the no-DP level as ϵ grows. The relationship between ϵ and fairness is less regular than the one between ϵ and accuracy: both fairness gaps move non-monotonically with the privacy budget. Several factors may contribute to this behavior, including the variance introduced by the noise at small budgets and the interaction between the noise and the reweighting mechanism. Overall, the no-DP run yields the best fairness on MovieLens-1M and remains the strongest or tied on COMPAS, suggesting that the cleanest fairness gains arise when the reweighting operates on noise-free updates.

5.6.5 Complexity Analysis

When compared with the solution in [95], SaFer-FL incurs the same complexity. For communication and storage, the client requires $O(n+m)$ and the server requires $O(n^2 + nm)$. For computation, the client requires $O(n^2 + m)$, while the server requires $O(n^2 + nm)$. Here, n denotes the number of clients and m denotes the number of model parameters.

5.7 Conclusion

We have presented SaFer-FL whereby the cryptographic building block is already protecting model updates in FL and it also carries the demographic statistics required to make those updates fair. The SA layer privately collects (per-group, label) counts, the server derives label-aware reweighting

coefficients from the resulting global totals, and a symmetric geometric discrete Laplace mechanism gives the entire pipeline a formal ϵ -DP guarantee – all without external MPC servers, trusted curators, or any infrastructure beyond what FL already deploys. On COMPAS and MovieLens-1M, this design narrows both equalized-odds and equal-opportunity gaps relative to standard FedAvg while maintaining competitive accuracy, and by varying the privacy budget we map the controllable trade-off space between privacy, fairness, and utility. In FL, privacy alone can hide unfairness in the model without fixing it. Fairness alone can risk revealing private information about clients. *SaFer-FL* solves both problems together by using SA to compute a fair global model in FL while still protecting privacy. Thus, this approach fulfills **C3: Privacy and Fairness (R1 + R5)**.

Chapter 6

Robust Blockchain-based Federated Learning

This chapter addresses the fourth challenge, **C4**, by jointly considering **privacy protection** and **adversarial robustness** in Federated Learning (FL), while ensuring the integrity of the aggregation process. As discussed in Chapter 1, FL systems may include malicious clients that submit poisoned updates to compromise the global model. In addition, a compromised server may manipulate the aggregation process or its output. To address these challenges, and in contrast to the approaches presented in the previous chapters, we propose a privacy-preserving FL framework that leverages Trusted Execution Environments (TEEs) within the Secret Network¹ blockchain to perform aggregation securely and correctly, while incorporating robust aggregation techniques to mitigate the impact of adversarial client updates.

The work presented in this chapter was accepted and published at the 11th International Conference on Information Systems Security and Privacy (ICISSP 2025).

6.1 INTRODUCTION

As presented in Chapter 1, adversarial robustness is an important requirement for trustworthy FL. SA can protect individual client updates from the aggregation server, it also prevents the server from directly inspecting these updates to determine whether they are legitimate or not. This creates a fundamental tension between update privacy and the detection of potentially malicious client behavior. Adversarial clients can exploit this limitation by submitting manipulated model updates with the objective of influencing the

¹<https://scret.network/>

global model. Such adversarial behavior may arise either from the training data, for example, when a malicious client manipulates its local dataset, or during protocol execution, for example, when a client submits a manipulated model update. Depending on the attack strategy, malicious updates degrade the overall model performance or target specific model behaviors. Consequently, an FL system should not only protect the confidentiality of client updates but also limit the influence of potentially malicious contributions while preserving the privacy guarantees provided by SA.

In addition to adversarial robustness, the integrity of the aggregation process is an important requirement for trustworthy FL. Clients should have assurance that their valid updates are processed according to the prescribed aggregation procedure and that the resulting global model has not been modified. In a conventional single-server architecture, the server is responsible for receiving client updates, executing the aggregation procedure, and producing the new global model. Consequently, a compromised server represents a potential single point of failure and may manipulate the aggregation process or its output. Although existing approaches have considered verifiable aggregation in FL [96, 97], providing such guarantees may require additional verification mechanisms and may incur substantial computational and communication overhead.

These observations motivate the need for an FL architecture that simultaneously provides update privacy, adversarial robustness, and aggregation integrity without relying on computationally expensive cryptographic protocols. To this end, we investigate the use of Trusted Execution Environments (TEEs) within the Secret Network (SN) blockchain to perform privacy-preserving and robust aggregation.

6.1.1 Problem Statement

Building on the requirements for trustworthy FL identified in Chapter 1, this chapter focuses on the following three requirements:

- **Privacy of client updates.** Each client’s local update should remain private from the aggregation server, while the server can recover only the aggregate required to generate the new global model.
- **Adversarial robustness.** The global model should remain robust against adversarial behavior arising either from the training data, such as a malicious client manipulating its local dataset, or from the protocol execution, such as a client submitting manipulated model updates. The aggregation process should therefore identify and limit the contribution of potentially malicious updates.

- **Aggregation integrity.** The aggregation procedure should be executed according to its prescribed specification, and the resulting global model should correctly reflect the intended aggregation of the submitted updates.

6.1.2 Our Contributions

To address these requirements, we leverage blockchain technology to provide integrity for the aggregation process while preserving the privacy of client updates. In particular, we use the Secret Network² (SN), a public blockchain that relies on Trusted Execution Environments (TEEs) to execute smart contracts while protecting the confidentiality of the data processed within them.

In our solution, the blockchain replaces the traditional FL server, with a set of nodes maintaining the ledger and executing the aggregation smart contract. This decentralized architecture avoids reliance on a single aggregation server and provides integrity and immutability through the underlying consensus mechanism. Client updates are encrypted before being submitted to SN and are decrypted only within the TEE, where the aggregation is performed. The smart contract further incorporates three robust aggregation techniques, namely Krum, Trimmed Mean, and Median, to reduce the influence of potentially adversarial client updates while keeping the updates protected within the TEE.

Our contributions are summarized as follows:

- We design a privacy-preserving aggregation framework that uses TEEs to process encrypted client updates without exposing individual updates to the aggregation server.
- We integrate robust aggregation mechanisms into a smart contract deployed within the SN framework. In particular, we implement and evaluate Krum, Trimmed Mean, and Median to mitigate the influence of adversarial client updates.
- We leverage the decentralized blockchain infrastructure and its consensus mechanism to provide integrity for the aggregation procedure and avoid reliance on a single aggregation server.
- We implement the proposed framework and evaluate it in practice using a medical dataset for arrhythmia prediction. We additionally conduct experiments on MNIST to compare the resulting model performance with the closest related scheme SABLE [2].

²<https://scret.network/>

6.2 Related Work

6.2.1 Multi-Party Computation (MPC)

Several works have previously explored privacy-preserving FL systems robust against adversarial attacks. Approaches such as [98, 25, 26, 27] use MPC along with other cryptographic techniques, such as Zero-Knowledge Proofs and Oblivious Transfer. Nevertheless, those solutions assume the involvement of two honest-but-curious servers that do not collude, which can be seen as too restrictive.

Other robust FL MPC-based solutions [25, 99] implement weaker robustness mechanisms, thereby compromising the convergence guarantees of the global models. For instance, the method proposed in [25] is based on FLTrust framework [100], which requires the server to have access to a trusted and clean dataset, which is a strong assumption compared to most of FL systems that leverage solely client gradients. Similarly, while the solution in [99] shows practical effectiveness, it lacks formal convergence guarantees, which are essential for adversarial robust FL systems [101, 102].

Works in [103, 104, 105] have employed MPC in combination with blockchain to provide privacy. However, these approaches primarily focus on protecting the privacy of the participating clients, without addressing the robustness needed to defend against malicious clients attempting to poison global models.

6.2.2 Differential Privacy (DP)

Works such as [106, 107, 108] have explored the use of Differential Privacy (DP) mechanism in adversarial robust FL algorithms to protect client privacy from malicious servers. These methods typically rely on noise injection into the gradients to ensure privacy, though this often leads to an acceptable but not an optimal trade-off between privacy and model accuracy. For example, in [107], the authors, through theoretical analysis and numerical experiments using publicly available datasets, demonstrate that it is impractical to achieve both DP and adversarial robustness simultaneously.

6.2.3 Homomorphic Encryption (HE)

Few prior works [109, 23, 24, 110] have explored the use of HE to ensure privacy in FL systems. For instance, the work in [109] proposes a blockchain-based FL solution that employs Paillier HE cryptosystem to implement the Multi-Krum aggregator. However, this approach does not fully implement

Multi-Krum within the encrypted domain due to operational limitations of the cryptosystem (i.e., additions only). As a result, this partial implementation leads to considerable leakage towards the aggregation nodes, significantly compromising the privacy of the clients. Moreover, the method proposed in [23] grants the server access to the evolving model at each step of the learning process, threatening the privacy of the clients. The solution in [24] presents a blockchain-based FL system to ensure privacy and robustness. The authors apply the Cheon-Kim-Kim-Song (CKKS) scheme, which implements fully HE to encrypt local gradients, thus enhancing privacy protection. Additionally, they utilize the FLTrust framework [100] for robustness, which restrictively relies on a trusted server with access to a clean dataset, as discussed earlier. Lastly, robust mechanisms for SA are presented in [110] such that malicious inputs can be first detected through norm bounding techniques, and then discarded. Nevertheless, extra overheads have a substantial impact on the resulting FL system, making the latter not deployable in practice.

A more recent work [2] utilizes HE to implement privately robust aggregation technique, more specifically, Trim Mean and Median. Since the solution sorts gradient values while being homomorphically encrypted at the server side for Trim Mean and Median, it incurs a high computational cost, particularly at the server end. Our blockchain-based approach shares strong similarities with [2] as it uses Trim Mean to deal with robustness issues. We choose to compare this work with ours to better assess the deployability of the latter in a real FL environment.

6.3 Background

6.3.1 Secret Network

Secret Network³ (SN) is an interoperable blockchain protocol offering privacy guarantees through the execution of smart contract within TEEs. Informally, SN encrypts the data before storing them in smart contracts, preserving the privacy of the information collected within dApps (decentralized applications). A network of SN nodes is responsible of securely processing smart contract computations through consensus mechanisms. The native cryptocurrency SCRT serves as a medium for participating in on-chain governance, facilitating network transactions, and rewarding users for securing the network through staking. A major use case is in DeFi (decentralized finance), where privacy-preserving smart contracts enable confidential transactions while facilitating various activities such as lending, borrowing and

³<https://scrt.network/>

trading. SN's design relies on two main components, namely the Cosmos SDK⁴ and the Tendermint⁵ consensus engine. This combination provides a robust foundation for scalable, private and permissionless smart contracts that seamlessly integrate with the broader interchain ecosystem.

6.3.1.1 Actors

The SN network consists of a set of nodes, called *Delegates*, who are elected by token holders, called *Delegators*, to represent and validate transactions on their behalf. Delegators participate by staking or delegating their tokens to specific delegates of their choice. The number of staked tokens determines the weight of a delegator's vote. Delegates campaign for votes from delegators, showcasing their technical competence, reliability and contributions to the network to gain support. The delegates with the most votes become active block producers, called *Validators*, taking turns to produce blocks and validate transactions. The SN protocol allows a limited number of 100 delegates.

6.3.1.2 Consensus

A consensus mechanism defines how nodes in the network agree on the state of a blockchain. SN utilizes the Delegated Proof of Stake (DPoS) mechanism for consensus [111]. Here, users participate in the validation and maintenance of the network by voting for a limited number of delegates. These few selected delegates, also known as the validators, are entrusted with the responsibility of validating blocks and ensuring the smooth operation of the network. This approach enhances the speed of the network compared to other Proof of Stake (PoS) systems, resulting in lower fees and increased throughput.

6.3.1.3 Transactions and Blocks

Transactions are verified and grouped into blocks by the active delegates. The latter then propose those blocks to the network for validation. Other nodes in the network, including standby delegates, verify them. Once consensus is reached, through the majority of the network (here, at least 2/3 of the validators) agreeing on the validity of the proposed blocks, the latter

⁴<https://docs.cosmos.network/>

⁵<https://tendermint.com/>

are added to the blockchain. Delegates who successfully produce blocks and participate in the consensus process receive rewards. In particular, rewards are distributed among both the validators and the delegators who staked their tokens with the former.

6.3.1.4 Cryptocurrency

SCRT is the native cryptocurrency of SN, enabling the following functions:

- *Governance*: SCRT holders participate in the governance of SN by voting on proposals. Each staked SCRT equals one vote, and validators vote with the combined total of all their delegators' staked SCRT. Governance proposals require a simple majority of staked SCRT to pass, and the voting period lasts for 7 days.
- *Gas Fees*: Gas fees are paid in SCRT for processing transactions. These fees compensate validators and stakers on the network for their efforts in securing and maintaining the network's operation.

6.3.2 Trusted Execution Environments (TEEs)

A TEE creates a secure area within a processor where data is isolated from other system components. This secure area, called *enclave*, acts as a black box for computations, ensuring that the internal state remains hidden. SN has chosen the Intel's Software Guard Extensions (SGX) implementation for its TEE framework [112]. In a smart contract, private metadata is encrypted before being sent to validators for processing. Validators only decrypt this data within their TEE, ensuring that it remains inaccessible to them. Computations are then performed on the decrypted data, and the output is encrypted before being sent off the TEE and stored on the blockchain. Enclaves in SN generate and store their attestation keys, which are used only once during registration. The enclaves create subsequent keys for communication with the network, in particular to decrypt private inputs of smart contracts and encrypt resulting outputs.

6.3.3 Robust Techniques

To mitigate the impact of adversarial or poisoned client updates, we employ and evaluate three robust aggregation techniques, namely *Krum* [113], *Trimmed Mean* [114], and *Median* [114]. Let $\mathcal{U} = \{u_1, \dots, u_n\}$ denote the set of all clients, and let $\mathcal{U}^\tau \subseteq \mathcal{U}$ denote the set of clients participating in round

τ . Each participating client $u \in \mathcal{U}^\tau$ provides a model update $w_{u,\tau}$. The parameter f denotes the assumed number of adversarial clients, and $m = |\mathcal{U}^\tau|$ denotes the number of participating clients in round τ .

6.3.3.1 Krum

Krum selects a single client update that is closest to the majority of the participating client updates. For each client $u \in \mathcal{U}^\tau$, Krum computes the squared Euclidean distance between its update $w_{u,\tau}$ and the update of every other participating client $v \in \mathcal{U}^\tau \setminus \{u\}$:

$$d(u, v) = \|w_{u,\tau} - w_{v,\tau}\|^2. \quad (6.1)$$

Let $\mathcal{N}_u$ denote the set of the $m - f - 2$ closest updates to client u . The Krum score for client u is then defined as

$$\text{Score}(u) = \sum_{v \in \mathcal{N}_u} d(u, v). \quad (6.2)$$

The aggregated update is the update of the client with the smallest score:

$$w_{\text{Krum}} = w_{u^*,\tau}, \quad u^* = \arg \min_{u \in \mathcal{U}^\tau} \text{Score}(u). \quad (6.3)$$

Thus, Krum reduces the influence of updates that deviate substantially from those of the majority of participating clients.

6.3.3.2 Trimmed Mean

Trimmed Mean performs coordinate-wise aggregation after sorting the corresponding coordinate values of the client updates. For each coordinate $j \in \{1, \dots, d\}$, let

$$x_j = \{w_{u,\tau}[j] : u \in \mathcal{U}^\tau\} \quad (6.4)$$

denote the multiset of j -th coordinate values contributed by the participating clients. Sort these values in ascending order:

$$x_j^{(1)} \leq x_j^{(2)} \leq \dots \leq x_j^{(m)}. \quad (6.5)$$

The largest f and smallest f values are discarded, and the remaining $m - 2f$ values are averaged:

$$w_{\text{TM}}[j] = \frac{1}{m - 2f} \sum_{k=f+1}^{m-f} x_j^{(k)}. \quad (6.6)$$

The final aggregated update is

$$w_{\text{TM}} = (w_{\text{TM}}[1], \dots, w_{\text{TM}}[d]). \quad (6.7)$$

By discarding extreme coordinate values, Trimmed Mean reduces the influence of potentially malicious client updates.

6.3.3.3 Median

Median performs coordinate-wise aggregation by selecting the median value among the corresponding coordinates of the client updates. For each coordinate $j \in \{1, \dots, d\}$, let

$$x_j = \{w_{u,\tau}[j] : u \in \mathcal{U}^\tau\} \quad (6.8)$$

and sort the values in ascending order:

$$x_j^{(1)} \leq x_j^{(2)} \leq \dots \leq x_j^{(m)}. \quad (6.9)$$

The coordinate-wise median is defined as

$$w_{\text{Median}}[j] = \begin{cases} x_j^{(\frac{m+1}{2})}, & \text{if } m \text{ is odd,} \\ \frac{1}{2} \left(x_j^{(\frac{m}{2})} + x_j^{(\frac{m}{2}+1)} \right), & \text{if } m \text{ is even.} \end{cases} \quad (6.10)$$

The final aggregated update is

$$w_{\text{Median}} = (w_{\text{Median}}[1], \dots, w_{\text{Median}}[d]). \quad (6.11)$$

The median is less sensitive to extreme values than the arithmetic mean and therefore limits the influence of potentially adversarial updates. The robustness of the method depends on the number of adversarial clients f relative to the number of participating clients m .

6.4 Robust SN-based FL Solution

6.4.1 Overview

FL clients train local model with their own dataset. Those local models are then sent encrypted to SN. The latter is in charge of aggregating those local models to obtain a new global model, in such a way that it learns nothing about the individual models but only the resulting aggregate. Moreover, SN

is responsible for verifying the integrity of each submitted individual model before aggregation. In particular, the blockchain verifies that every model proposed by a client is correct and will not poison the future global model.

Each client $u \in \mathcal{U}$ trains the model locally on its own dataset. For example, client u_1 submits its resulting model update to SN after encrypting it with pairwise key k_1 shared with SN (1).

As soon as an encrypted input arrives, SN launches the robust aggregation process (2). To do so, inside the TEE, inputs are decrypted and possibly ordered, and the robust techniques (e.g. Krum, Trim Mean, Median) are calculated on those inputs. For instance, the input from u_1 is decrypted using k_1 inside the TEE. SN first includes it into an initially empty list that represents the buffer. SN collects other inputs until the list is full. Steps (1) and (2) launch the transaction $Tr1$, which is financially covered by u_1 . Once the list is full, SN aggregates all inputs in this list such that outliers are discarded (based on the selected robust technique), resulting in a new global model (3). This model can be queried by each client. Then, the list becomes empty. This step triggers a transaction since the global model has been updated and the list has been emptied. This transaction is paid by the owner of the smart contract.

We aim for deploying our solution in buffered FL settings [115]. Clients execute their training as their pace as long as the used global model is fresh enough (it does not need to be the latest model accessible by query). When submitting their inputs to SN, clients must include the hash of the transaction from which the global model was computed. Hence, SN can check the freshness of the inputs before proceeding to the robust aggregation step, using the transaction hash (giving the time and the block). Moreover, the list plays the role of the buffer in our solution. SN is responsible of collecting enough correct inputs from clients before moving towards aggregation. Figure 6.1 depicts the flow of our blockchain-based scenario.

6.4.2 SN-based System

6.4.2.1 System Phases

Setup: During the setup phase, the clients register to the SN ecosystem. They become basic users of the network, with some stake but without the intention of participating in the maintenance of the blockchain (i.e. through the roles of delegates and delegators). Clients aim to launch transactions to submit their local inputs to SN for robust aggregation. They also receive the appropriate key material to communicate with SN, allowing them to encrypt their inputs and decrypt the outputs respectively.

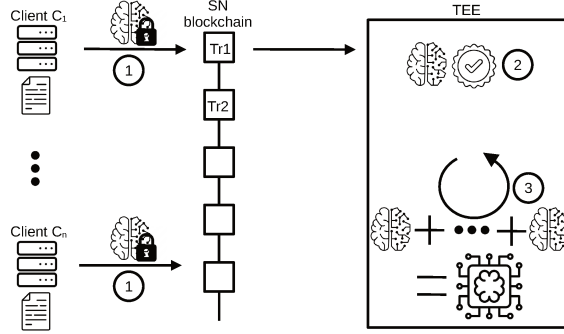


Figure 6.1: Overview of our FL scenario.

Training: Clients locally train the model on their own dataset. Once training is over, they submit their updates to SN by encrypting them using a symmetric key shared between each client and the network. This kind of submission triggers a transaction, meaning that clients must pay fees to SN where some are used for gas and other for paying delegates.

Robust Aggregation: Clients' inputs are stored in a buffer of a FL-oriented smart contract until the buffer is full. SN, through this smart contract, first aggregates the stored inputs and obtains the aggregate. The latter serves as the new global model and can be queried by clients for another training round.

To enhance robust aggregation, the smart contract embeds robust techniques, namely Krum, Trim Mean and Median. More specifically, the smart contract receives local inputs from clients through transaction in their encrypted form. Within the TEE, these inputs are decrypted, ordered and aggregated following one of the three techniques.

6.4.2.2 Smart Contract

Workflow: The smart contract proceeds as follows to achieve robust aggregation. The smart contract first receives local updates from clients and adds them to its buffer. Note that only one input (i.e., vector) is accepted per client per buffer round to prevent DoS attacks. Once the buffer is full, it checks their validity by ordering them according to the chosen robust technique. For instance, a bound on the number of corrupted clients is defined that excludes the most left and right inputs in the ordered list. Once input exclusion has happened, the smart contract aggregates all the remaining up-

dates, resulting into a new global model. The latter can be queried by clients.

Client Management: The smart contract is owned by a party belonging to SN. The owner can add and revoke clients. Clients are defined as authorised when the smart contract is instantiated. This means that only authorised clients can submit local inputs through transactions. On the other side, any party in SN, not only the authorised clients, can query the smart contract. For instance, anyone can query the list of authorised clients and the latest global model. The size of the buffer is defined as being at least half of the number of authorised clients.

Smart Contract Management: The owner is in charge of activating the smart contract. Additionally, she can deactivate it at any time. The smart contract has been developed such that the owner can make it not queriable (except to check whether the smart contract is active) and not modifiable.

Input Management: Clients' inputs are vectors whose size can be consequent, of the order of thousands. To fit within the SN framework, vectors can be sent in smaller segments. Clients are asked to append the round information of the current buffer. They find such information when accessing the new global model to train. Clients are restricted to submit one input per buffer round, even if this input is rejected (not validated). Once the buffer is full (i.e., its maximum size is reached), the aggregation of all inputs within the buffer is launched.

6.5 Experimental Evaluation

6.5.1 Experimental Setting

Datasets and Distribution among clients: To implement and evaluate our solution, we consider two scenarios with two datasets: (i) a local medical dataset that consists of 2126 patients' (fully anonymized) medical information such as age, blood pressure, pulse, waist, uric acid, etc., and helps predict heart arrhythmia; (ii) the MNIST dataset for image classification [116]. We consider a cross-silo distributed system composed of SN (which can be seen as a single server) and n clients. More concretely, these two medical and MNIST datasets are distributed among 10 and 15 FL clients, respectively. In a realistic cross-silo environment, clients often have different data distributions; hence, in our setting, similar to the one in SABLE [2], the dataset is partitioned among all clients following a Dirichlet distribution parameterized

with ζ [117]. To capture the desired level of data heterogeneity and examine its impact, we set ζ in $\{0.5, 1, 5\}$ for the medical dataset (the higher ζ the more homogeneous the distribution among clients). We choose $\zeta = 1$ as in SABLE for the MNIST dataset, reflecting a heterogeneous distribution.

Adversarial nodes and Attacks: Training a FL model on the medical and MNIST datasets while keeping it privacy-protected becomes even more complex when defending against a subset of Adversarial nodes who try to deter the model’s performance. We have implemented the four classes of poisoning attacks that are enumerated in SABLE:

- In a *Fall Of Empires* (FOE) attack [118], adversarial nodes send $(1 - \tau)\mathbf{v}_t$, where $\mathbf{v}_t$ is the average of vectors sent by honest nodes at round t and τ is the attack factor.
- In an *A Little Is Enough* (ALIE) attack [119], adversarial nodes send $\mathbf{v}_t + \tau\sigma_t$, where σ_t is the coordinate-wise standard deviation of honest vectors.
- In a *Label Flipping* (LF) attack [120], adversarial nodes flip each label l to $9 - l$, where labels range from $\{0, \dots, 9\}$, and compute gradients on these modified labels.
- In a *mimic* attack [121], adversarial nodes imitate the honest node that incurs the largest variance with respect to the gradients and transmit the same values to SN. To evaluate the impact of such attacks, we introduce f adversarial nodes in our two scenarios, with f varying from 0 to 3 and 0 to 5, respectively.

Models and Hyperparameters: For the medical scenario, each client trains a Feed Forward Neural Network (NN) composed of three fully connected layers (FCLayers) with two activation functions, with model batch size $b = 32$ and learning rate $\gamma = 0.01$.

Because SN only works with integers, we quantize the model parameters and set the precision value $\delta = 10^4$. On the other hand, for the MNIST scenario, we adopt the same setting of the one used in SABLE in order to be able to compare easily. For both scenarios, the model is trained in 100 rounds (we observe that this number is sufficient for the model to converge).

6.5.2 Experimental Results

We first evaluate the performance of our SN-based FL solution within the medical scenario. In particular, we assess the learning performance of the

model, in comparison to its non-attacked counterpart, while gradually increasing the number f of malicious clients. We then evaluate our solution using the MNIST dataset to accurately compare our results with those obtained in SABLE. In both scenarios, we also evaluate the communication and computation costs.

Medical scenario: Firstly, in our experiments, we notice that, when we move towards more homogeneous data distributions, all robust aggregation techniques converge faster, independently of the underlying attack. This is depicted in Figure 6.2 for FOE attacks (we observe the same behavior for the other attacks but because of lack of space, we do not show them here.). Therefore, we decide to set $\alpha = 1$ for all remaining experiments, as depicted in Figure 6.3.

Figure 6.3 then resents the accuracy results of the trained model using the three robust aggregation techniques (results with Krum, Trim Mean, and Median are depicted in the first, second and last column, respectively). Each row corresponds to the study of one specific attack and each graph shows results with four case-studies: one honest and three malicious cases, i.e., $f \in \{0, 1, 2, 3\}$.

Regarding the quality of the aggregation technique, we observe that Krum shows the greatest resilience in more heterogeneous environments in the case of FOE attacks. We believe the reason for this performance is that changing the sign does not affect the Euclidean distance between the values. On the other hand, when ALIE or LF attacks are implemented, all aggregation techniques seem to show similar accuracy results, and compared to FOE attacks, these two attacks do not create harm to the model. Under the mimic attack, Figure 6.3 (d) shows that the Trim Mean aggregation technique is less robust than Krum and Median techniques. Nevertheless, we realize that the training still converges to an acceptable accuracy level ($\approx 90\%$).

Technique	t_{tran} (s)	t_{agg} (ms)	BW (KB)
Krum	4.2	120	13.67
Trim Mean	4.0	110	13.67
Median	4.1	100	13.67

Table 6.1: Communication and computational costs of Krum, Trim Mean and Median for the medical scenario. “BW” denotes bandwidth.

The communication and computational costs for the medical scenario are presented in Table 6.1, where $n = 10$, $f = 3$ and $\delta = 10^4$. Here, t_{tran} is defined as the time required for each client to submit their local update to

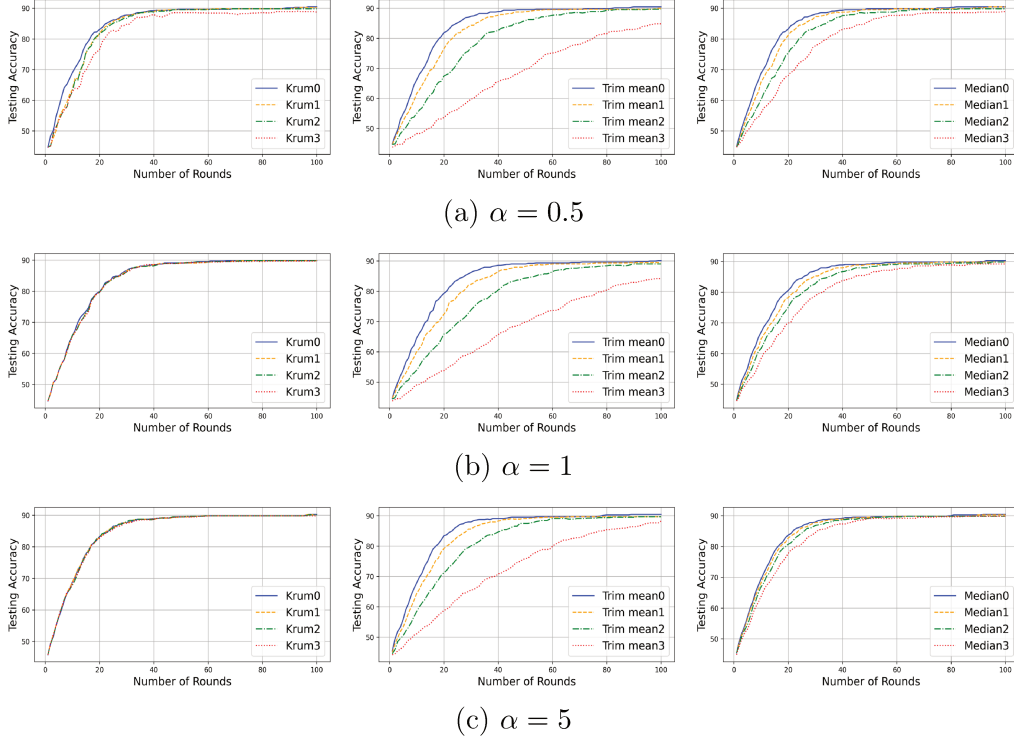


Figure 6.2: Accuracy results for KRUM, Trim Mean, and Median under FOE attacks over the medical dataset.

SN, specifically the time needed to upload their parameters to the blockchain via a transaction. On the other hand, t_{agg} denotes the time taken by SN to execute the smart contract by applying the selected robust technique to the clients' updates, aggregating them, and sending the updated values to all FL clients.

As seen in Table 6.1, the Trim Mean technique achieves the best trade-off between communication and computational costs, with a slightly lower t_{tran} compared to the other methods. The Median technique requires marginally less aggregation time t_{agg} . Hence, the variations in computational are mainly due to the processing complexities unique to each technique. Moreover, all three techniques exhibit the same bandwidth usage (i.e., 13.67 KB) since each client utilizes the same CNN model for training, resulting in identical gradient sizes. Each gradient value is represented by an integer quantized with a factor of 10^4 and can be stored in 16 bits.

MNIST scenario: Figure 6.4 illustrates the learning performance of our solution on the MNIST scenario, using the same model and hyperparam-

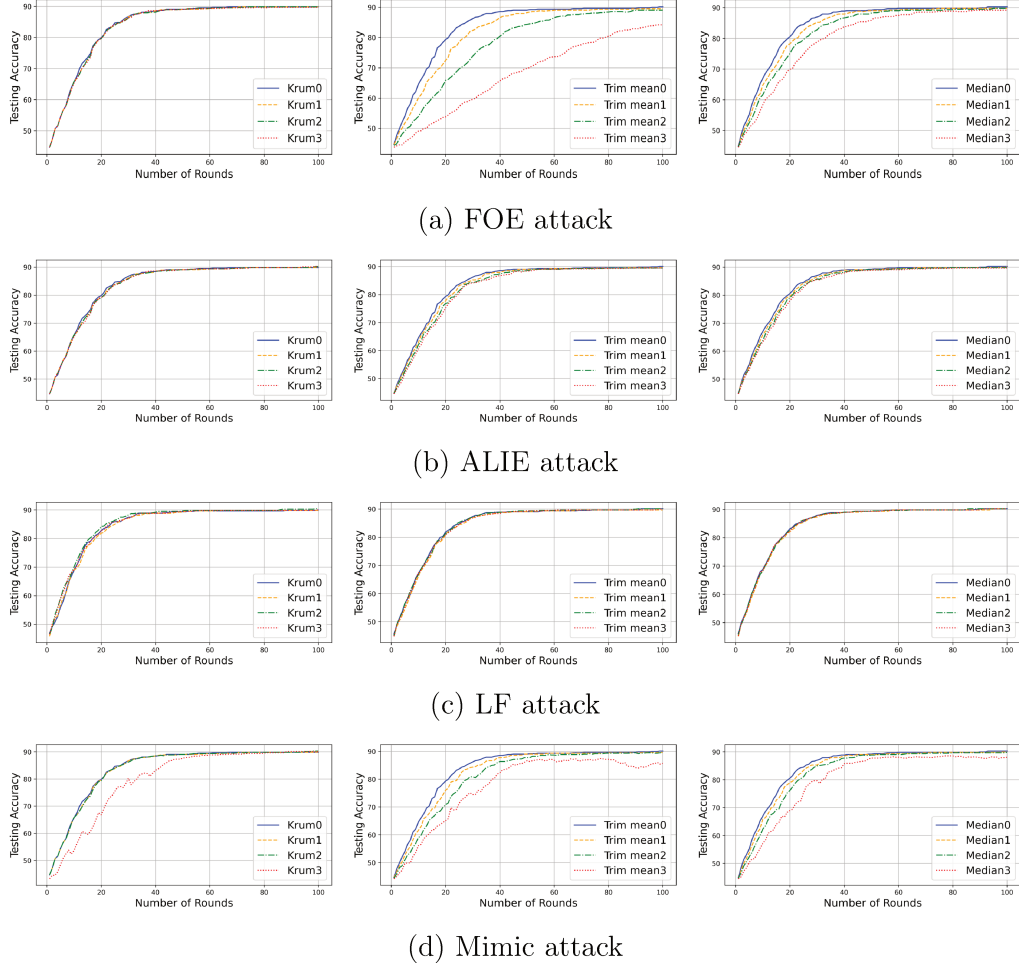


Figure 6.3: Accuracy results for KRUM, Trim Mean, and Median under the four attacks over the medical dataset.

ter settings outlined in SABLE. In particular, $n = 15$, $f \in \{0, 1, 3, 5\}$ and $\alpha = 1$. We launch the four aforementioned attacks, as in SABLE, to properly compare our solution with the latter. Table 6.2 offers a detailed comparison between our work and SABLE in terms of communication and computational costs, based on the MNIST dataset, with $n = 15$ and $f = 5$. This configuration simulates a realistic scenario in which up to a third of the nodes may be compromised SABLE. Within this context, t_{tran} denotes the time each client needs to submit its local update to the SN, while t_{agg} indicates the time taken by the SN to execute the smaer contract.

SABLE aims to support robust aggregation techniques over homomorphically encrypted data. It has only been implemented with Trim Mean in

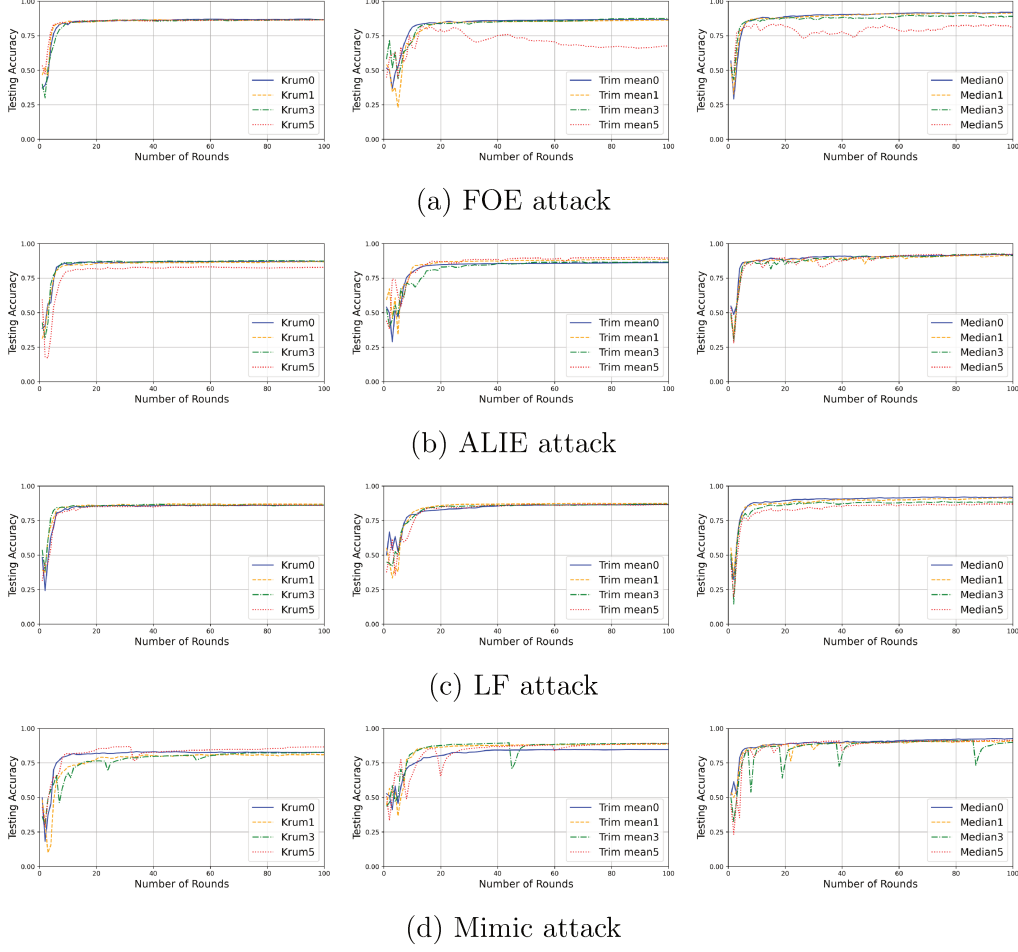


Figure 6.4: Accuracy results for KRUM, Trim Mean, and Median under the four attacks over the MNIST dataset.

practice. Specifically, the Median method has been suggested as an extension of Trim Mean, with similar results, while Krum has not been deployed due to its potential incompatibility with the HE scheme used in SABLE. Moreover, in SABLE, the entire homomorphic aggregation process is not completed at the server side as expected. Instead, the final averaging step (i.e., division by the total number of local inputs), is executed by each client, introducing additional computational overhead on the client side. Additionally, the bandwidth usage is estimated to 5 KB in SABLE but not explicitly confirmed neither theoretically nor experimentally.

In contrast, our work offers a fully integrated and practical implementation of three robust aggregation techniques, namely Trim Mean, Median,

System	Technique	t_{tran} (s)	t_{agg} (s)	BW (KB)
SABLE	Trim Mean	not available	1040.4	5*
Our Work	Trim Mean	4.1	2.96	19.42
	Median	4.1	2.61	19.42
	Krum	4.2	3.48	19.42

Table 6.2: Performance comparison on the MNIST dataset with 79,510 model parameters. (* Such a value is suggested in [2] but is not explained.)

and Krum. Moreover, the full aggregation process (including dividing the aggregate by the number of updates) occurs within the SN, and not at the client side. Our solution also improves the overall system efficiency, as evidenced by the reduced aggregation times t_{agg} by a factor of approximately 300 compared to SABLE. The bandwidth usage of our solution requires 19.42 KB (with 2-bit representation of parameters), while authors of SABLE claim to consume approximately 5 KB (we had difficulties to compute this number although adopting the same setting and quantization strategy). The bandwidth cost, in our case, still remains very low. On the other hand, we observe substantial gains in computational speed. Finally, we have evaluated the performance of the three aggregation techniques (i.e., Trim Mean along with both Median and Krum), which does not vary much in terms of computational and communication costs.

Regarding Figure 6.4, we show that we obtain a similar behavior compared with SABLE, which only implements Trim Mean. Moreover, as also observed in the medical scenario, we realize that Krum is the most suitable technique to overcome FOE attacks, whereas all three methods perform similarly under ALIE and LF attacks. Finally, the mimic attack seems to be the most difficult attack to prevent, and for this one, the worst aggregation method seems to be Trim Mean.

To summarize, based on this experimental study, we can claim that Krum seems the most suitable technique to overcome the four types of attacks with a negligible overhead in terms of computation and communication.

6.5.3 Conclusion

To address **C4**, we developed a robust and privacy-preserving FL framework that combines Trusted Execution Environments (TEEs) with the Secret Network blockchain. The TEE protects client updates during aggregation, while the blockchain provides verifiability of the aggregation process. Robust aggregation mechanisms are further employed to mitigate the influence of ma-

licious client updates. The evaluation considers both the effectiveness of the robust aggregation mechanisms against adversarial attacks and the overhead introduced by the blockchain and TEE components.

Chapter 7

Conclusion and Future Work

In this thesis, we studied the design of SA schemes to address several trustworthiness requirements in FL. We focused on privacy protection, client dropout robustness, data heterogeneity, unlearning, fairness, and adversarial robustness. We observed that these requirements introduce different challenges when combined with privacy-preserving learning. In particular, data-heterogeneity-based aggregation, fairness-aware aggregation, and FU can be incorporated without violating privacy by protecting the additional information required for these operations, such as label counts and client model updates. Ensuring adversarial robustness is more challenging because identifying malicious contributions may require access to information that SA is designed to hide.

Accordingly, for privacy-preserving aggregation in non-IID FL, we proposed SAAFL, which addresses the problem of using client label distributions to determine data-dependent aggregation weights without revealing the underlying information. SAAFL also supports client dropouts during the aggregation process. Our evaluation shows that label-aware aggregation can be performed while protecting the label information used to determine the aggregation weights and maintaining the required functionality in the presence of dropouts. Since SAAFL already processes label distributions, an interesting extension is to investigate whether the same protected information can also be used to incorporate fairness into the aggregation process. This would allow data heterogeneity and fairness objectives to be considered simultaneously without introducing an additional disclosure of client-level label statistics.

For unlearning, we proposed PPFU, which extends FedRecovery [1] with SA and dropout robustness. PPFU prevents the server from directly accessing individual client updates while allowing the contribution of a client to be removed from an already trained model. Our results shows that SA can be

extended beyond its conventional use for protecting the aggregation result to support a mutable operation such as FU. On the other hand, the current solution considers the unlearning of a single client. An important extension is therefore to investigate the unlearning of multiple clients, either simultaneously or through successive unlearning requests, and to study how the number of clients to be unlearned affects the model utility, computational cost, and communication overhead.

For privacy-preserving and fairness-aware aggregation, we proposed SaFer, which enables sensitive client statistics to be used for determining fairness-aware aggregation weights without revealing these statistics to the aggregation server. Our evaluation shows that fairness can be incorporated into the aggregation process while preserving the privacy of the information used to achieve it. A possible extension is to investigate whether the protected fairness-related statistics can also be used to identify an outlying client contribution with respect to a selected fairness objective and subsequently remove or unlearn its contribution. Such a solution would combine fairness-aware aggregation with FU while maintaining the privacy of the sensitive information involved in both operations.

Finally, for privacy-preserving adversarially robust aggregation, we proposed a blockchain-based architecture using the Secret Network and a Trusted Execution Environment (TEE). The blockchain coordinates the aggregation process, while smart contracts executed in the TEE enable robust aggregation methods, namely Krum, Trimmed Mean, and Median, to operate on protected client inputs. The blockchain consensus also provides integrity for the aggregation process. We evaluated the architecture against several poisoning attacks and showed how the different robust aggregation methods behave under malicious client behavior. This approach therefore allows client inputs to be processed for robust aggregation while keeping them private. However, this additional protection comes at a cost due to blockchain coordination, consensus, smart-contract execution, and TEE-based computation. An important direction for future work is consequently to evaluate and reduce this overhead, particularly as the number of clients and the size of the FL model increase. Another direction is to extend this architecture to support fairness-aware aggregation and FU within the blockchain-based TEE environment. This would investigate whether fairness-related information and unlearning operations can be securely processed within the TEE while preserving the privacy guarantees of the proposed architecture.

Overall, the results of this thesis show that the requirements of trustworthy FL should not necessarily be addressed independently. Information and mechanisms introduced to satisfy one requirement can provide a basis for addressing another. The label distributions protected by SAAFL may

also support fairness-aware aggregation, while the protected fairness statistics used by SaFer may support decisions about subsequent unlearning the outlier. Similarly, the extension of SA in PPFU shows that a mechanism designed to protect aggregation can also support operations that modify the contributions represented in a trained model. At the same time, the results highlight that combining these requirements involves trade-offs. In particular, adversarial robustness creates a stronger tension with privacy because robust aggregation needs to inspect client contributions whereas privacy protection seeks to hide them. The blockchain and TEE architecture provide one way to address this tension by performing robust aggregation within TEE through smart contract but they introduce additional assumptions and costs.

While future work for individual contributions have been discussed in previous paragraphs, we can also propose general directions. The proposed solutions demonstrate that cryptographic mechanisms can be adapted to support privacy with dropout robustness or with data heterogeneity or with unlearning or with fairness and adversarial robustness, combining several of these requirements within a single FL system remains challenging. Such combinations may introduce conflicting objectives and additional computational and communication costs. For example, fairness-aware aggregation and unlearning require access to information about client contributions, whereas adversarial robustness may require identifying or limiting malicious contributions without exposing the information protected by SA. Designing mechanisms that can satisfy these requirements simultaneously while preserving privacy and model utility remains an open problem.

Another direction is to investigate hybrid architectures that combine cryptographic mechanisms with trusted hardware and distributed infrastructures. The use of TEEs or blockchain together with cryptographic protocols could provide complementary properties, such as confidential computation, verifiability, and integrity. However, such combinations also introduce new trust assumptions, system dependencies, and performance costs that need to be carefully analyzed. In particular, it would be useful to determine which operations should be performed using cryptographic protocols and which could be delegated to a TEE or blockchain, and how these choices affect the overall efficiency of FL.

The requirements considered in this thesis can also be extended to other dimensions of trustworthy FL, including explainability, verifiability, and transparency. These requirements raise additional questions about how the training and aggregation processes can be made auditable and understandable without revealing private client information. For example, providing evidence that an aggregation was correctly performed, that an unlearning request was

effectively applied, or that fairness constraints were respected should not require exposing the underlying client data or individual model updates.

Finally, an important research question is how to prioritize these requirements when they cannot all be satisfied simultaneously. A possible direction is to establish a layered approach in which adversarial robustness and integrity form an initial foundation, since malicious contributions can otherwise compromise subsequent operations. On top of this foundation, privacy-preserving mechanisms could protect client information, while fairness, unlearning, and other requirements could then be integrated without weakening the underlying guarantees. This perspective could lead to a systematic design methodology for trustworthy FL in which the interaction between requirements is considered from the beginning rather than addressed independently.

Bibliography

- [1] Lefeng Zhang, Tianqing Zhu, Haibin Zhang, Ping Xiong, and Wanlei Zhou. FedRecovery: Differentially Private Machine Unlearning for Federated Learning Frameworks. *IEEE Trans. on Information Forensics and Security*, 18, 2023.
- [2] Antoine Choffrut, Rachid Guerraoui, Rafael Pinot, Renaud Sirdey, John Stephan, and Martin Zuber. Practical homomorphic aggregation for Byzantine ML. *arXiv preprint arXiv:2309.05395*, 2023.
- [3] Massimo La Rosa, Antonino Fiannaca, Laura La Paglia, and Alfonso Urso. A graph neural network approach for the analysis of sirna-target biological networks. *International Journal of Molecular Sciences*, 23(22), 2022.
- [4] Andre Esteva, Alexandre Robicquet, Bharath Ramsundar, Volodymyr Kuleshov, Mark DePristo, Katherine Chou, Claire Cui, Greg Corrado, Sebastian Thrun, and Jeff Dean. A guide to deep learning in healthcare. *Nature medicine*, 25(1):24–29, 2019.
- [5] Riccardo Miotto, Fei Wang, Shuang Wang, Xiaoqian Jiang, and Joel T Dudley. Deep learning for healthcare: review, opportunities and challenges. *Briefings in Bioinformatics*, 19(6):1236–1246, 05 2017.
- [6] Thomas Fischer and Christopher Krauss. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2):654–669, 2018.
- [7] Shihao Gu, Bryan Kelly, and Dacheng Xiu. Empirical asset pricing via machine learning. *arXiv preprint arXiv:1809.04788*, 2018.
- [8] Tianqi Chen, Markus Pelger, and Jason Zhu. Machine learning in asset pricing. *arXiv preprint arXiv:1904.00745*, 2019.

- [9] Hang Qi, Jieping Luo, Qiyue Li, and Jingjin Wu. A comparative trade-off analysis on accuracy and efficiency for federated learning in demand forecasting. *Applied Soft Computing*, page 113561, 2025.
- [10] Ian McGraw, Rohit Prabhavalkar, Raziel Alvarez, Montse Gonzalez Arenas, Kanishka Rao, David Rybach, Ouais Alsharif, Hasim Sak, Alexander Gruenstein, Françoise Beaufays, and Carolina Parada. Personalized speech recognition on mobile devices. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5955–5959, 2016.
- [11] Tianming Zhao, Yucheng Xie, Yan Wang, Jerry Cheng, Xiaonan Guo, Bin Hu, and Yingying Chen. A survey of deep learning on mobile devices: Applications, optimizations, challenges, and research opportunities. *Proceedings of the IEEE*, 110(3):334–354, 2022.
- [12] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS 2017)*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282. PMLR, 2017.
- [13] Reza Shokri, Marco Stronati, and Vitaly Shmatikov. Membership inference attacks against machine learning models. *CoRR*, abs/1610.05820, 2016.
- [14] Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (general data protection regulation). OJ L 119, 4.5.2016, p. 1–88, 2016. Accessed: date.
- [15] Matias Barenstein. Propublica’s compas data revisited. *arXiv preprint arXiv:1906.04711*, 2019.
- [16] Asadullah Tariq, Mohamed Adel Serhani, Farag Sallabi, Tariq Qayyum, Ezedin S. Barka, and Khaled A. Shuaib. Trustworthy federated learning: A comprehensive review, architecture, key challenges, and future research prospects. *IEEE Open Journal of the Communications Society*, 5:4920–4998, 2024.
- [17] Titus Tunduny and Bernard Shibwabo. Explainable ai approaches in federated learning: Systematic review. *JMIR AI*, 5, 2026.

- [18] Aditya Pribadi Kalapaaking, Ibrahim Khalil, Xun Yi, Kwok-Yan Lam, Guang-Bin Huang, and Ning Wang. Auditable and verifiable federated learning based on blockchain-enabled decentralization. *IEEE Transactions on Neural Networks and Learning Systems*, 36(1):102–115, 2025.
- [19] Melek Önen and Refik Molva. Secure data aggregation with multiple encryption. In *European Conference on Wireless Sensor Networks*, pages 117–132. Springer, 2007.
- [20] Claude Castelluccia, Einar Mykletun, and Gene Tsudik. Efficient aggregation of encrypted data in wireless sensor networks. In *The second annual international conference on mobile and ubiquitous systems: networking and services*, pages 109–117. IEEE, 2005.
- [21] Keith Bonawitz, Fariborz Salehi, Jakub Konečný, Brendan McMahan, and Marco Gruteser. Federated learning with autotuned communication-efficient secure aggregation. In *2019 53rd Asilomar Conference on Signals, Systems, and Computers*, pages 1222–1226. IEEE, 2019.
- [22] Mohamad Mansouri, Melek Önen, and Wafa Ben Jaballah. Learning from Failures: Secure and Fault-Tolerant Aggregation for Federated Learning. In *38th Annual Computer Security Applications Conference (ACSAC)*, 2022.
- [23] Yogachandran Rahulamathavan, Charuka Herath, Xiaolan Liu, Sangarapillai Lambotharan, and Carsten Maple. FheFL: fully homomorphic encryption friendly privacy-preserving federated learning with byzantine users. *arXiv preprint arXiv:2306.05112*, 2023.
- [24] Yinbin Miao, Ziteng Liu, Hongwei Li, Kim-Kwang Raymond Choo, and Robert H Deng. Privacy-preserving Byzantine-robust federated learning via blockchain systems. *IEEE Trans. on Information Forensics and Security*, 17:2848–2861, 2022.
- [25] Meng Hao, Hongwei Li, Guowen Xu, Hanxiao Chen, and Tianwei Zhang. Efficient, private and robust Federated Learning. In *37th Annual Computer Security Applications Conference*, pages 45–60, 2021.
- [26] Thien Duc Nguyen, Phillip Rieger, Huili Chen, Hossein Yalame, Helen Möllering, Hossein Fereidooni, Samuel Marchal, Markus Miettinen, Azalia Mirhoseini, Shaza Zeitouni, Farinaz Koushanfar, Ahmad-Reza Sadeghi, and Thomas Schneider. FLGUARD: secure and private federated learning. *arXiv preprint arXiv:2101.02281*, 2021.

- [27] Zhuangzhuang Zhang, Libing Wu, Chuanguo Ma, Jianxin Li, Jing Wang, Qian Wang, and Shui Yu. LSFL: a lightweight and secure federated learning scheme for edge computing. *IEEE Trans. on Information Forensics and Security*, 18:365–379, 2022.
- [28] James Henry Bell, Kallista A. Bonawitz, Adrià Gascón, Tancrède Lepoint, and Mariana Raykova. Secure single-server aggregation with (Poly)Logarithmic overhead. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS 2020)*, pages 1253–1269. ACM, 2020.
- [29] Harish Karthikeyan and Antigoni Polychroniadou. OPA: One-shot private aggregation with single client interaction and its applications to federated learning. In *Advances in Cryptology – CRYPTO*, 2025.
- [30] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography, Third Theory of Cryptography Conference (TCC 2006)*, volume 3876 of *Lecture Notes in Computer Science*, pages 265–284. Springer, 2006.
- [31] Mohamed Sabt, Mohammed Achemlal, and Abdelmadjid Bouabdallah. Trusted execution environment: What it is, and what it is not. In *2015 IEEE Trustcom/BigDataSE/ISPA*, pages 57–64, 2015.
- [32] Mihir Bellare and Chanathip Namprempre. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 531–545. Springer, 2000.
- [33] Shamir Adi. How to share a secret. *Commun. ACM*, 22:612–613, 1979.
- [34] Erik Meijering. A chronology of interpolation: from ancient astronomy to modern signal and image processing. *Proceedings of the IEEE*, 90(3):319–342, 2002.
- [35] Tal Rabin. A Simplified Approach to Threshold and Proactive RSA. In Hugo Krawczyk, editor, *Advances in Cryptology – CRYPTO ’98*, pages 89–104, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg.
- [36] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloe Kiddon, Jakub Konečný, Stefano Mazzocchi, Brendan McMahan, et al. Towards federated learning at scale: System design. 2019.

- [37] Phan The Duy, Huynh Nhat Hao, Huynh Minh Chu, and Van-Hau Pham. A secure and privacy preserving federated learning approach for IoT intrusion detection system. In *Network and System Security*, volume 13041 of *Lecture Notes in Computer Science*, pages 353–368, Cham, 2021. Springer International Publishing.
- [38] Chuanxin Zhou, Yi Sun, and Degang Wang. Federated learning with gaussian differential privacy. In *Proceedings of the 2020 2nd International Conference on Robotics, Intelligent Control and Artificial Intelligence*, RICAI 2020, pages 163–167, New York, NY, USA, 2020. Association for Computing Machinery.
- [39] Fan Mo, Hamed Haddadi, Kleomenis Katevas, Eduard Marin, Diego Perino, and Nicolas Kourtellis. PPFL: Privacy-preserving federated learning with trusted execution environments. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services*, MobiSys '21, pages 94–108, New York, NY, USA, 2021. Association for Computing Machinery.
- [40] Yuhui Zhang, Zhiwei Wang, Jiangfeng Cao, Rui Hou, and Dan Meng. ShuffleFL: Gradient-preserving federated learning using trusted execution environment. In *Proceedings of the 18th ACM International Conference on Computing Frontiers*, CF '21, pages 161–168, New York, NY, USA, 2021. Association for Computing Machinery.
- [41] Elaine Shi, HTH Chan, Eleanor Rieffel, Richard Chow, and Dawn Song. Privacy-preserving aggregation of time-series data. In *Annual Network & Distributed System Security Symposium (NDSS)*. Internet Society., 2011.
- [42] Iraklis Leontiadis, Kaoutar Elkhayaoui, Melek Önen, and Refik Molva. Puda—privacy and unforgeability for data aggregation. In *International Conference on Cryptology and Network Security*, pages 3–18. Springer, 2015.
- [43] Marc Joye and Benoît Libert. A scalable scheme for privacy-preserving aggregation of time-series data. In *International Conference on Financial Cryptography and Data Security*, pages 111–125. Springer, 2013.
- [44] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *Advances in Cryptology — EUROCRYPT '99*, pages 223–238, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.

- [45] Cynthia Dwork. Differential privacy. In Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener, editors, *Automata, Languages and Programming, 33rd International Colloquium, ICALP 2006, Venice, Italy, July 10-14, 2006, Proceedings, Part II*, volume 4052 of *Lecture Notes in Computer Science*, pages 1–12. Springer, 2006.
- [46] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.*, 9(3–4):211–407, August 2014.
- [47] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. 9(3–4):211–407, aug 2014.
- [48] Ahmad Khalil, Aidmar Wainakh, Ephraim Zimmer, Javier Parra-Arnau, Antonio Fernandez Anta, Tobias Meuser, and Ralf Steinmetz. Label-aware aggregation for improved federated learning. In *International Conference on Fog and Mobile Edge Computing (FMEC)*, 2023.
- [49] Poojan Oza and Vishal M Patel. Federated learning-based active authentication on mobile devices. In *IEEE international joint conference on biometrics (IJCB)*, 2021.
- [50] Peng Xiao, Samuel Cheng, Vladimir Stankovic, and Dejan Vukobratovic. Averaging is probably not the optimum way of aggregating parameters in federated learning. *Entropy*, 22(3):314, 2020.
- [51] Oualid Zari, Chuan Xu, Javier Parra-Arnau, Ayşe Ünsal, and Melek Önen. Link Inference Attacks in Vertical Federated Graph Learning. In *40th Annual Computer Security Applications Conference (ACSAC)*, 2024.
- [52] Yiping Ma, Jess Woods, Sebastian Angel, Antigoni Polychroniadou, and Tal Rabin. Flamingo: Multi-round single-server secure aggregation with applications to private federated learning. In *Proc. of the IEEE Symposium on Security and Privacy (S&P)*, 2023.
- [53] Riccardo Taiello, Melek Önen, Clémentine Gritti, and Marco Lorenzi. Let them drop: Scalable and efficient federated learning solutions agnostic to client stragglers. In *International Conference on Availability, Reliability and Security (ARES)*, 2024.
- [54] Mohammad Mansouri, Melek Önen, Wafa Ben Jaballah, and Mauro Conti. SoK: secure aggregation based on cryptographic schemes for

federated learning. *Proc. on Privacy Enhancing Technologies (PETS)*, 2023.

- [55] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical Secure Aggregation for Privacy-Preserving Machine Learning. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017.
- [56] Jiawei Shao, Yuchang Sun, Songze Li, and Jun Zhang. DReS-FL: dropout-resilient secure federated learning for non-IID clients via secret data sharing. In *36th International Conference on Neural Information Processing Systems (NIPS)*, 2022.
- [57] Lin Chen, Xiaofeng Ding, Zheng Bao, Pan Zhou, and Hai Jin. Differentially Private Federated Learning on Non-iid Data: Convergence Analysis and Adaptive Optimization. *IEEE Transactions on Knowledge and Data Engineering*, 36:4567 – 4581, 2024.
- [58] Marc Joye and Benoît Libert. A scalable scheme for privacy-preserving aggregation of time-series data. In Ahmad-Reza Sadeghi, editor, *Financial Cryptography and Data Security*, pages 111–125, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- [59] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and Andre Van Schaik. Emnist: Extending mnist to handwritten letters. In *International Joint Conference on Neural Networks (IJCNN)*, 2017.
- [60] Nicolò Romandini, Alessio Mora, Carlo Mazzocca, Rebecca Montanari, and Paolo Bellavista. Federated unlearning: A survey on methods, design guidelines, and evaluation metrics. *IEEE Transactions on Neural Networks and Learning Systems*, 36(7):11697–11717, 2024.
- [61] Gaoyang Liu, Xiaoqiang Ma, Yang Yang, Chen Wang, and Jiangchuan Liu. Federaser: Enabling efficient client-level data removal from federated learning models. In *Proc. of the IEEE/ACM Int. Symposium on Quality of Service (IWQOS)*, 2021.
- [62] Wei Yuan, Hongzhi Yin, Fangzhao Wu, Shijie Zhang, Tieke He, and Hao Wang. Federated unlearning for on-device recommendation. In *Proc. of the ACM Int. Conf. on Web Search and Data Mining (WSDM)*, 2023.

- [63] Xiaoyu Cao, Jinyuan Jia, Zaixi Zhang, and Neil Zhenqiang Gong. FedRecover: Recovering from Poisoning Attacks in Federated Learning using Historical Information . In *Proc. of the IEEE Symposium on Security and Privacy (S&P)*, 2023.
- [64] Xintong Guo, Pengfei Wang, Sen Qiu, Wei Song, Qiang Zhang, Xiaopeng Wei, and Dongsheng Zhou. FAST: Adopting federated unlearning to eliminating malicious terminals at server side. *IEEE Trans. on Network Science and Engineering*, 11(2), 2024.
- [65] Anisa Halimi, Swanand Kadhe, Ambrish Rawat, and Nathalie Baracaldo. Federated unlearning: How to efficiently erase a client in FL? arXiv 2207.05521, 2023.
- [66] Guanghao Li, Li Shen, Yan Sun, Yue Hu, Han Hu, and Dacheng Tao. Subspace based Federated Unlearning. arXiv 2302.12448, 2023.
- [67] Manaar Alam, Hithem Lamri, and Michail Maniatakos. Get rid of your trail: Remotely erasing backdoors in federated learning. *IEEE Trans. on Artificial Intelligence*, 5(12), 2024.
- [68] Chen Wu, Sencun Zhu, and Prasenjit Mitra. Federated unlearning with knowledge distillation. arXiv 2201.09441, 2022.
- [69] Yian Zhao, Pengfei Wang, Heng Qi, Jianguo Huang, Zongzheng Wei, and Qiang Zhang. Federated unlearning with momentum degradation. *IEEE Internet of Things Journal*, 11(5):8860–8870, 2024.
- [70] Jinu Gong, Joonhyuk Kang, Osvaldo Simeone, and Rahif Kassab. Forget-SVGD: Particle-based bayesian federated unlearning. In *Proc. of the IEEE Data Science and Learning Workshop (DSLW)*, 2022.
- [71] Weiqi Wang, Zhiyi Tian, Chenhan Zhang, An Liu, and Shui Yu. BFU: Bayesian federated unlearning with parameter self-sharing. In *Proc. of the ACM Asia Conf. on Comp. and Comm. Sec. (AsiaCCS)*, 2023.
- [72] Ningxin Su and Baochun Li. Asynchronous federated unlearning. In *Proc. of the IEEE Conf. on Computer Communications (INFOCOM)*, 2023.
- [73] Seth Neel, Aaron Roth, and Saeed Sharifi-Malvajerdi. Descent-to-delete: Gradient-based methods for machine unlearning. In *Proc. of the 32nd Int. Conf. on Algorithmic Learning Theory, (PMLR)*, 2021.

- [74] Ayush K. Varshney and Vicenç Torra. Efficient federated unlearning under plausible deniability. *Machine Learning*, 114(1), 2025.
- [75] Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. Our data, ourselves: Privacy via distributed noise generation. In *Advances in Cryptology - EUROCRYPT*, 2006.
- [76] J. V. Linnik and I. V. Ostrovskii. Decomposition of random variables & vectors. *Translations of Mathematical Monographs*, 48(4), 1978.
- [77] Shiva Prasad Kasiviswanathan, Homin K. Lee, Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. What can we learn privately? In *Proc. of the IEEE Symposium on Foundations of Computer Science (FOCS)*, 2008.
- [78] Mark Bun and Thomas Steinke. Concentrated differential privacy: Simplifications, extensions, and lower bounds. In *Theory of Cryptography (TCC)*, 2016.
- [79] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS '16*, page 308–318, New York, NY, USA, 2016. Association for Computing Machinery.
- [80] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis (full version). *Journal of Privacy and Confidentiality*, 7(3):17–51, 2017.
- [81] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17*, page 1175–1191, New York, NY, USA, 2017. Association for Computing Machinery.
- [82] Yann Le Cunn. MNIST Database of Handwritten Digits. 10.24432/C53K8Q, 1998.
- [83] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.
- [84] Jonathan J. Hull. A database for handwritten text recognition research. *IEEE Trans. on pattern analysis and machine intelligence*, 16(5), 2002.

- [85] Yunwen Lei, Tao Sun, and Mingrui Liu. Minibatch and local sgd: Algorithmic stability and linear speedup in generalization. *arXiv* 2310.01139, 2025.
- [86] Michael Khavkin and Eran Toch. Differential privacy configurations in the real world: A comparative analysis. *IEEE Trans. on Knowledge and Data Engineering*, 37(11), 2025.
- [87] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership Inference Attacks Against Machine Learning Models. In *Proc. of the IEEE Symposium on Security and Privacy (S&P)*, 2017.
- [88] Sikha Pentya, Nicola Neophytou, Anderson Nascimento, Martine De Cock, and Golnoosh Farnadi. Privfairfl: Privacy-preserving group fairness in federated learning. *arXiv preprint arXiv:2205.11584*, 2022.
- [89] Tian Li, Maziar Sanjabi, Ahmad Beirami, and Virginia Smith. Fair resource allocation in federated learning. In *8th International Conference on Learning Representations (ICLR 2020)*, 2020.
- [90] Tian Li, Shengyuan Hu, Ahmad Beirami, and Virginia Smith. Ditto: Fair and robust federated learning through personalization. In *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*, volume 139 of *Proceedings of Machine Learning Research*, pages 6357–6368. PMLR, 2021.
- [91] Eugene Bagdasaryan, Omid Poursaeed, and Vitaly Shmatikov. Differential privacy has disparate impact on model accuracy. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, pages 15479–15488. Curran Associates, 2019.
- [92] Ferdinando Fioretto, Cuong Tran, Pascal Van Hentenryck, and Keyu Zhu. Differential privacy and fairness in decisions and learning tasks: A survey. *arXiv preprint arXiv:2202.08187*, 2022.
- [93] Seetha V. Lekshmi and Simi Sebastian. A Skewed Generalized Discrete Laplace Distribution. *International Journal of Mathematics and Statistics Invention*, 2(3):95–102, 2014.
- [94] F Maxwell Harper and Joseph A Konstan. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*, 5(4):1–19, 2015.

- [95] Aftab Akram, Harry N. H. Pham, Melek Önen, and Clémentine Gritti. SAAFL: Secure aggregation for label-aware federated learning. In *ICT Systems Security and Privacy Protection — 40th IFIP TC 11 International Conference (IFIP SEC 2025)*, IFIP Advances in Information and Communication Technology, pages 32–46. Springer, Cham, 2025.
- [96] Xiaojie Guo, Zheli Liu, Jin Li, Jiqiang Gao, Boyu Hou, Changyu Dong, and Thar Baker. Verifl: Communication-efficient and fast verifiable aggregation for federated learning. *IEEE Transactions on Information Forensics and Security*, 16:1736–1751, 2021.
- [97] Baturalp Buyukates, Jinhyun So, Hessam Mahdavifar, and Salman Avestimehr. Lightverifl: Lightweight and verifiable secure federated learning. In *Workshop on Federated Learning: Recent Advances and New Challenges (in Conjunction with NeurIPS 2022)*, 2022.
- [98] Henry Corrigan-Gibbs and Dan Boneh. Prio: Private, robust, and scalable computation of aggregate statistics. In *14th USENIX Symposium on Networked Systems Design and Implementation*, pages 259–282, 2017.
- [99] Amrita Roy Chowdhury, Chuan Guo, Somesh Jha, and Laurens van der Maaten. EIFFeL: ensuring integrity for federated learning. In *Proc. of the ACM Conf. on Comp. and Comm. Sec. (CCS)*, 2022.
- [100] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. FLtrust: Byzantine-robust Federated Learning via trust bootstrapping. *arXiv preprint arXiv:2012.13995*, 2020.
- [101] Youssef Allouah, Sadegh Farhadkhani, Rachid Guerraoui, Nirupam Gupta, Rafaël Pinot, and John Stephan. Fixing by mixing: A recipe for optimal Byzantine ML under heterogeneity. In *Int. Conf. on Artificial Intelligence and Statistics*, pages 1232–1300, 2023.
- [102] Youssef Khazbak, Tianxiang Tan, and Guohong Cao. Mlguard: Mitigating poisoning attacks in privacy preserving distributed collaborative learning. In *2020 29th international conference on computer communications and networks (ICCCN)*.
- [103] Jingwei Liu, Xinyu He, Rong Sun, Xiaojiang Du, and Mohsen Guizani. Privacy-preserving data sharing scheme with FL via MPC in financial permissioned blockchain. In *IEEE International Conf. on Communications*, pages 1–6, 2021.

- [104] Yunlong Lu, Xiaohong Huang, Yueyue Dai, Sabita Maharjan, and Yan Zhang. Blockchain and federated learning for privacy-preserved data sharing in industrial IoT. *IEEE Trans. on Industrial Informatics*, 16(6):4177–4186, 2019.
- [105] Pathum Chamikara Mahawaga Arachchige, Peter Bertok, Ibrahim Khalil, Dongxi Liu, Seyit Camtepe, and Mohammed Atiquzzaman. A trustworthy privacy preserving framework for Machine Learning in industrial IoT systems. *IEEE Trans. on Industrial Informatics*, 16(9):6092–6102, 2020.
- [106] Youssef Allouah, Rachid Guerraoui, Nirupam Gupta, Rafaël Pinot, and John Stephan. On the privacy-robustness-utility trilemma in distributed learning. In *Int. Conf. on Machine Learning*, pages 569–626, 2023.
- [107] Rachid Guerraoui, Nirupam Gupta, Rafaël Pinot, Sébastien Rouault, and John Stephan. Differential Privacy and Byzantine resilience in SGD: Do they add up? In *ACM Symposium on Principles of Distributed Computing*, pages 391–401, 2021.
- [108] Heng Zhu and Qing Ling. Bridging differential privacy and byzantine-robustness via model aggregation. *arXiv preprint arXiv:2205.00107*, 2022.
- [109] Naiyu Wang, Wenti Yang, Zhitao Guan, Xiaojiang Du, and Mohsen Guizani. BPFL: a blockchain based privacy-preserving federated learning scheme. In *IEEE Global Communications Conference*, pages 1–6, 2021.
- [110] James Bell, Adrià Gascón, Tancrede Lepoint, Baiyu Li, Sarah Meiklejohn, Mariana Raykova, and Cathie Yun. ACORN: Input validation for secure aggregation. In *32nd USENIX Security Symposium*, pages 4805–4822, 2023.
- [111] Sheikh Munir Skh Saad, Raja Zahilah Raja Mohd Radzi, and Siti Hajar Othman. Comparative analysis of the blockchain consensus algorithm between proof of stake and delegated proof of stake. In *International Conference on Data Science and Its Applications*, pages 175–180, 2021.
- [112] Newton C. Will and Carlos A. Maziero. Intel software guard extensions applications: A survey. *ACM Computing Survey*, 55, 2023.

- [113] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine Learning with adversaries: Byzantine tolerant gradient descent. *Adv. in Neural Information Processing Systems*, 30:118–128, 2017.
- [114] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *Int. Conf. on Machine Learning*, pages 5650–5659, 2018.
- [115] John Nguyen, Kshitiz Malik, Hongyuan Zhan, Ashkan Yousefpour, Mike Rabbat, Mani Malek, and Dzmitry Huba. Federated learning with buffered asynchronous aggregation. In *International Conference on Artificial Intelligence and Statistics*, pages 3581–3607, 2022.
- [116] Li Deng. The MNIST database of handwritten digit images for Machine Learning research. *IEEE signal processing magazine*, 29(6):141–142, 2012.
- [117] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. Measuring the effects of non-identical data distribution for federated visual classification. *arXiv preprint arXiv:1909.06335*, 2019.
- [118] Cong Xie, Oluwasanmi Koyejo, and Indranil Gupta. Fall of empires: Breaking byzantine-tolerant sgd by inner product manipulation. In *Uncertainty in Artificial Intelligence*, pages 261–270, 2020.
- [119] Gilad Baruch, Moran Baruch, and Yoav Goldberg. A little is enough: Circumventing defenses for distributed learning. *Advances in Neural Information Processing Systems*, 32, 2019.
- [120] Zeyuan Allen-Zhu, Faeze Ebrahimi, Jerry Li, and Dan Alistarh. Byzantine-resilient non-convex stochastic gradient descent. *arXiv preprint arXiv:2012.14368*, 2020.
- [121] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via bucketing. *arXiv preprint arXiv:2006.09365*, 2020.